

Réécriture de code récursif à l'aide de boucles

en langage Caml Light

Présentation par **Raphaël Boué-Morel (50792)**

travail réalisé avec **Mylo Fawcett (15980)**

Boucles

Exemple :

```
1  let somme_liste (l: int list) : int =  
2    let resultat = ref 0 in  
3    let l' = ref l in  
4    while !l' <> [] do  
5      resultat := !resultat + List.hd !l';  
6      l' := List.tl !l'  
7    done;  
8    !resultat
```

Boucles

Exemple :

```
1  let somme_liste (l: int list) : int =  
2    let resultat = ref 0 in  
3    let l' = ref l in  
4    while !l' <> [] do  
5      resultat := !resultat + List.hd !l';  
6      l' := List.tl !l'  
7    done;  
8    !resultat
```

Avantages :

Boucles

Exemple :

```
1  let somme_liste (l: int list) : int =  
2    let resultat = ref 0 in  
3    let l' = ref l in  
4    while !l' <> [] do  
5      resultat := !resultat + List.hd !l';  
6      l' := List.tl !l'  
7    done;  
8    !resultat
```

Avantages :

- ▶ Plus facile à compiler ou à interpréter ;
- ▶ Moins de problèmes mémoires.

Fonctions récursives

Exemple :

```
1 | let rec somme_liste (l: int list) : int =  
2 |   match l with  
3 |   | [] -> 0  
4 |   | x::q -> x + somme_liste q
```

Fonctions récursives

Exemple :

```
1 | let rec somme_liste (l: int list) : int =  
2 |   match l with  
3 |   | [] -> 0  
4 |   | x::q -> x + somme_liste q
```

- Avantage : parfois préférable en terme de lisibilité de code.

Fonctions récursives

Exemple :

```
1 | let rec somme_liste (l: int list) : int =  
2 |   match l with  
3 |   | [] -> 0  
4 |   | x::q -> x + somme_liste q
```

- ▶ Avantage : parfois préférable en terme de lisibilité de code.
- ▶ Problème : peut entraîner des dépassements de pile.

Fonctions récursives : problème

```
[raphael@arch ~]$ ocaml boucle.ml
```

```
1249999975000000
```

```
[raphael@arch ~]$ ocaml fonction_recursive.ml
```

```
Stack overflow during evaluation (looping recursion?).
```


Passage des fonctions récursives aux boucles

Passage des fonctions récursives aux boucles

- ▶ Étape classique de l'optimisation lors de la compilation de langages fonctionnels

Passage des fonctions récursives aux boucles

- ▶ Étape classique de l'optimisation lors de la compilation de langages fonctionnels
- ▶ Comment ça marche en pratique ?

Passage des fonctions récursives aux boucles

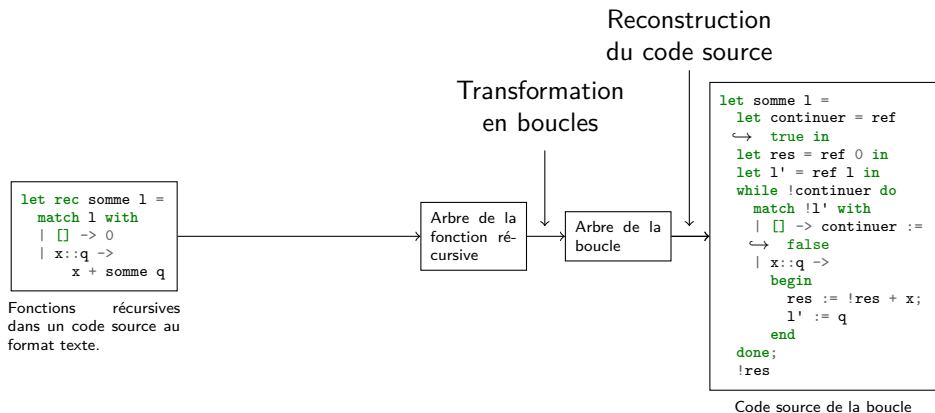
```
let rec somme l =
  match l with
  | [] -> 0
  | x::q ->
    x + somme q
```

Fonctions récursives
dans un code source au
format texte.

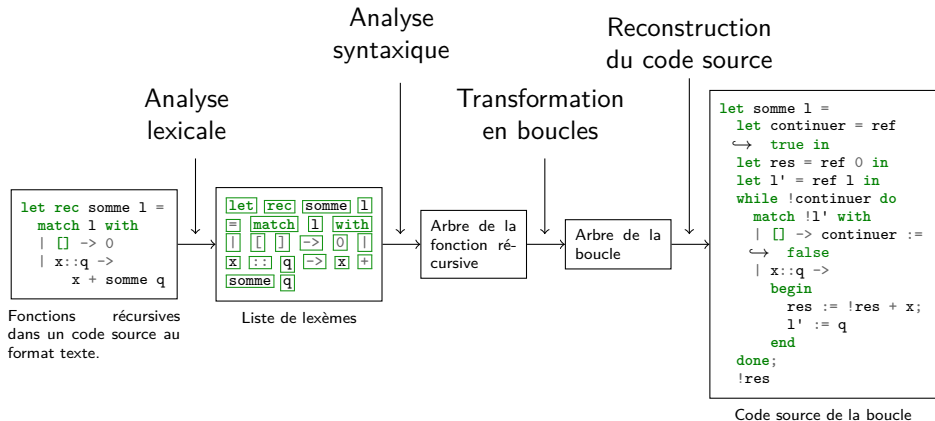
```
let somme l =
  let continuer = ref
  ↪ true in
  let res = ref 0 in
  let l' = ref l in
  while !continuer do
    match !l' with
    | [] -> continuer :=
    ↪ false
    | x::q ->
      begin
        res := !res + x;
        l' := q
      end
  done;
  !res
```

Code source de la boucle

Passage des fonctions récursives aux boucles



Passage des fonctions récursives aux boucles



Plan

Analyse lexicale

```
let rec somme 1 =
  match 1 with
  | [] -> 0
  | x::q ->
    x + somme q
```

Fonctions récursives
dans un code source au
format texte.

```
let rec somme 1
= match 1 with
| [] -> 0 |
x::q -> x +
somme q
```

Liste de lexèmes

Analyse syntaxique

Transformation en boucles

Arbre de la
fonction ré-
cursive

Arbre de la
boucle

Reconstruction du code source

```
let somme 1 =
  let continuer = ref
  ↪ true in
  let res = ref 0 in
  let l' = ref 1 in
  while !continuer do
    match !l' with
    | [] -> continuer :=
  ↪ false
    | x::q ->
      begin
        res := !res + x;
        l' := q
      end
  done;
  !res
```

Code source de la boucle

Analyse lexicale

But : construire une liste de lexèmes à partir du code source.

Analyse lexicale

But : construire une liste de lexèmes à partir du code source.

Exemple :

```
let plus_un x = x+1
```

Analyse lexicale

But : construire une liste de lexèmes à partir du code source.

Exemple :

```
let plus_un x = x+1
```

l	e	t		p	l	u	s	_	u	n		x		=		x	+	1
---	---	---	--	---	---	---	---	---	---	---	--	---	--	---	--	---	---	---

Analyse lexicale

But : construire une liste de lexèmes à partir du code source.

Exemple :

```
let plus_un x = x+1
```

l	e	t		p	l	u	s	_	u	n		x		=		x	+	1
---	---	---	--	---	---	---	---	---	---	---	--	---	--	---	--	---	---	---

let	plus_un	x	=	x	+	1
-----	---------	---	---	---	---	---

Analyse lexicale

But : construire une liste de lexèmes à partir du code source.

Exemple :

```
let plus_un x = x+1
```

l	e	t		p	l	u	s	_	u	n		x		=		x	+	1
---	---	---	--	---	---	---	---	---	---	---	--	---	--	---	--	---	---	---

let	plus_un	x	=	x	+	1
-----	---------	---	---	---	---	---

↓
let

Analyse lexicale

But : construire une liste de lexèmes à partir du code source.

Exemple :

```
let plus_un x = x+1
```

l	e	t		p	l	u	s	_	u	n		x		=		x	+	1
---	---	---	--	---	---	---	---	---	---	---	--	---	--	---	--	---	---	---

let	plus_un	x	=	x	+	1
-----	---------	---	---	---	---	---

↓
let

↓
identifiant

Analyse lexicale

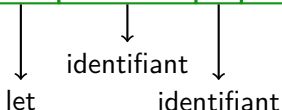
But : construire une liste de lexèmes à partir du code source.

Exemple :

```
let plus_un x = x+1
```

l	e	t		p	l	u	s	_	u	n		x		=		x	+	1
---	---	---	--	---	---	---	---	---	---	---	--	---	--	---	--	---	---	---

let	plus_un	x	=	x	+	1
-----	---------	---	---	---	---	---



Analyse lexicale

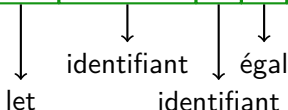
But : construire une liste de lexèmes à partir du code source.

Exemple :

```
let plus_un x = x+1
```

l	e	t		p	l	u	s	_	u	n		x		=		x	+	1
---	---	---	--	---	---	---	---	---	---	---	--	---	--	---	--	---	---	---

let	plus_un	x	=	x	+	1
-----	---------	---	---	---	---	---



Analyse lexicale

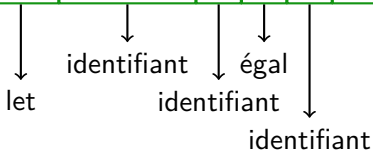
But : construire une liste de lexèmes à partir du code source.

Exemple :

```
let plus_un x = x+1
```

l	e	t		p	l	u	s	_	u	n		x		=		x	+	1
---	---	---	--	---	---	---	---	---	---	---	--	---	--	---	--	---	---	---

let	plus_un	x	=	x	+	1
-----	---------	---	---	---	---	---



Analyse lexicale

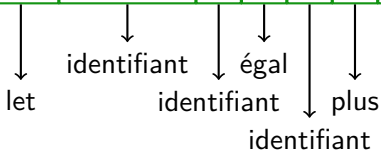
But : construire une liste de lexèmes à partir du code source.

Exemple :

```
let plus_un x = x+1
```

l	e	t		p	l	u	s	_	u	n		x		=		x	+	1
---	---	---	--	---	---	---	---	---	---	---	--	---	--	---	--	---	---	---

let	plus_un	x	=	x	+	1
-----	---------	---	---	---	---	---



Analyse lexicale

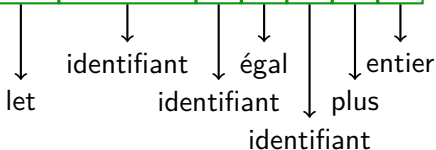
But : construire une liste de lexèmes à partir du code source.

Exemple :

```
let plus_un x = x+1
```

l	e	t		p	l	u	s	_	u	n		x		=		x	+	1
---	---	---	--	---	---	---	---	---	---	---	--	---	--	---	--	---	---	---

let	plus_un	x	=	x	+	1
-----	---------	---	---	---	---	---



Analyse lexicale

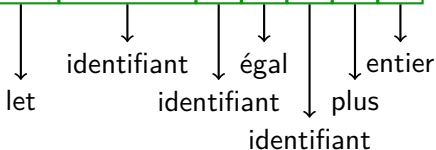
But : construire une liste de lexèmes à partir du code source.

Exemple :

```
let plus_un x = x+1
```

l	e	t		p	l	u	s	_	u	n		x		=		x	+	1
---	---	---	--	---	---	---	---	---	---	---	--	---	--	---	--	---	---	---

let	plus_un	x	=	x	+	1
-----	---------	---	---	---	---	---

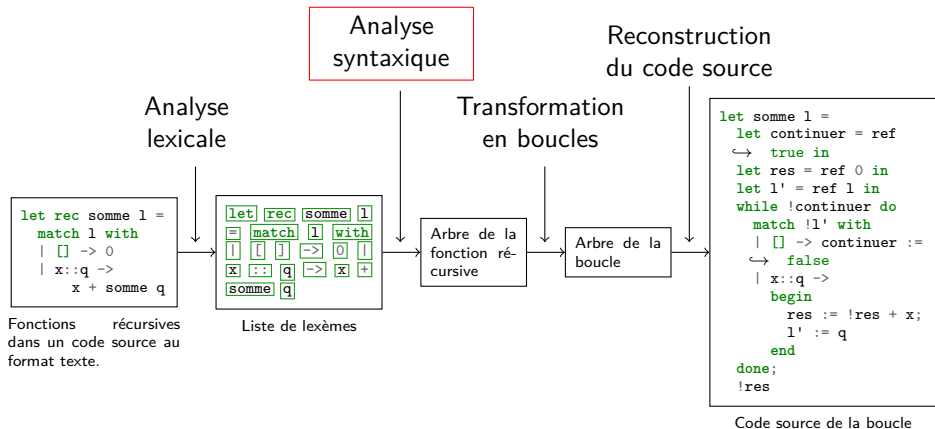


```

1 | type tok_type = | Identifiant | Entier | Plus | ...
2 | type token = { typ : tok_type; value : string;
   |   ↪ start : int; length : int }

```

Plan



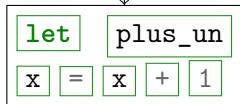
Analyse syntaxique

But : construire un arbre de syntaxe à partir de la liste de lexèmes.

Analyse syntaxique

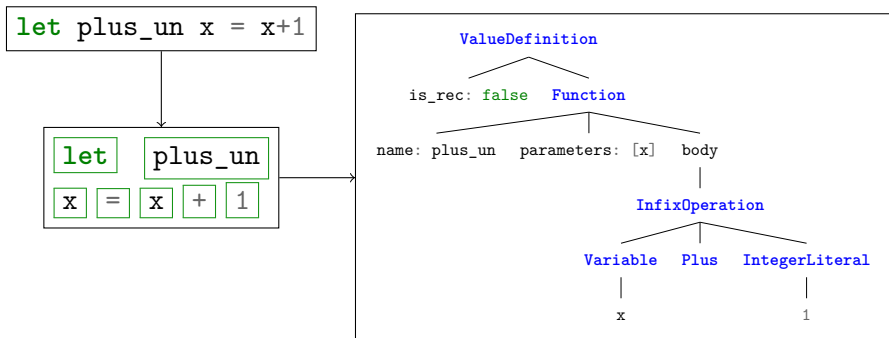
But : construire un arbre de syntaxe à partir de la liste de lexèmes.

```
let plus_un x = x+1
```



Analyse syntaxique

But : construire un arbre de syntaxe à partir de la liste de lexèmes.



Grammaires

Pour décrire les « règles de construction » dans nos arbres, on introduit la notion de grammaire.

Grammaires

$x = (3 \times (1 + 2))$

Grammaires

$$\underbrace{x = (3 \times (1 + 2))}_{\text{PHRASE}}$$

PHRASE $\rightarrow$ VARIABLE égal EXPR

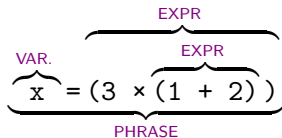
Grammaires

$$\underbrace{\overbrace{x}^{\text{VAR.}} = (3 \times (1 + 2))}_{\text{PHRASE}}$$

PHRASE → VARIABLE égal EXPR

VARIABLE → identifiant

Grammaires



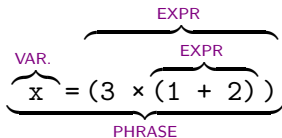
PHRASE $\rightarrow$ VARIABLE égal EXPR

VARIABLE $\rightarrow$ identifiant

EXPR $\rightarrow$ littéral_d'entier

EXPR $\rightarrow$ parenthèse_g EXPR OPÉRATEUR EXPR parenthèse_d

Grammaires



PHRASE → VARIABLE égal EXPR

VARIABLE → identifiant

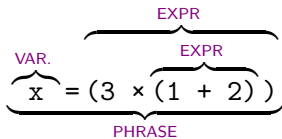
EXPR → littéral_d'entier

EXPR → parenthèse_g EXPR OPÉRATEUR EXPR parenthèse_d

OPÉRATEUR → plus

OPÉRATEUR → fois

Grammaires



$\underline{P} \rightarrow V \text{ égal } E$

$V \rightarrow \text{id}$

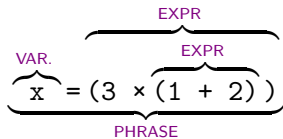
$E \rightarrow \text{ent}$

$E \rightarrow \text{parg } E \text{ O } E \text{ pard}$

$O \rightarrow \text{plus}$

$O \rightarrow \text{fois}$

Grammaires



$\underline{P} \rightarrow V \text{ égal } E$

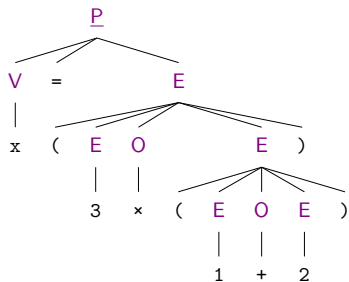
$\underline{V} \rightarrow \text{id}$

$\underline{E} \rightarrow \text{ent}$

$\underline{E} \rightarrow \text{parg } E \ O \ E \ \text{pard}$

$\underline{O} \rightarrow \text{plus}$

$\underline{O} \rightarrow \text{fois}$



Problème : comment passer d'une grammaire et d'un code source à un arbre de syntaxe ?

Automates

Automates

L'algorithme LR(0) repose sur l'utilisation **d'automates**.

Automates

L'algorithme LR(0) repose sur l'utilisation **d'automates**.

Définition

***États** LR(0) = ensemble de règles de grammaires munies de positions.*
***Transitions** étiquetées par des symboles.*

Automates

L'algorithme LR(0) repose sur l'utilisation **d'automates**.

Définition

États LR(0) = ensemble de règles de grammaires munies de positions.
Transitions étiquetées par des symboles.

L'automate LR(0) correspondant à une grammaire peut être construit à partir de cette grammaire.

Automates

P → V égal E

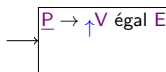
V → id

E → ent

E → parg E O E pard

O → plus

O → fois



Automates

P → V égal E

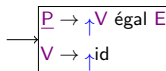
V → id

E → ent

E → parg E O E pard

O → plus

O → fois



Automates

$\underline{P} \rightarrow V \text{ égal } E$

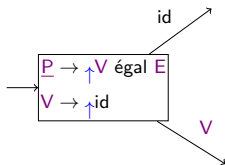
$V \rightarrow \text{id}$

$E \rightarrow \text{ent}$

$E \rightarrow \text{parg } E \text{ O } E \text{ pard}$

$O \rightarrow \text{plus}$

$O \rightarrow \text{fois}$



Automates

$\underline{P} \rightarrow V \text{ égal } E$

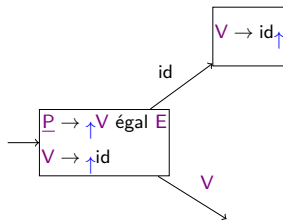
$V \rightarrow id$

$E \rightarrow ent$

$E \rightarrow \text{parg } E \text{ O } E \text{ pard}$

$O \rightarrow \text{plus}$

$O \rightarrow \text{fois}$



Automates

$\underline{P} \rightarrow V \text{ égal } E$

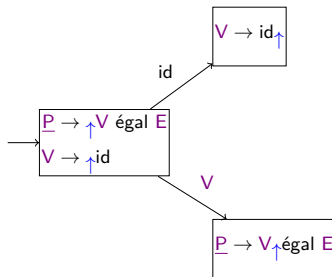
$V \rightarrow \text{id}$

$E \rightarrow \text{ent}$

$E \rightarrow \text{parg } E \text{ O } E \text{ pard}$

$O \rightarrow \text{plus}$

$O \rightarrow \text{fois}$



Automates

$\underline{P} \rightarrow V \text{ égal } E$

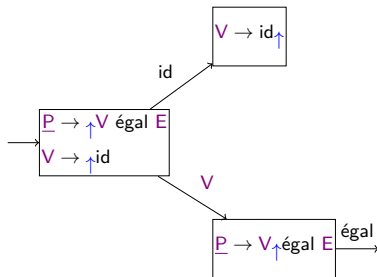
$V \rightarrow id$

$E \rightarrow ent$

$E \rightarrow \text{parg } E \text{ } O \text{ } E \text{ pard}$

$O \rightarrow plus$

$O \rightarrow fois$



Automates

$\underline{P} \rightarrow V \text{ égal } E$

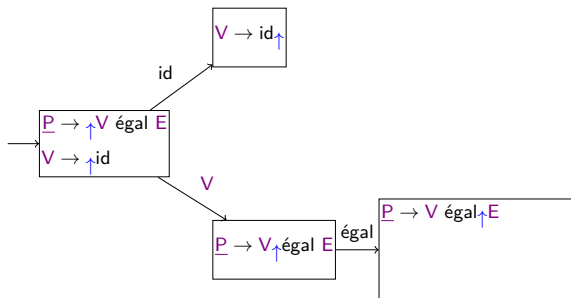
$V \rightarrow \text{id}$

$E \rightarrow \text{ent}$

$E \rightarrow \text{parg } E \text{ O } E \text{ pard}$

$O \rightarrow \text{plus}$

$O \rightarrow \text{fois}$



Automates

$\underline{P} \rightarrow V \text{ égal } E$

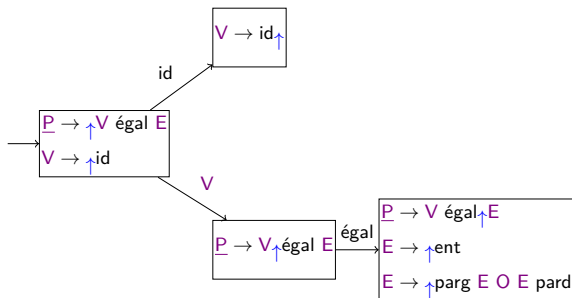
$V \rightarrow \text{id}$

$E \rightarrow \text{ent}$

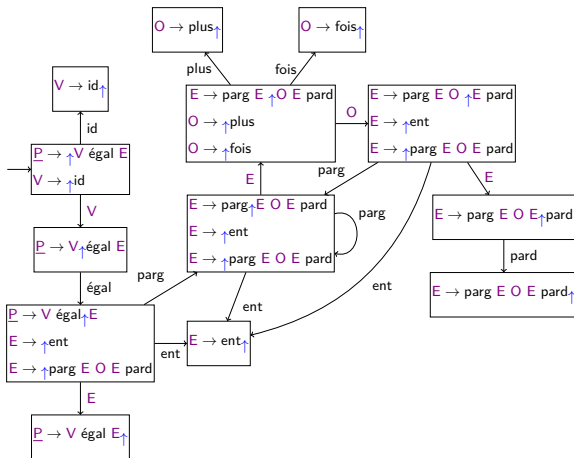
$E \rightarrow \text{parg } E \text{ O } E \text{ pard}$

$O \rightarrow \text{plus}$

$O \rightarrow \text{fois}$

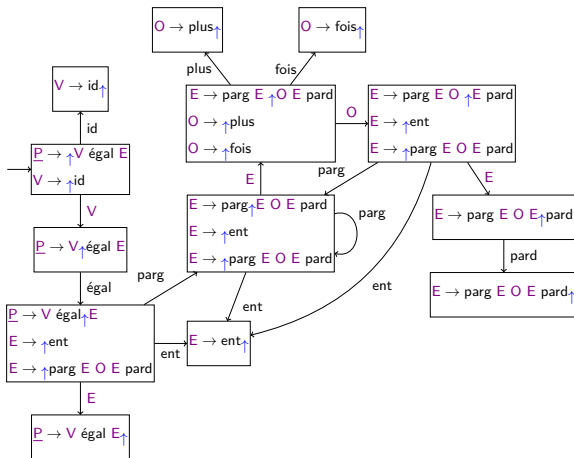


$\underline{P} \rightarrow V \text{ égal } E$
 $V \rightarrow \text{id}$
 $E \rightarrow \text{ent}$
 $E \rightarrow \text{parg } E \text{ O } E \text{ pard}$
 $O \rightarrow \text{plus}$
 $O \rightarrow \text{fois}$



Algorithme LR(0) sur un exemple

Exemple : $x = (3 \times (1 + 2))$



$\underline{P} \rightarrow V \text{ égal } E$

$V \rightarrow id$

$E \rightarrow ent$

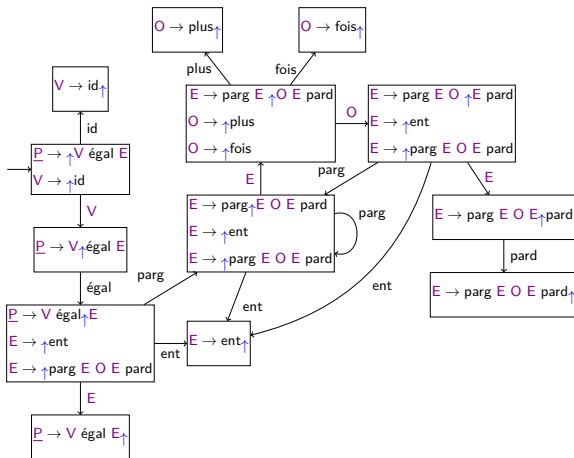
$E \rightarrow \text{par } E \text{ O } E \text{ pard}$

$O \rightarrow plus$

$O \rightarrow fois$

Algorithme LR(0) sur un exemple

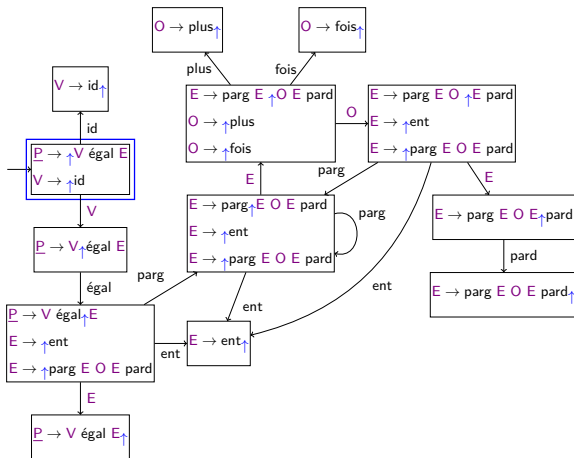
Exemple : $x = (3 \times (1 + 2))$



Algorithme LR(0) sur un exemple

Exemple : $x = (3 \times (1 + 2))$

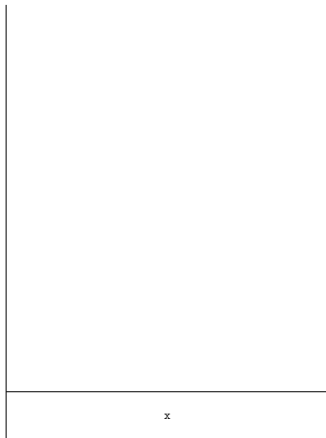
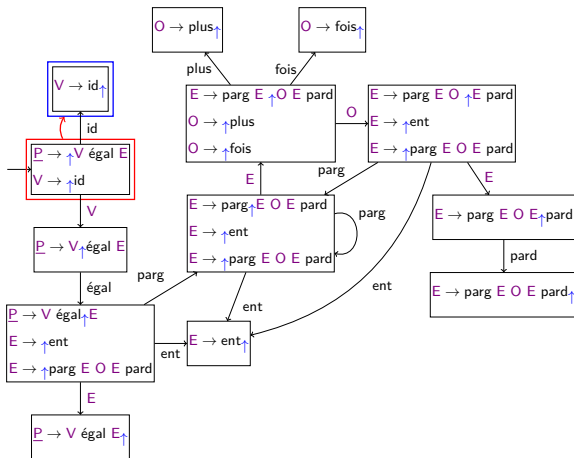
↑ $x = (3 \times (1 + 2))$



Algorithme LR(0) sur un exemple

Exemple : $x = (3 \times (1 + 2))$

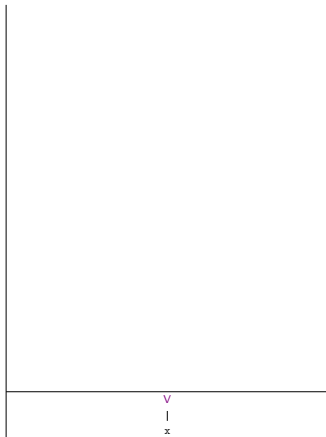
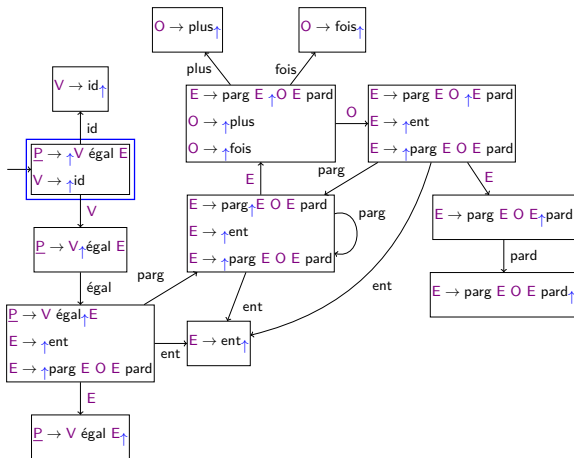
$x = (3 \times (1 + 2))$



Algorithme LR(0) sur un exemple

Exemple : $x = (3 \times (1 + 2))$

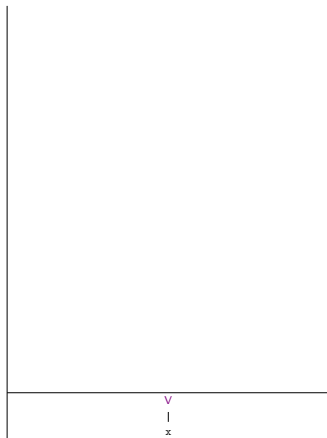
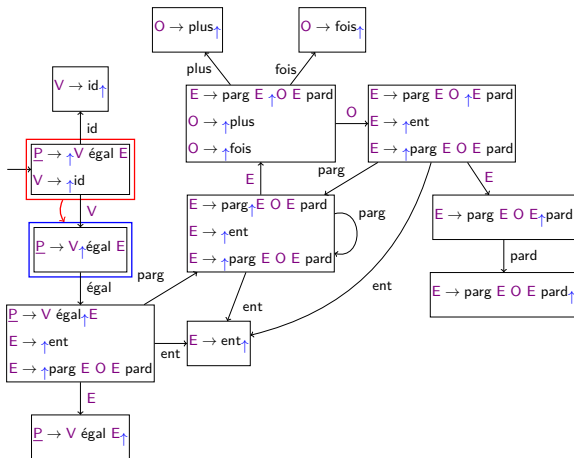
$\uparrow V = (3 \times (1 + 2))$



Algorithme LR(0) sur un exemple

Exemple : $x = (3 \times (1 + 2))$

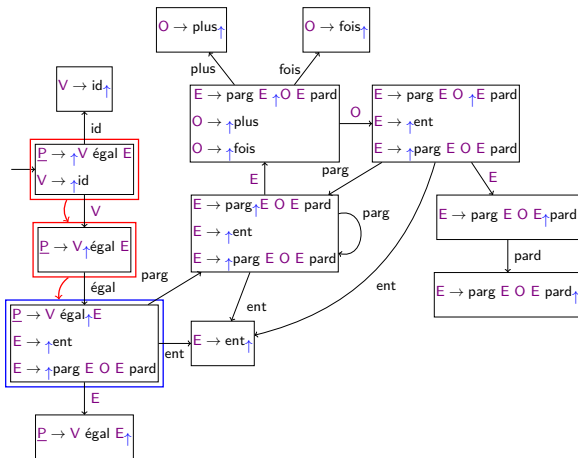
$V \uparrow = (3 \times (1 + 2))$



Algorithme LR(0) sur un exemple

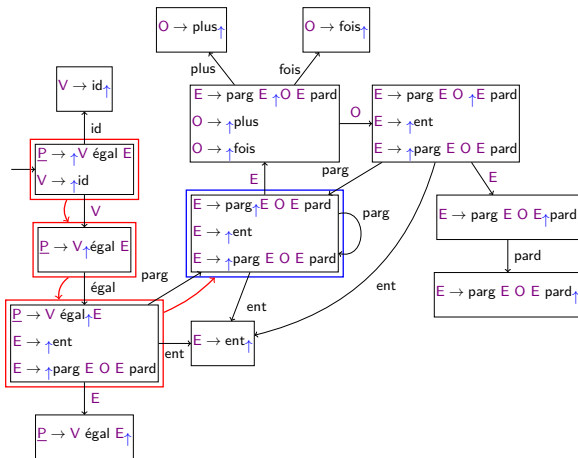
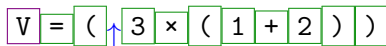
Exemple : $x = (3 \times (1 + 2))$

V = (3 × (1 + 2))



=
V
x

Exemple : $x = (3 \times (1 + 2))$

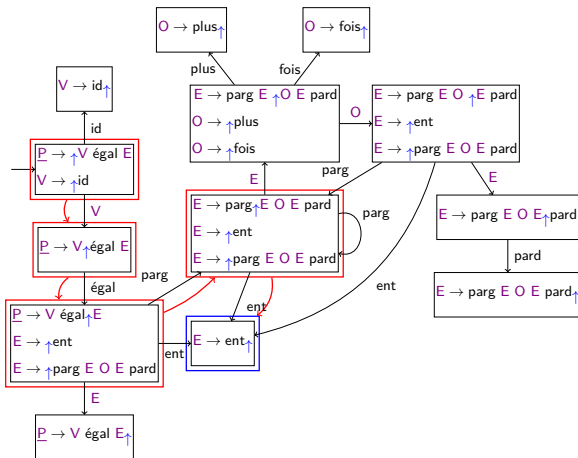


(
=
V x

Algorithme LR(0) sur un exemple

Exemple : $x = (3 \times (1 + 2))$

V = (3 ↑ × (1 + 2))

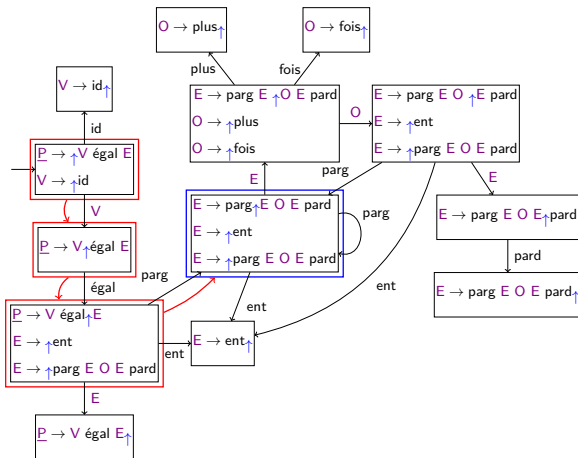


3
(
=
V
x

Algorithme LR(0) sur un exemple

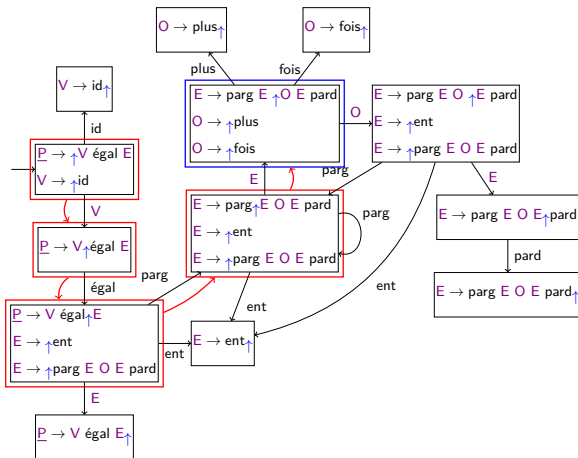
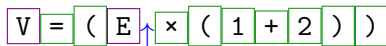
Exemple : $x = (3 \times (1 + 2))$

V = ($\uparrow$ E $\times$ (1 + 2))



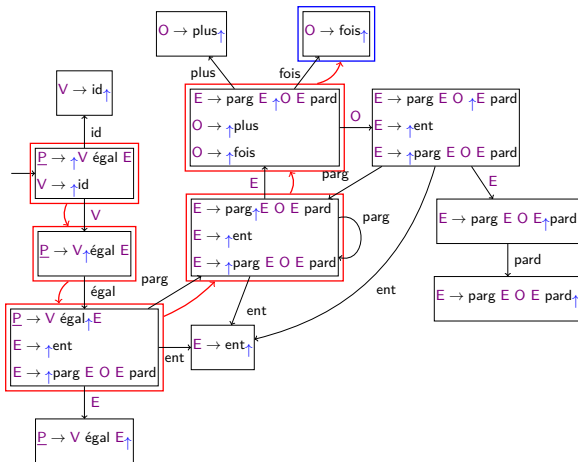
E
3
(
=
V
x

Exemple : $x = (3 \times (1 + 2))$



	E
	3
	(
	=
	V
	x

Exemple : $x = (3 \times (1 + 2))$

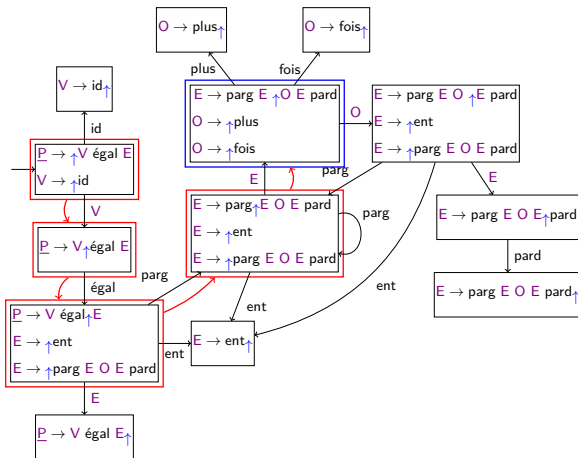


x
E 3
(
=
V x

Algorithme LR(0) sur un exemple

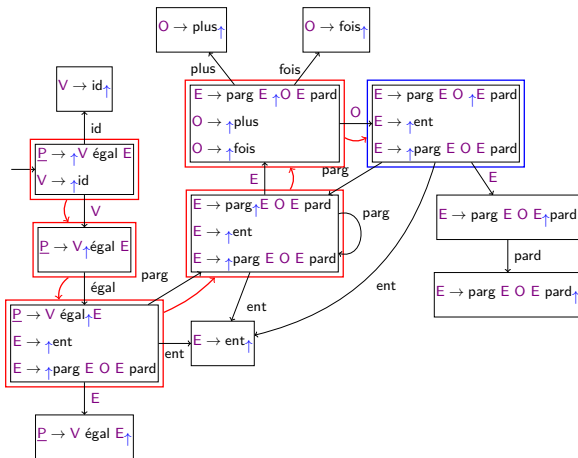
Exemple : $x = (3 \times (1 + 2))$

V = (E 0 (1 + 2))



O
x
E
3
(
=
V
x

Exemple : $x = (3 \times (1 + 2))$

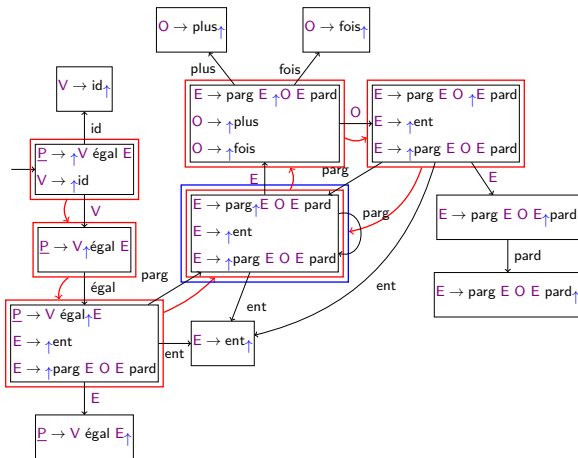


	O
	x
E	
	3
(	
=	
V	
	x

Algorithme LR(0) sur un exemple

Exemple : $x = (3 \times (1 + 2))$

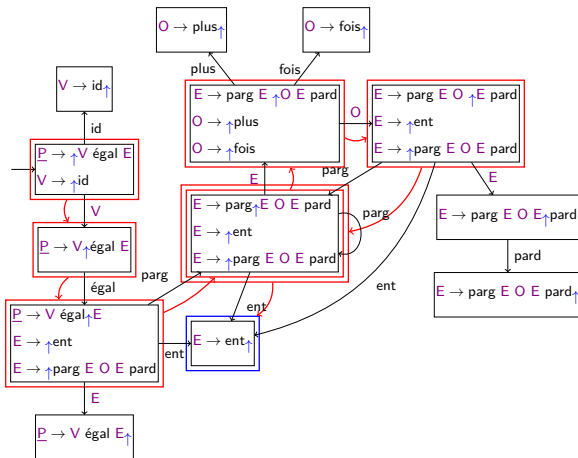
V = (E O (1 + 2))



(
O
x
E
3
(
=
V
x

Exemple : $x = (3 \times (1 + 2))$

V = (E 0 (1 + 2))

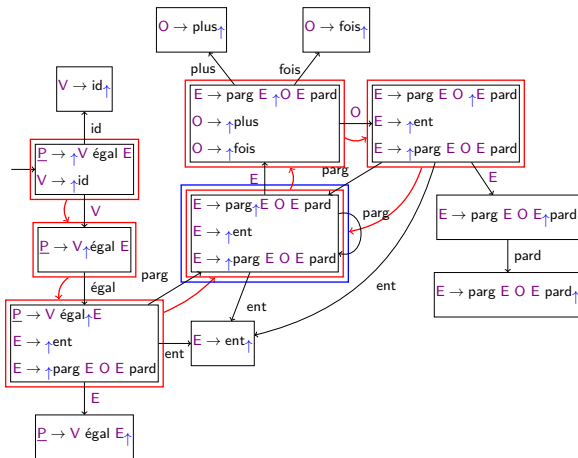


1
(
O x
E 3
(
=
V x

Algorithme LR(0) sur un exemple

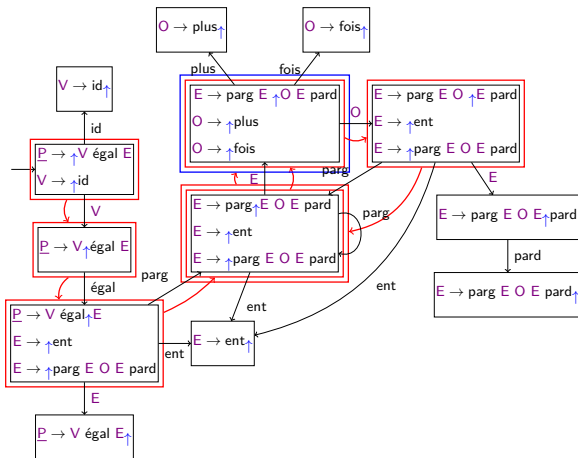
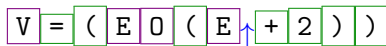
Exemple : $x = (3 \times (1 + 2))$

V = (E O (↑ E + 2)))



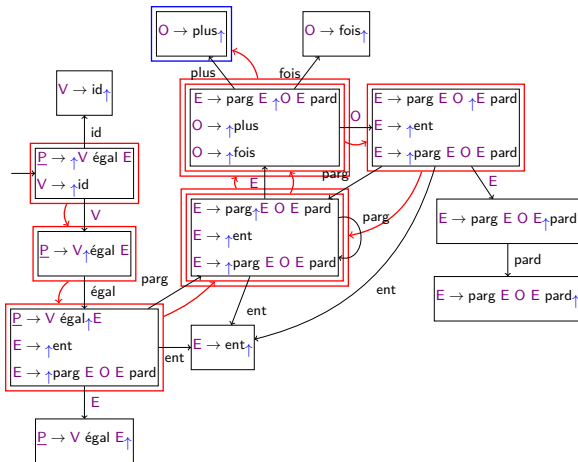
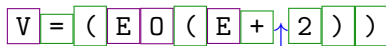
E
1
(
O
x
E
3
(
=
V
x

Exemple : $x = (3 \times (1 + 2))$



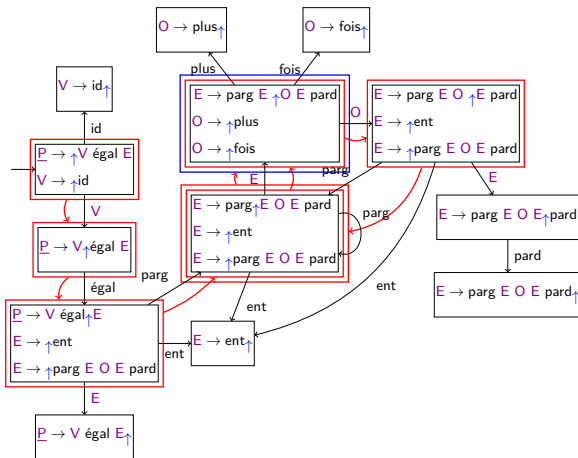
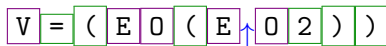
	E
	1
	(
	O
	x
	E
	3
	(
	=
	V
	x

Exemple : $x = (3 \times (1 + 2))$



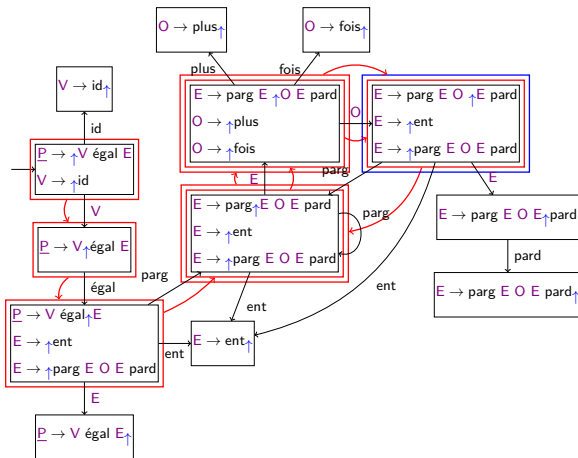
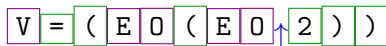
+
E 1
(
O x
E 3
(
=
V x

Exemple : $x = (3 \times (1 + 2))$



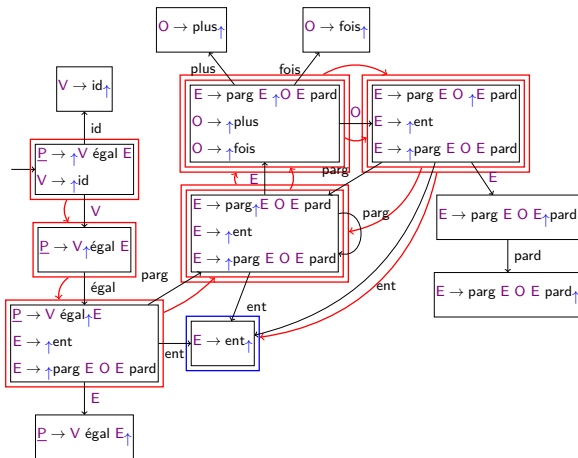
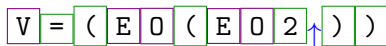
	O +
	E 1
	(
	O x
	E 3
	(
	=
	V x

Exemple : $x = (3 \times (1 + 2))$



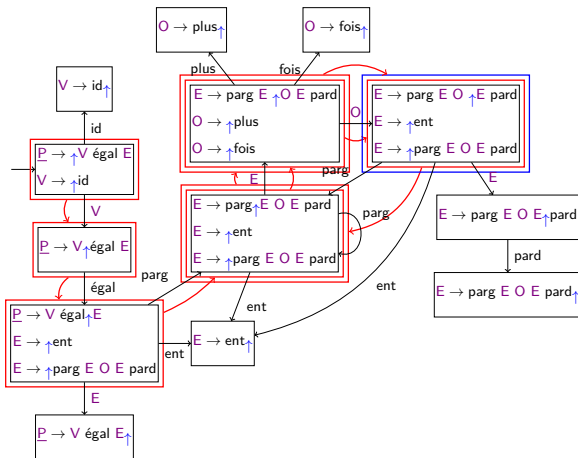
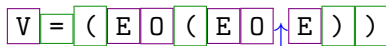
	O +
	E 1
	(
	O x
	E 3
	(
	=
	V x

Exemple : $x = (3 \times (1 + 2))$



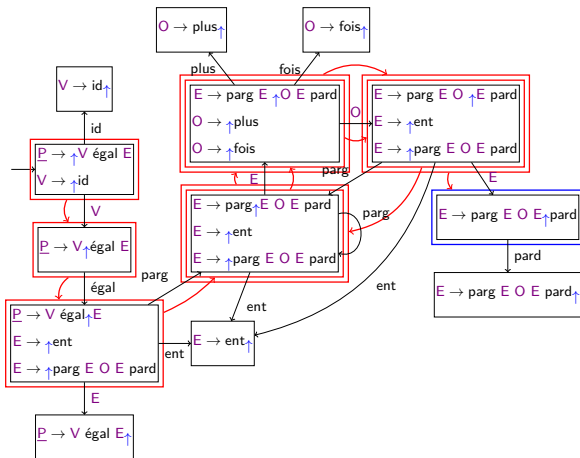
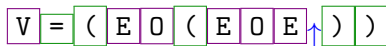
2
O +
E 1
(
O x
E 3
(
=
V x

Exemple : $x = (3 \times (1 + 2))$



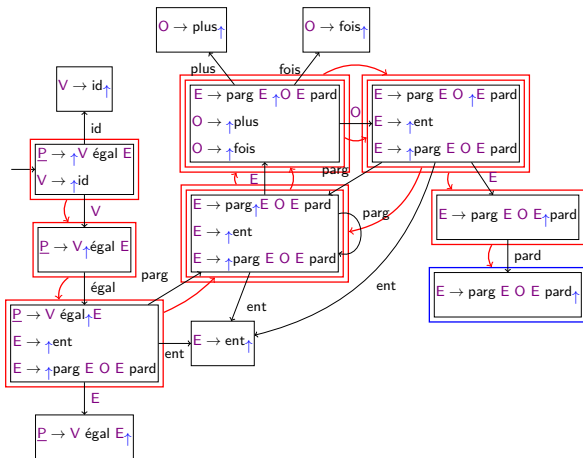
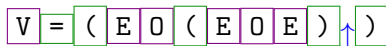
	E
	2
	O
	+
	E
	1
	(
	O
	x
	E
	3
	(
	=
	V
	x

Exemple : $x = (3 \times (1 + 2))$



	E 2
	O +
	E 1
	(
	O x
	E 3
	(
	=
	V x

Exemple : $x = (3 \times (1 + 2))$

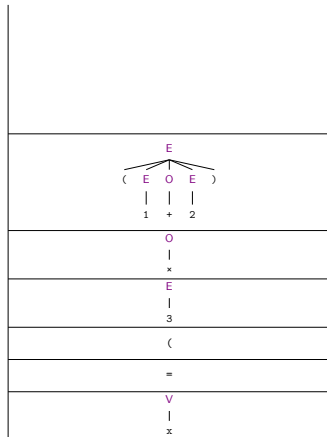
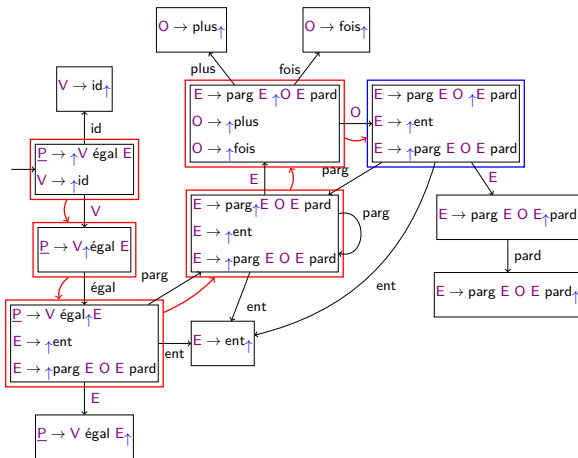


	)
	E 2
	O +
	E 1
	(
	O x
	E 3
	(
	=
	V x

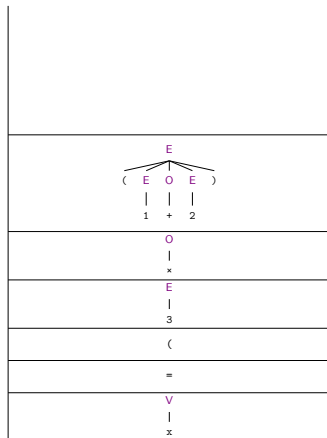
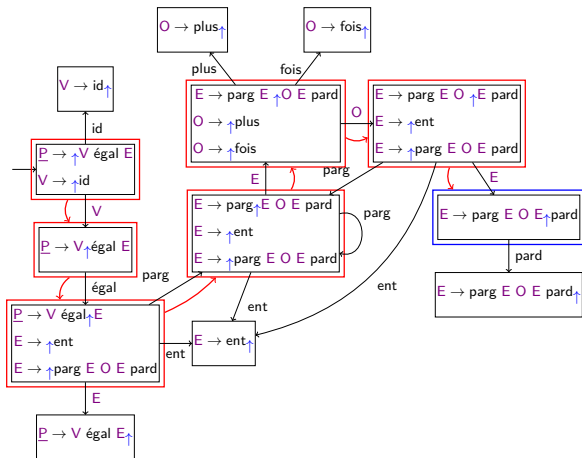
Algorithme LR(0) sur un exemple

Exemple : $x = (3 \times (1 + 2))$

$V = (E O \uparrow E)$



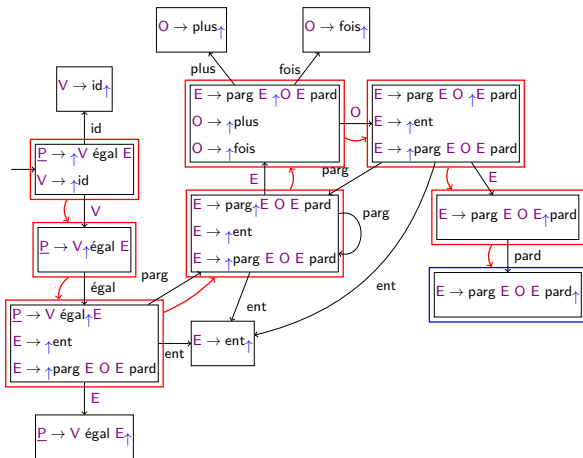
Exemple : $x = (3 \times (1 + 2))$

$$V = (EOE \uparrow)$$


Algorithme LR(0) sur un exemple

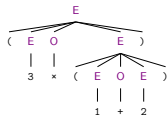
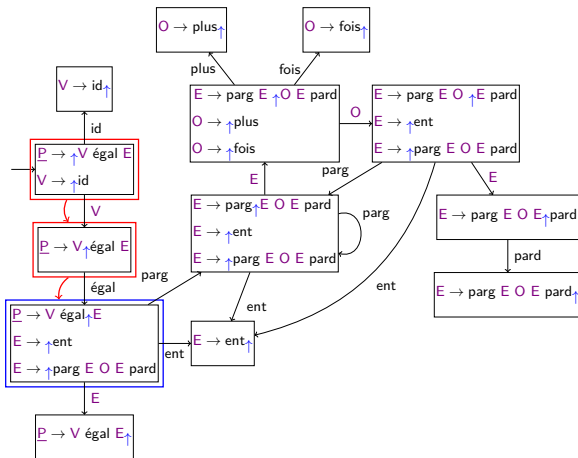
Exemple : $x = (3 \times (1 + 2))$

$V = (E O E) \uparrow$



)
$ \begin{array}{c} E \\ \swarrow \quad \downarrow \quad \searrow \\ (\quad E \quad O \quad E \quad) \\ \quad \quad \\ 1 \quad + \quad 2 \end{array} $
O
x
E
3
(
=
V
x

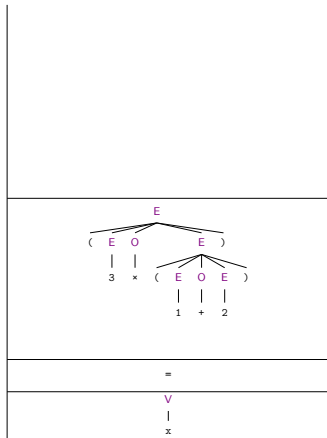
Exemple : $x = (3 \times (1 + 2))$



==

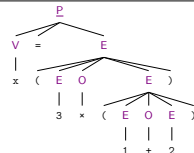
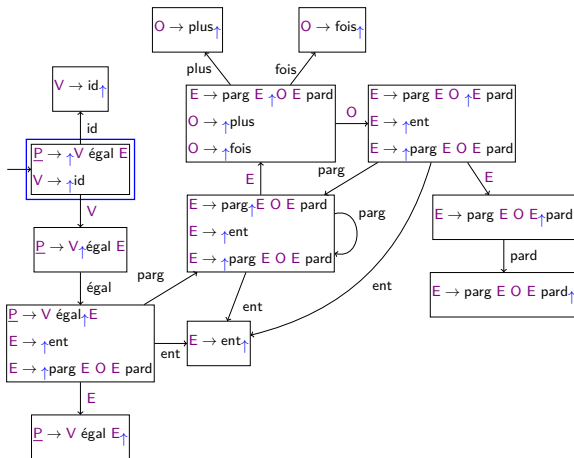
v

Exemple : $x = (3 \times (1 + 2))$ $V = E$

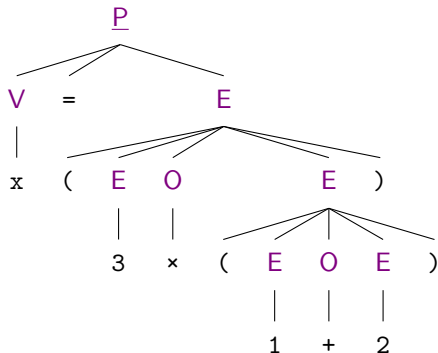
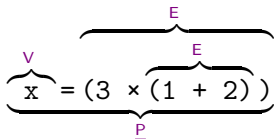


Algorithme LR(0) sur un exemple

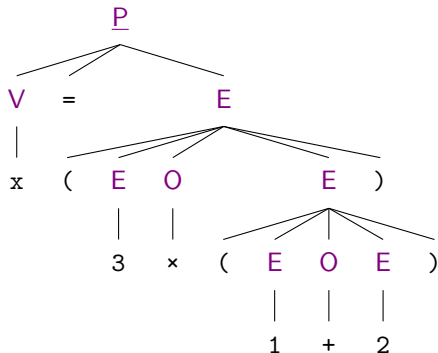
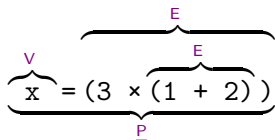
Exemple : $x = (3 \times (1 + 2))$ $\uparrow \underline{P}$



Algorithme LR(0) sur un exemple

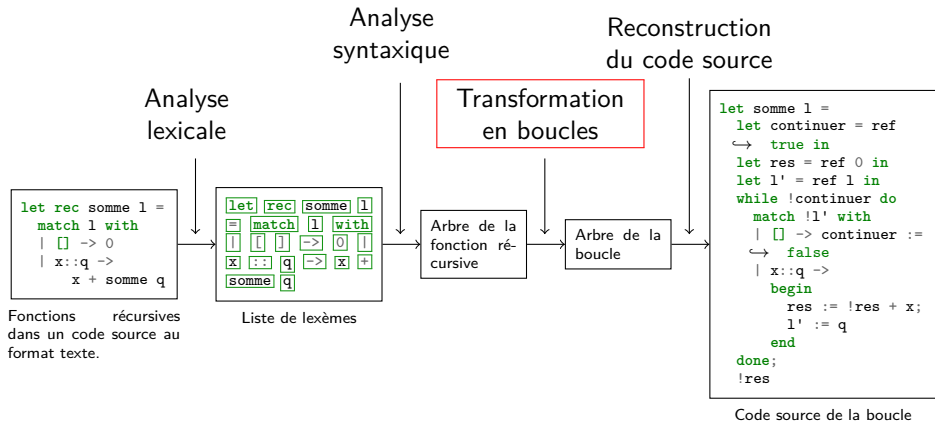


Algorithme LR(0) sur un exemple



En pratique, LR(0) a quelques limitations et on utilise sa variante LR(1).

Plan



Mise sous forme récursive terminale

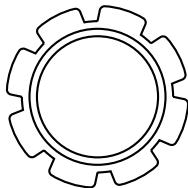
```
1 | let rec somme l =  
2 |   match l with  
3 |   | [] -> 0  
4 |   | x::q ->  
5 |     x + somme q
```

```
1 | let rec somme_rt l a =  
2 |   match l with  
3 |   | [] -> a  
4 |   | x::q ->  
5 |     somme_rt q (a+x)
```

Mise sous forme récursive terminale

```
1  let rec somme l =  
2    match l with  
3    | [] -> 0  
4    | x::q ->  
5      x + somme q
```

```
1  let rec somme_rt l a =  
2    match l with  
3    | [] -> a  
4    | x::q ->  
5      somme_rt q (a+x)
```



Algorithme de
passage sous forme
récursive terminale

Remplacement des appels récursifs

```
1  let rec somme_rt l a =  
2  
3    match l with  
4    | [] -> a  
5    | x::q ->  
6      somme_rt q (a+x)
```

Remplacement des appels récursifs

```
1 let rec somme_rt l a =  
2     ←  
3     match l with  
4     | [] -> a  
5     | x::q ->  
6         somme_rt q (a+x)
```

```
1 let c = ref true in  
2 let res = ref None in  
3 let l' = ref l in  
4 let a' = ref a in
```

Remplacement des appels récursifs

```
1  let rec somme_rt l a =  
2    let c = ref true in  
3    let res = ref None in  
4    let l' = l in  
5    let a' = a in  
6    match l with  
7    | [] -> a  
8    | x::q ->  
9      somme_rt q (a+x)
```

Remplacement des appels récursifs

```
1 let rec somme_rt l a =  
2   let c = ref true in  
3   let res = ref None in  
4   let l' = l in  
5   let a' = a in  
6   match l with  
7   | [] -> a  
8   | x::q ->  
9     somme_rt q (a+x)
```

Remplacement des appels récursifs

```
1 let rec somme_rt l a =  
2   let c = ref true in  
3   let res = ref None in  
4   let l' = l in  
5   let a' = a in  
6   match l with  
7   | [] -> a  
8   | x::q ->  
9     somme_rt q (a+x)
```

Diagram illustrating the replacement of recursive calls with a loop:

- Line 7: `| [] -> a` (The variable `a` is circled in blue).
- Line 8: `| x::q ->`
- Line 9: `somme_rt q (a+x)` (The entire recursive call expression is circled in red).

Annotations on the right side of the code:

- Line 1: `c := false;`
- Line 2: `res := Some !a'`

An arrow points from the circled `a` in line 7 to the `c := false;` annotation in line 1.

Remplacement des appels récursifs

```
1  let rec somme_rt l a =  
2    let c = ref true in  
3    let res = ref None in  
4    let l' = l in  
5    let a' = a in  
6    match l with  
7      | [] -> a  
8      | x::q ->  
9        somme_rt q (a+x)
```

1		c := false;
2		res := Some !a'
1		l' := q;
2		a' := !a'+x

Remplacement des appels récursifs

```
1 let rec somme_rt l a =  
2   let c = ref true in  
3   let res = ref None in  
4   let l' = l in  
5   let a' = a in  
6   match l with  
7   | [] -> a  
8   | x::q ->  
9     somme_rt q (a+x)
```

Diagram illustrating the replacement of recursive calls with loop logic:

1		c := false;
2		res := Some !a'
1		l' := q;
2		a' := !a'+x

Annotations: The variable `l` in the `match` expression is circled in orange. The variable `a` in the `match` expression is circled in blue. The recursive call `somme_rt q (a+x)` is circled in red. Arrows indicate the flow of data from the recursive call to the loop logic.

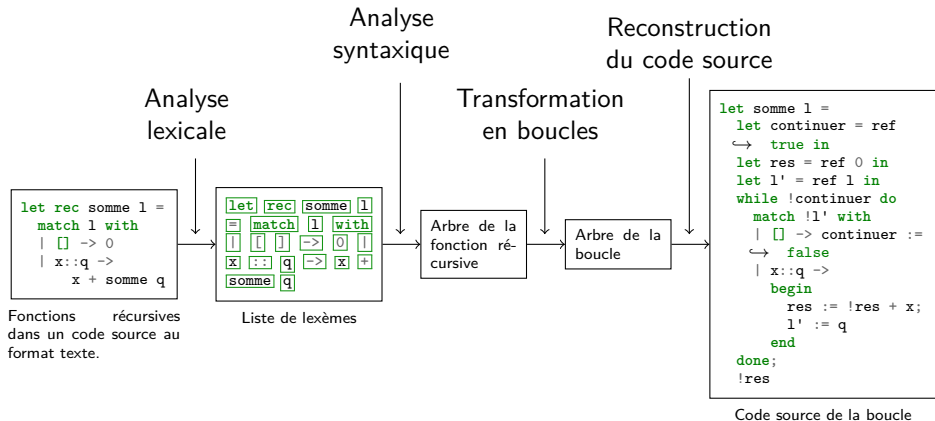
Remplacement des appels récursifs

```
1  let somme_rt l a =  
2    let c = ref true in  
3    let res = ref None in  
4    let l' = ref l in  
5    let a' = ref a in  
6    match !l' with  
7      | [] ->  
8        begin  
9          c := false;  
10         res := Some a'  
11        end  
12      | x::q ->  
13        begin  
14          l' := q;  
15          a' := !a'+x  
16        end
```


Remplacement des appels récursifs

```
1  let somme_rt l a =  
2    let c = ref true in  
3    let res = ref None in  
4    let l' = ref l in  
5    let a' = ref a in  
6    while !c do  
7      match !l' with  
8      | [] ->  
9        begin  
10         c := false;  
11         res := Some a'  
12       end  
13     | x::q ->  
14       begin  
15         l' := q;  
16         a' := !a'+x  
17       end  
18   done;  
19   Option.get !res
```

Plan



Résultats

Temps d'exécution sur la fonction récursive somme :

Résultats

Temps d'exécution sur la fonction récursive `somme` :

- ▶ Analyse lexicale : 0,0803 s ;

Résultats

Temps d'exécution sur la fonction récursive `somme` :

- ▶ Analyse lexicale : 0,0803 s ;
- ▶ Construction de l'automate LR(1) : 4,6386 s ;

Résultats

Temps d'exécution sur la fonction récursive `somme` :

- ▶ Analyse lexicale : 0,0803 s ;
- ▶ Construction de l'automate LR(1) : 4,6386 s ;
- ▶ Analyse syntaxique : 0,0910 s ;

Résultats

Temps d'exécution sur la fonction récursive `somme` :

- ▶ Analyse lexicale : 0,0803 s ;
- ▶ Construction de l'automate LR(1) : 4,6386 s ;
- ▶ Analyse syntaxique : 0,0910 s ;
- ▶ Transformation en boucle : 0,0006 s.

Résultats

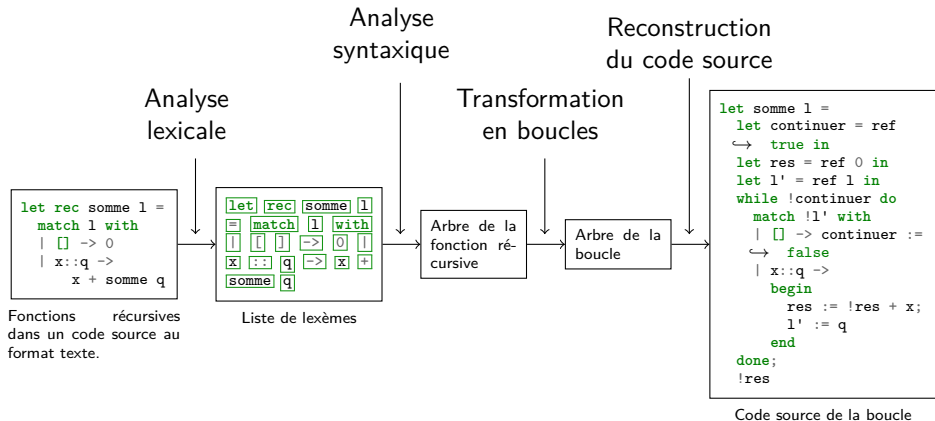
Résultats sur la somme des éléments d'une liste à 50×10^6 éléments :

Résultats

Résultats sur la somme des éléments d'une liste à 50×10^6 éléments :

- ▶ Dépassement de pile avec la méthode récursive naïve
- ▶ Exécution en 5 secondes avec le code généré (utilisant une boucle)

Plan



Annexes

5. Annexes

Rightmost derivation

Limitations de LR(0) : exemples de conflits

Automate LR(1)

Nécessité de la récursivité terminale

Exemple de fonction sans accumulateur

Code

Rightmost derivation

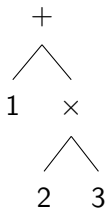
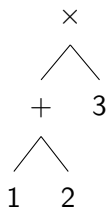
$$\underline{E} \rightarrow \underline{E} + \underline{E}$$

$$\underline{E} \rightarrow \underline{E} \times \underline{E}$$

$$\underline{E} \rightarrow \text{int}$$

$$1 + 2 \times 3$$

Rightmost derivation

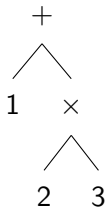
 $\underline{E} \rightarrow \underline{E} + \underline{E}$ $\underline{E} \rightarrow \underline{E} \times \underline{E}$ $\underline{E} \rightarrow \text{int}$ $1 + 2 \times 3$ *Leftmost**Rightmost*

Rightmost derivation

$$\underline{E} \rightarrow \underline{E} + \underline{E}$$

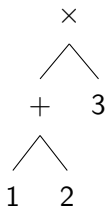
$$\underline{E} \rightarrow \underline{E} \times \underline{E}$$

$$\underline{E} \rightarrow \text{int}$$



Leftmost

$$\begin{array}{c}
 \text{leftmost} \\
 \underbrace{1 + 2} \times 3 \\
 \text{rightmost}
 \end{array}$$



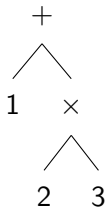
Rightmost

Rightmost derivation

$$\underline{E} \rightarrow \underline{E} + \underline{E}$$

$$\underline{E} \rightarrow \underline{E} \times \underline{E}$$

$$\underline{E} \rightarrow \text{int}$$

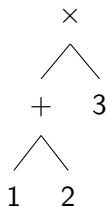


Leftmost

Analyse descendante

$$\overbrace{1 + 2}^{\text{leftmost}} \times 3$$

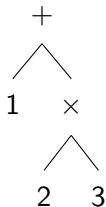
$\underbrace{2 \times 3}_{\text{rightmost}}$



Rightmost

Analyse ascendante

Rightmost derivation

$$\underline{E} \rightarrow \underline{E} + \underline{E}$$
$$\underline{E} \rightarrow \underline{E} \times \underline{E}$$
$$\underline{E} \rightarrow \text{int}$$


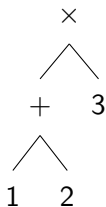
Leftmost

Analyse descendante
LL(1)

leftmost

$$\overbrace{1 + 2} \times 3$$

rightmost



Rightmost

Analyse ascendante
LR(0), LR(1), LALR(1), ...

Limitations de LR(0) : exemples de conflits

IF_EXPR → si bool **EXPR**

IF_EXPR → si bool **EXPR** sinon **EXPR**

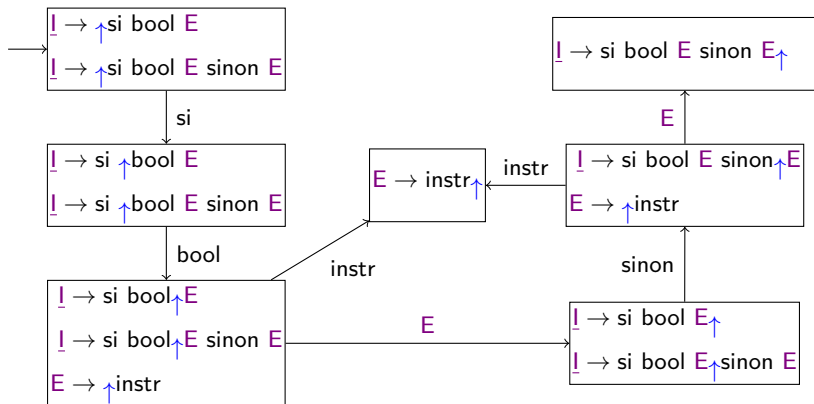
EXPR → instruction

Limitations de LR(0) : exemples de conflits

IF_EXPR → si bool EXPR

IF_EXPR → si bool EXPR sinon EXPR

EXPR → instruction

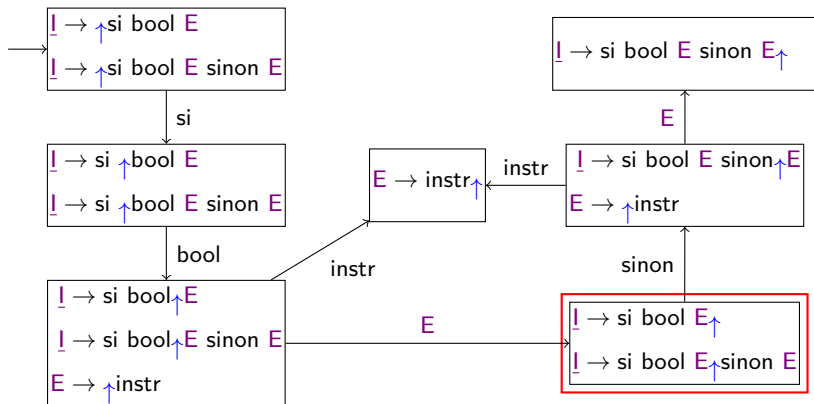


Limitations de LR(0) : exemples de conflits

IF_EXPR → si bool EXPR

IF_EXPR → si bool EXPR sinon EXPR

EXPR → instruction



Limitations de LR(0) : exemples de conflits

$\underline{I} \rightarrow \text{si bool } E \uparrow$ $\underline{I} \rightarrow \text{si bool } E \uparrow \text{sinon } E$
--

Que faire ?

Limitations de LR(0) : exemples de conflits

$\underline{I} \rightarrow \text{si bool } E \uparrow$ $\underline{I} \rightarrow \text{si bool } E \uparrow \text{ sinon } E$

Que faire ?

► Réduire $\underline{I} \rightarrow \text{si bool } E$?

Limitations de LR(0) : exemples de conflits

$\underline{I} \rightarrow \text{si bool } E \uparrow$ $\underline{I} \rightarrow \text{si bool } E \uparrow \text{sinon } E$
--

Que faire ?

- ▶ Réduire $\underline{I} \rightarrow \text{si bool } E$?
- ▶ Essayer de lire *sinon* ?

Limitations de LR(0) : exemples de conflits

$\begin{array}{l} \underline{I} \rightarrow \text{si bool } E \uparrow \\ \underline{I} \rightarrow \text{si bool } E \uparrow \text{sinon } E \end{array}$

Que faire ?

- ▶ Réduire $\underline{I} \rightarrow \text{si bool } E$?
- ▶ Essayer de lire *sinon* ?

C'est un conflit lire/réduire.

Limitations de LR(0) : différence avec LR(1)

I → si bool $E \uparrow$ } { $\diamond$ }

I → si bool $E \uparrow$ sinon E } { $\diamond$ }

Limitations de LR(0) : différence avec LR(1)

I → si bool $E \uparrow$ } { $\diamond$ }

I → si bool $E \uparrow$ sinon E } { $\diamond$ }

On regarde le lexème suivant :

Limitations de LR(0) : différence avec LR(1)

$$\underline{I} \rightarrow \text{si bool } E \uparrow \wr \{\diamond\}$$

$$\underline{I} \rightarrow \text{si bool } E \uparrow \text{sinon } E \wr \{\diamond\}$$

On regarde le lexème suivant :

- Si c'est la fin du fichier ($\diamond$), réduire $\underline{I} \rightarrow \text{si bool } E$;

Limitations de LR(0) : différence avec LR(1)

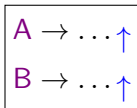
$$\underline{I} \rightarrow \text{si bool } E \uparrow \wr \{\diamond\}$$

$$\underline{I} \rightarrow \text{si bool } E \uparrow \text{sinon } E \wr \{\diamond\}$$

On regarde le lexème suivant :

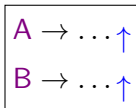
- ▶ Si c'est la fin du fichier ($\diamond$), réduire $\underline{I} \rightarrow \text{si bool } E$;
- ▶ Si c'est *sinon*, lire.

Limitations de LR(0) : conflits réduire/réduire



Deux choix :

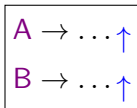
Limitations de LR(0) : conflits réduire/réduire



Deux choix :

- Réduire $A \rightarrow \dots$;

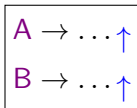
Limitations de LR(0) : conflits réduire/réduire



Deux choix :

- ▶ Réduire $A \rightarrow \dots$;
- ▶ Réduire $B \rightarrow \dots$.

Limitations de LR(0) : conflits réduire/réduire



Deux choix :

- ▶ Réduire $A \rightarrow \dots$;
- ▶ Réduire $B \rightarrow \dots$.

C'est un conflit réduire/réduire.

Limitations de LR(0) : conflits réduire/réduire

$$\begin{array}{l} A \rightarrow \dots \uparrow \mid \{a, b, \diamond\} \\ B \rightarrow \dots \uparrow \mid \{c, d\} \end{array}$$

On regarde le lexème suivant :

Limitations de LR(0) : conflits réduire/réduire

$$\begin{array}{l} A \rightarrow \dots \uparrow \mid \{a, b, \diamond\} \\ B \rightarrow \dots \uparrow \mid \{c, d\} \end{array}$$

On regarde le lexème suivant :

- Si c'est a , b ou la fin du fichier ($\diamond$), réduire $A \rightarrow \dots$;

Limitations de LR(0) : conflits réduire/réduire

$$\begin{array}{l} A \rightarrow \dots \uparrow \mid \{a, b, \diamond\} \\ B \rightarrow \dots \uparrow \mid \{c, d\} \end{array}$$

On regarde le lexème suivant :

- ▶ Si c'est a , b ou la fin du fichier ($\diamond$), réduire $A \rightarrow \dots$;
- ▶ Si c'est c ou d , réduire $B \rightarrow \dots$.

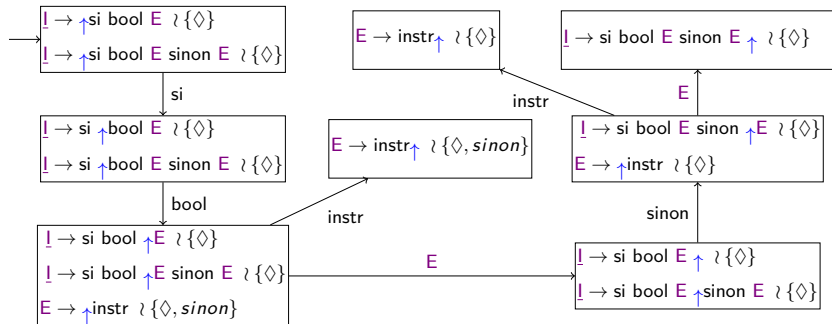
Automate LR(1)

IF_EXPR → si bool **EXPR**

IF_EXPR → si bool **EXPR** sinon **EXPR**

EXPR → instruction

Automate LR(1)

IF_EXPR $\rightarrow$ si bool EXPRIF_EXPR $\rightarrow$ si bool EXPR sinon EXPREXPR $\rightarrow$ instruction

Nécessité de la récursivité terminale

```
1  let rec somme l =  
2    match l with  
3    | [] -> 0  
4    | x::q ->  
5      x + somme q
```

Nécessité de la récursivité terminale

```
1  let rec somme l =  
2    match l with  
3    | [] -> 0  
4    | x::q ->  
5      x + l' := q
```

palindrome

```
1 | let rec palindrome (l: 'a list) : 'a list =  
2 |   match l with  
3 |   | [] -> []  
4 |   | x::q -> (palindrome q)@[x]
```

palindrome

```
1  let palindrome_rectified_whilified (l : 'a list) (cont : 'a list -> 'ret)
2      : 'ret =
3      let res_ref = ref None in
4      let cont_ref = ref cont in
5      let l_ref = ref l in
6      while !res_ref = None do
7          let cont = !cont_ref in
8          let l = !l_ref in
9          match l with
10         | [] -> res_ref := Some (cont [])
11         | x :: q ->
12             let new_cont (a_6 : 'a list) : 'ret = cont (a_6 @ [ x ]) in
13             l_ref := q;
14             cont_ref := new_cont
15      done;
16      Option.get !res_ref
17
18 let palindrome (l : 'a list) : 'a list =
19     palindrome_rectified_whilified l (fun res -> res)
```


Code

5. Annexes

Code

- caml_light.grammar
- automaton.ml
- caml_light.ml
- grammar_grammar.ml
- lexer.ml
- parser.ml
- rectify_helper.ml
- rectify.ml
- regex_grammar.ml
- regex.ml
- utils.ml
- while_cli.ml
- while.ml
- Tests

caml_light.grammar

See <https://caml.inria.fr/pub/distrib/caml-light-0.74/cl74refman.txt>

The axiom for this grammar is IMPLEMENTATION

P11

lowercase_ident := _?[a-z][A-Za-z0-9_]*

uppercase_ident := [A-Z][A-Za-z0-9_]*

integer_literal := -?([0-9]+|(0[xX][0-9A-Fa-f]+)|(0[oO][0-7]+)|(0[bB][0-1]+))

float_literal := -?[0-9]+(.[0-9]*)?([eE][\+-]?[0-9]+)?

P12

It's not fully clear what a regular-char is supposed to be. Here we assume it's any printable char

↪ that is not ` or \.

char_literal := `([ -\[\]-\_a-z]|(\\[\\`ntbr])|(\\[0-9][0-9][0-9]))`

u0023 =

Avoids starting a comment.

string_literal := "([!\$--\u0023]|(\\\"))*)"

Reserved words

and := and

else := else

if := if

of := of

try := try

as := as

end := end

in := in

or := or

type := type

begin := begin

exception := exception

let := let

```
prefix := prefix
value := value
do := do
for := for
match := match
rec := rec
where := where
done := done
fun := fun
mutable := mutable
then := then
while := while
downto := downto
function := function
not := not
to := to
with := with
mod := mod
```

```
hash := #
bang := !
bang_equals := !=
ampersand := &
open_paren := \(
close_paren :=\)
star := \*
star_dot := \*.
plus := \+
plus_dot := \+.
comma := ,
minus := -
minus_dot := -.
right_arrow := ->
dot := .
```

```
dot_paren := .\  
slash := /  
slash_dot := /.  
colon := :  
double_colon := ::  
colon_equals := :=  
semicolon := ;  
double_semicolon := ;;  
less := <  
less_dot := <.  
left_arrow := <-  
leq := <=  
leq_dot := <=.  
neq := <>  
neq_dot := <>.  
eq := =  
eq_dot := =.  
double_eq := ==  
greater := >  
greater_dot := >.  
geq := >=  
geq_dot := >=.  
at := @  
open_bracket := \  
open_bracket_bar := \[\  
close_bracket := \  
caret := ^  
underscore := _  
double_underscore := __  
open_brace := {  
pipe := \  
close_bracket_bar := \]  
close_brace := }  
apostrophe := '
```

5 – Annexes • 5.6 Code

```

double_star := \*\*

# P13
# Don't use global names: instead use lower/upper case identifiers.
#GLOBAL_NAME -> ident
#GLOBAL_NAME -> ident double_underscore ident

# TODO: Do we support the `prefix operator-name` variant?
VARIABLE -> lowercase_ident
# C_CONSTR and NC_CONSTR merged into a single CONSTR variant
#C_CONSTR -> uppercase_ident
#C_CONSTR -> open_bracket close_bracket
#C_CONSTR -> open_paren close_paren
#NC_CONSTR -> uppercase_ident
CONSTR -> uppercase_ident
CONSTR -> open_bracket close_bracket
CONSTR -> open_paren close_paren
TYPE_CONSTR -> lowercase_ident
LABEL -> lowercase_ident

# P15
TYP_EXPR_0 -> apostrophe lowercase_ident
TYP_EXPR_0 -> open_paren TYP_EXPR close_paren

TYP_EXPR_1 -> TYP_EXPR_0
TYP_EXPR_1 -> TYPE_CONSTR
TYP_EXPR_1 -> TYP_EXPR_1 TYPE_CONSTR
MORE_TYP_CONSTR_ARGS -> TYP_EXPR
MORE_TYP_CONSTR_ARGS -> TYP_EXPR comma MORE_TYP_CONSTR_ARGS
TYP_EXPR_1 -> open_paren TYP_EXPR comma MORE_TYP_CONSTR_ARGS close_paren TYPE_CONSTR

TYP_EXPR_2 -> TYP_EXPR_1
TYP_EXPR_2 -> TYP_EXPR_1 star TYP_EXPR_2

```

5 – Annexes • 5.6 Code

```
TYP_EXPR_3 -> TYP_EXPR_2
TYP_EXPR_3 -> TYP_EXPR_2 right_arrow TYP_EXPR_3

TYP_EXPR -> TYP_EXPR_3

# P16
CONSTANT -> integer_literal
CONSTANT -> float_literal
CONSTANT -> char_literal
CONSTANT -> string_literal
CONSTANT -> CONSTR

# P17
PATTERN_0 -> lowercase_ident
PATTERN_0 -> underscore
PATTERN_0 -> open_paren PATTERN close_paren
PATTERN_0 -> open_paren PATTERN colon TYP_EXPR close_paren
PATTERN_0 -> CONSTANT
FIELD_PATTERN -> LABEL eq PATTERN
FIELD_PATTERN_LIST -> FIELD_PATTERN
FIELD_PATTERN_LIST -> FIELD_PATTERN semicolon FIELD_PATTERN_LIST
PATTERN_0 -> open_brace FIELD_PATTERN_LIST close_brace
# Covered by PATTERN_0 -> CONSTANT -> CONSTR -> open_bracket close_bracket
#PATTERN_0 -> open_bracket close_bracket
A_PATTERN_LIST -> PATTERN
A_PATTERN_LIST -> PATTERN semicolon A_PATTERN_LIST
PATTERN_0 -> open_bracket A_PATTERN_LIST close_bracket

PATTERN_1 -> PATTERN_0
PATTERN_1 -> CONSTR PATTERN_0

PATTERN_2 -> PATTERN_1
PATTERN_2 -> PATTERN_1 double_colon PATTERN_2
```

```

PATTERN_3 -> PATTERN_2
PATTERN_3 -> PATTERN_3 comma PATTERN_2

PATTERN_4 -> PATTERN_3
PATTERN_4 -> PATTERN_4 pipe PATTERN_3

PATTERN_5 -> PATTERN_4
PATTERN_5 -> PATTERN_5 as lowercase_ident

PATTERN -> PATTERN_5

# P19
# Covered by the VARIABLE case.
#EXPR_0 -> ident
EXPR_0 -> VARIABLE
EXPR_0 -> CONSTANT
EXPR_0 -> open_paren EXPR close_paren
EXPR_0 -> begin EXPR end
EXPR_0 -> open_paren EXPR colon TYP_EXPR close_paren
EXPR_0 -> open_bracket EXPR close_bracket # EXPR already contains the
↳ machinery to handle semicolon separated expressions
EXPR_0 -> open_bracket_bar EXPR close_bracket_bar
# Grammar says LABEL EXPR but practical checking shows only precedence level 19.
FIELD_EXPR -> LABEL eq EXPR_18
FIELD_EXPR_LIST -> FIELD_EXPR
FIELD_EXPR_LIST -> FIELD_EXPR semicolon FIELD_EXPR_LIST
EXPR_0 -> open_brace FIELD_EXPR_LIST close_brace
EXPR_0 -> while EXPR do EXPR done
EXPR_0 -> for lowercase_ident eq EXPR to EXPR do EXPR done
EXPR_0 -> for lowercase_ident eq EXPR downto EXPR do EXPR done

EXPR_1 -> EXPR_0

```

5 – Annexes • 5.6 Code

```
EXPR_1 -> bang EXPR_0

EXPR_2 -> EXPR_1
EXPR_2 -> EXPR_1 dot LABEL
EXPR_2 -> EXPR_1 dot_paren EXPR close_paren

EXPR_3 -> EXPR_2
EXPR_3 -> EXPR_3 EXPR_2

EXPR_4 -> EXPR_3
# Already covered: EXPR_3 -> EXPR_3 EXPR_2 can be derived with a CONSTR in first position.
#EXPR_4 -> CONSTR EXPR_3

EXPR_5 -> EXPR_4
EXPR_5 -> minus EXPR_4
EXPR_5 -> minus_dot EXPR_4

EXPR_6 -> EXPR_5
EXPR_6 -> EXPR_5 double_star EXPR_6

EXPR_7 -> EXPR_6
EXPR_7 -> EXPR_7 mod EXPR_6

EXPR_8 -> EXPR_7
EXPR_8 -> EXPR_8 star EXPR_7
EXPR_8 -> EXPR_8 star_dot EXPR_7
EXPR_8 -> EXPR_8 slash EXPR_7
EXPR_8 -> EXPR_8 slash_dot EXPR_7

EXPR_9 -> EXPR_8
EXPR_9 -> EXPR_9 plus EXPR_8
EXPR_9 -> EXPR_9 plus_dot EXPR_8
EXPR_9 -> EXPR_9 minus EXPR_8
EXPR_9 -> EXPR_9 minus_dot EXPR_8
```



```
EXPR_10 -> EXPR_9  
EXPR_10 -> EXPR_9 double_colon EXPR_10
```

```
EXPR_11 -> EXPR_10  
EXPR_11 -> EXPR_10 at EXPR_11  
EXPR_11 -> EXPR_10 caret EXPR_11
```

```
EXPR_12 -> EXPR_11  
EXPR_12 -> EXPR_12 eq EXPR_11  
EXPR_12 -> EXPR_12 neq EXPR_11  
EXPR_12 -> EXPR_12 double_eq EXPR_11  
EXPR_12 -> EXPR_12 bang_equals EXPR_11  
EXPR_12 -> EXPR_12 less EXPR_11  
EXPR_12 -> EXPR_12 leq EXPR_11  
EXPR_12 -> EXPR_12 greater EXPR_11  
EXPR_12 -> EXPR_12 geq EXPR_11  
EXPR_12 -> EXPR_12 less_dot EXPR_11  
EXPR_12 -> EXPR_12 leq_dot EXPR_11  
EXPR_12 -> EXPR_12 greater_dot EXPR_11  
EXPR_12 -> EXPR_12 geq_dot EXPR_11
```

```
EXPR_13 -> EXPR_12  
EXPR_13 -> not EXPR_12
```

```
EXPR_14 -> EXPR_13  
EXPR_14 -> EXPR_14 ampersand EXPR_13
```

```
EXPR_15 -> EXPR_14  
EXPR_15 -> EXPR_15 or EXPR_14
```

```
EXPR_16 -> EXPR_15  
EXPR_16 -> EXPR_16 comma EXPR_15
```

```
EXPR_17 -> EXPR_16
EXPR_17 -> EXPR_1 dot LABEL left_arrow EXPR_17
EXPR_17 -> EXPR_1 dot_paren EXPR close_paren left_arrow EXPR_17
EXPR_17 -> EXPR_16 colon_equals EXPR_17

EXPR_18_DANGLING_THEN -> if EXPR then EXPR_18_DANGLING_THEN
EXPR_18_DANGLING_THEN -> if EXPR then EXPR_18_NO_DANGLING_THEN
EXPR_18_DANGLING_THEN -> if EXPR then EXPR_18_NO_DANGLING_THEN else EXPR_18_DANGLING_THEN
EXPR_18_NO_DANGLING_THEN -> if EXPR then EXPR_18_NO_DANGLING_THEN else EXPR_18_NO_DANGLING_THEN
EXPR_18_NO_DANGLING_THEN -> EXPR_17

EXPR_18 -> EXPR_18_DANGLING_THEN
EXPR_18 -> EXPR_18_NO_DANGLING_THEN

EXPR_19 -> EXPR_18
EXPR_19 -> EXPR_18 semicolon EXPR_19

EXPR_20_NO_DANGLING_MATCHING -> EXPR_19
EXPR_20_NO_DANGLING_MATCHING -> let LET_BINDING_LIST in EXPR_20_NO_DANGLING_MATCHING
EXPR_20_NO_DANGLING_MATCHING -> let rec LET_BINDING_LIST in EXPR_20_NO_DANGLING_MATCHING

EXPR_20_DANGLING_MATCHING -> match EXPR with SIMPLE_MATCHING
EXPR_20_DANGLING_MATCHING -> try EXPR with SIMPLE_MATCHING
EXPR_20_DANGLING_MATCHING -> fun MULTIPLE_MATCHING
EXPR_20_DANGLING_MATCHING -> function MULTIPLE_MATCHING
EXPR_20_DANGLING_MATCHING -> let LET_BINDING_LIST in EXPR_20_DANGLING_MATCHING
EXPR_20_DANGLING_MATCHING -> let rec LET_BINDING_LIST in EXPR_20_DANGLING_MATCHING

EXPR_20 -> EXPR_20_DANGLING_MATCHING
EXPR_20 -> EXPR_20_NO_DANGLING_MATCHING

EXPR -> EXPR_20
EXPR_NO_DANGLING_MATCHING -> EXPR_20_NO_DANGLING_MATCHING
```

5 – Annexes • 5.6 Code

```

SIMPLE_MATCHING -> PATTERN right_arrow EXPR_NO_DANGLING_MATCHING
SIMPLE_MATCHING -> PATTERN right_arrow EXPR_NO_DANGLING_MATCHING pipe SIMPLE_MATCHING

MULTIPLE_MATCHING -> B_PATTERN_LIST right_arrow EXPR_NO_DANGLING_MATCHING
MULTIPLE_MATCHING -> B_PATTERN_LIST right_arrow EXPR_NO_DANGLING_MATCHING pipe MULTIPLE_MATCHING

# Must use PATTERN_0 and not PATTERN as it causes conflicts (is ConstructorA ConstructorB,
↳ ConstructorA(ConstructorB) or two separate patterns? )
B_PATTERN_LIST -> PATTERN_0
B_PATTERN_LIST -> PATTERN_0 B_PATTERN_LIST

LET_BINDING_LIST -> LET_BINDING
LET_BINDING_LIST -> LET_BINDING and LET_BINDING_LIST
LET_BINDING -> PATTERN eq EXPR
LET_BINDING -> VARIABLE B_PATTERN_LIST eq EXPR

# 27

TYPE_DEFINITION -> type TYPEDEF_LIST
TYPEDEF_LIST -> TYPEDEF
TYPEDEF_LIST -> TYPEDEF and TYPEDEF_LIST

TYPE_CONSTR_DECL_LIST -> CONSTR_DECL
TYPE_CONSTR_DECL_LIST -> CONSTR_DECL pipe TYPE_CONSTR_DECL_LIST

LABEL_DECL_LIST -> LABEL_DECL
LABEL_DECL_LIST -> LABEL_DECL semicolon LABEL_DECL_LIST

TYPEDEF -> lowercase_ident eq TYPE_CONSTR_DECL_LIST # e.g type foo
↳ = One | Two | Three of foo
TYPEDEF -> lowercase_ident eq open_brace LABEL_DECL_LIST close_brace # e.g type foo
↳ = { bar : string }
TYPEDEF -> lowercase_ident double_eq TYP_EXPR # e.g type foo
↳ == list (type alias)

```

5 – Annexes • 5.6 Code

```

TYPEDEF -> lowercase_ident                                # e.g type foo
TYPEDEF -> TYPE_PARAMS lowercase_ident eq TYPE_CONSTR_DECL_LIST # Same as
↳ above, but with 'a or ('a, 'b)
TYPEDEF -> TYPE_PARAMS lowercase_ident eq open_brace LABEL_DECL_LIST close_brace # ^
TYPEDEF -> TYPE_PARAMS lowercase_ident double_eq TYP_EXPR      # ^
TYPEDEF -> TYPE_PARAMS lowercase_ident                      # ^

TYPE_PARAMS -> apostrophe lowercase_ident
TYPE_PARAM_LIST -> apostrophe lowercase_ident
TYPE_PARAM_LIST -> apostrophe lowercase_ident comma TYPE_PARAM_LIST
TYPE_PARAMS -> open_paren TYPE_PARAM_LIST close_paren

CONSTR_DECL -> uppercase_ident
CONSTR_DECL -> uppercase_ident of TYP_EXPR

LABEL_DECL -> lowercase_ident colon TYP_EXPR
LABEL_DECL -> mutable lowercase_ident colon TYP_EXPR

# 28

EXCEPTION_CONSTR_DECL_LIST -> CONSTR_DECL
EXCEPTION_CONSTR_DECL_LIST -> CONSTR_DECL and EXCEPTION_CONSTR_DECL_LIST
EXCEPTION_DEFINITION -> exception EXCEPTION_CONSTR_DECL_LIST

# TODO: Add directives once we can specify # in grammar source files.

IMPLEMENTATION -> IMPL_PHRASE double_semicolon
IMPLEMENTATION -> IMPL_PHRASE double_semicolon IMPLEMENTATION

IMPL_PHRASE -> EXPR
IMPL_PHRASE -> VALUE_DEFINITION
IMPL_PHRASE -> TYPE_DEFINITION
IMPL_PHRASE -> EXCEPTION_DEFINITION
#IMPL_PHRASE -> DIRECTIVE

```

```

VALUE_DEFINITION -> let LET_BINDING_LIST
VALUE_DEFINITION -> let rec LET_BINDING_LIST

# P32
# Removed: modern OCaml no longer supports this, so no way to determine precedence.
#EXPR -> EXPR where LET_BINDING
#EXPR -> EXPR where rec LET_BINDING

# P33
dot_open_bracket := .\[
EXPR_2 -> EXPR_1 dot_open_bracket EXPR close_bracket
EXPR_17 -> EXPR_1 dot_open_bracket EXPR close_bracket left_arrow EXPR_17

# TODO: Implement this.
#infix_symbol := [=<>@~\|&~\+\/$%-] [!$%&\*~\+./:;<=>\?@~\|~--]*
#prefix_symbol := [!\?][!$%&\*~\+./:;<=>\?@~\|~--]*

#EXPR -> prefix_symbol EXPR
#EXPR -> EXPR infix_symbol EXPR

EXPR_19 -> EXPR_18 semicolon
FIELD_EXPR_LIST -> FIELD_EXPR semicolon
EXPR_20_DANGLING_MATCHING -> match EXPR with pipe SIMPLE_MATCHING
EXPR_20_DANGLING_MATCHING -> try EXPR with pipe SIMPLE_MATCHING
EXPR_20_DANGLING_MATCHING -> fun pipe MULTIPLE_MATCHING
EXPR_20_DANGLING_MATCHING -> function pipe MULTIPLE_MATCHING

double_ampersand := &&
double_pipe := \| \|

EXPR_14 -> EXPR_14 double_ampersand EXPR_13
EXPR_15 -> EXPR_15 double_pipe EXPR_14

```

5 – Annexes • 5.6 Code

```

# Our changes to make Caml Light more like modern OCaml (for "seamless" parsing of OCaml programs)

# Character literals with ' instead of `
modern_char_literal := '([ \[\]-_a--]|(\\[\\'ntbr])|(\\[0-9][0-9][0-9]))'
CONSTANT -> modern_char_literal

# Module implementations without required double semicolons
IMPLEMENTATION_NO_EXPR_BEGIN -> IMPL_PHRASE_NO_EXPR double_semicolon
IMPLEMENTATION_NO_EXPR_BEGIN -> IMPL_PHRASE_NO_EXPR double_semicolon IMPLEMENTATION
IMPLEMENTATION_NO_EXPR_BEGIN -> IMPL_PHRASE_NO_EXPR
IMPLEMENTATION_NO_EXPR_BEGIN -> IMPL_PHRASE_NO_EXPR IMPLEMENTATION_NO_EXPR_BEGIN

IMPL_PHRASE_NO_EXPR -> VALUE_DEFINITION
IMPL_PHRASE_NO_EXPR -> TYPE_DEFINITION
IMPL_PHRASE_NO_EXPR -> EXCEPTION_DEFINITION

IMPLEMENTATION -> IMPL_PHRASE
IMPLEMENTATION -> IMPL_PHRASE IMPLEMENTATION_NO_EXPR_BEGIN

# Empty array literals
CONSTR -> open_bracket_bar close_bracket_bar

# begin end as synonym for ()
CONSTR -> begin end

# Single = for type aliases
TYPEDEF -> lowercase_ident eq TYP_EXPR
TYPEDEF -> TYPE_PARAMS lowercase_ident eq TYP_EXPR

# True and false as constants
true := true
false := false

CONSTANT -> true

```

```
CONSTANT -> false
```

```
# Allow specifying function return types
```

```
LET_BINDING -> VARIABLE B_PATTERN_LIST colon TYP_EXPR eq EXPR
```

automaton.ml

```

1  open Utils
2
3  type ('symbol, 'state) deterministic_automaton = {
4    states : 'state Hashset.t;
5    initial_state : 'state;
6    final_states : 'state Hashset.t;
7    transition : 'state -> 'symbol -> 'state option;
8  }
9
10 type ('symbol, 'state) automaton = {
11   states : 'state list;
12   initial_states : 'state list;
13   final_states : 'state list;
14   transitions : ('state * 'symbol * 'state) list;
15 }
16
17 type 'symbol or_epsilon = Epsilon | Symbol of 'symbol
18
19 type ('symbol, 'state) epsilon_automaton =
20   ('symbol or_epsilon, 'state) automaton
21
22 type ('symbol, 'state) execution_state = {
23   automaton : ('symbol, 'state) deterministic_automaton;
24   current_state : 'state;
25 }
26
27 (** Returns an execution of a starting in a's initial_state *)
28 let start_execution (a : ('symbol, 'state) deterministic_automaton) :
29   ('symbol, 'state) execution_state =
30   { automaton = a; current_state = a.initial_state }
31
32 (** Returns the state following e after reading s. Returns None if the automaton
33 has no such state. *)

```


5 – Annexes • 5.6 Code

```

34 let next_state (e : ('symbol, 'state) execution_state) (s : 'symbol) :
35   ('symbol, 'state) execution_state option =
36   let next_state = e.automaton.transition e.current_state s in
37   match next_state with
38   | None -> None
39   | Some next_state ->
40     Some { automaton = e.automaton; current_state = next_state }
41
42 (** Returns true if e is a final state. *)
43 let is_final_state (e : _ execution_state) : bool =
44   Hashset.mem e.automaton.final_states e.current_state
45
46 (** `remove_epsilon_transitions a` returns an automaton equivalent to a with all
47 epsilon-transitions removed.
48
49 The returned automaton may not be deterministic. *)
50 let remove_epsilon_transitions (a : ('symbol, 'state) epsilon_automaton) :
51   ('symbol, 'state) automaton =
52   let incoming_epsilon_transitions = Hashtbl.create 2 in
53   let outgoing_epsilon_transitions = Hashtbl.create 2 in
54   let result_transitions = Hashtbl.create 2 in
55   let result_final_states = Hashset.create () in
56
57   List.iter (Hashset.add result_final_states) a.final_states;
58   List.iter
59     (fun (state0, symbol, state1) ->
60       match symbol with
61       | Epsilon ->
62         let incoming_states =
63           match Hashtbl.find_opt incoming_epsilon_transitions state1 with
64           | None ->
65             let t = Hashset.create () in
66             Hashtbl.add incoming_epsilon_transitions state1 t;
67             t

```

```

68         | Some t -> t
69     in
70     Hashset.add incoming_states state0;
71     let outgoing_states =
72         match Hashtbl.find_opt outgoing_epsilon_transitions state0 with
73     | None ->
74         let t = Hashset.create () in
75         Hashtbl.add outgoing_epsilon_transitions state0 t;
76         t
77     | Some t -> t
78     in
79     Hashset.add outgoing_states state1
80 | Symbol s ->
81     let outgoing_transitions =
82         match Hashtbl.find_opt result_transitions state0 with
83     | None ->
84         let t = Hashtbl.create 2 in
85         Hashtbl.add result_transitions state0 t;
86         t
87     | Some t -> t
88     in
89     let transitions_for_symbol =
90         match Hashtbl.find_opt outgoing_transitions s with
91     | None ->
92         let t = Hashset.create () in
93         Hashtbl.add outgoing_transitions s t;
94         t
95     | Some t -> t
96     in
97     Hashset.add transitions_for_symbol state1)
98 a.transitions;
99
100 while Hashtbl.length incoming_epsilon_transitions > 0 do
101     let state, incoming_states =

```

```

102     hashtable_remove_one incoming_epsilon_transitions
103   in
104
105   if List.mem state a.final_states then
106     Hashset.iter (Hashset.add result_final_states) incoming_states;
107
108   (* Remove outgoing transitions from start states. *)
109   Hashset.iter
110     (fun start_state ->
111       let outgoing_transitions =
112         Hashtable.find outgoing_epsilon_transitions start_state
113       in
114         Hashset.remove outgoing_transitions state)
115     incoming_states;
116
117   (* Add transitions from all incoming states to reachable states, bypassing this state. *)
118   (match Hashtable.find_opt result_transitions state with
119   | None -> ()
120   | Some state_transitions ->
121     Hashtable.iter
122       (fun symbol end_states ->
123         Hashset.iter
124           (fun start_state ->
125             let start_state_transitions =
126               match Hashtable.find_opt result_transitions start_state with
127             | None ->
128               let t = Hashtable.create 2 in
129               Hashtable.add result_transitions start_state t;
130               t
131             | Some t -> t
132           in
133             let transitions_for_symbol =
134               match Hashtable.find_opt start_state_transitions symbol with
135             | None ->

```

```

136         let t = Hashset.create () in
137         Hashtbl.add start_state_transitions symbol t;
138         t
139     | Some t -> t
140 in
141     Hashset.iter (Hashset.add transitions_for_symbol) end_states)
142     incoming_states)
143     state_transitions);
144
145     (* Add epsilon transitions from incoming states bypassing this state *)
146     match Hashtbl.find_opt outgoing_epsilon_transitions state with
147     | None -> ()
148     | Some end_states ->
149         (* Don't introduce e-loops. *)
150         Hashset.remove end_states state;
151         Hashset.iter
152             (fun start_state ->
153                 let outgoing_transitions =
154                     Hashtbl.find outgoing_epsilon_transitions start_state
155                 in
156                     Hashset.iter (Hashset.add outgoing_transitions) end_states)
157             incoming_states;
158         Hashset.iter
159             (fun end_state ->
160                 let incoming_transitions =
161                     Hashtbl.find incoming_epsilon_transitions end_state
162                 in
163                     Hashset.iter (Hashset.add incoming_transitions) incoming_states)
164             end_states
165     done;
166
167 {
168     states = a.states;
169     initial_states = a.initial_states;

```

5 – Annexes • 5.6 Code

```

170     final_states = Hashset.to_list result_final_states;
171     transitions =
172         Hashtbl.to_seq result_transitions
173         |> List.of_seq
174         |> List.map (fun (state0, transitions) ->
175             Hashtbl.to_seq transitions |> List.of_seq
176             |> List.map (fun (symbol, end_states) ->
177                 (state0, symbol, end_states)))
178         |> List.flatten
179         |> List.map (fun (state0, symbol, end_states) ->
180             Hashset.to_list end_states
181             |> List.map (fun state1 -> (state0, symbol, state1)))
182         |> List.flatten;
183 }
184
185 (** `determinize a` returns an automaton equivalent to `a` that is deterministic
186 and complete. *)
187 let determinize (a : ('symbol, 'state) automaton) :
188     ('symbol, 'state list) deterministic_automaton =
189     let initial_state = List.sort compare a.initial_states in
190     let alphabet = Hashset.create () in
191     List.iter (fun (_, s, _) -> Hashset.add alphabet s) a.transitions;
192
193     (* multi_transitions[state0][symbol] is the set of all states reachable from state0 by
194     ↪ reading symbol.
195     This saves us iterating over all transitions for every super-state we create. *)
196     let multi_transitions = Hashtbl.create 2 in
197
198     (* Populate multi_transitions. *)
199     List.iter
200         (fun (state0, symbol, state1) ->
201             let transitions_for_state0 =
202                 match Hashtbl.find_opt multi_transitions state0 with
203                 | None ->

```

5 – Annexes • 5.6 Code

```

203         let t = Hashtbl.create 2 in
204         Hashtbl.add multi_transitions state0 t;
205         t
206     | Some t -> t
207 in
208 let transitions_for_symbol =
209     match Hashtbl.find_opt transitions_for_state0 symbol with
210     | None ->
211         let t = Hashset.create () in
212         Hashtbl.add transitions_for_state0 symbol t;
213         t
214     | Some t -> t
215 in
216 Hashset.add transitions_for_symbol state1)
217 a.transitions;
218
219 (* transitions_for state0 is a dictionary d such that d[symbol] is the
220    super-state reached by reading symbol from the super-state state0. *)
221 let transitions_for (state : 'state list) : ('symbol, 'state list) Hashtbl.t =
222     (* end_state_for_symbol symbol is the super-state reached by reading symbol from the
223        ↪ super-state `state` *)
224     let end_state_for_symbol (s : 'symbol) =
225         state
226     |> List.map (fun simple_state ->
227         match Hashtbl.find_opt multi_transitions simple_state with
228         | None -> []
229         | Some transitions -> (
230             match Hashtbl.find_opt transitions s with
231             | None -> []
232             | Some s -> Hashset.to_list s))
233     |> List.flatten |> List.sort_uniq compare
234 in
235 let res = Hashtbl.create 2 in
236 Hashset.iter (fun s -> Hashtbl.add res s (end_state_for_symbol s)) alphabet;

```

5 – Annexes • 5.6 Code

```

236     res
237 in
238
239   (* transitions[state0][symbol] is the super-state reached by reading symbol from the
    ↪ super-state state0. *)
240   let transitions = Hashtbl.create 2 in
241   (* A running collection of all super-states we have created. *)
242   let states = Hashset.create () in
243   (* Newly created super-states that have not yet been processed. *)
244   let pending_states = Hashset.create () in
245
246   Hashset.add pending_states initial_state;
247
248   while not (Hashset.is_empty pending_states) do
249     let current_state = Hashset.remove_one pending_states in
250     Hashset.add states current_state;
251     let current_transitions = transitions_for current_state in
252     Hashtbl.add transitions current_state current_transitions;
253     Hashtbl.iter
254       (fun _ end_state ->
255         if not (Hashset.mem states end_state) then
256           Hashset.add pending_states end_state)
257     current_transitions
258   done;
259
260   (* Compute final states. *)
261   let final_states = Hashset.create () in
262   Hashset.iter
263     (fun state ->
264       if
265         List.exists
266           (fun simple_state -> List.mem simple_state a.final_states)
267           state
268       then Hashset.add final_states state)

```

5 – Annexes • 5.6 Code

```

269     states;
270
271   {
272     initial_state;
273     states;
274     final_states;
275     transition =
276       (fun state0 symbol ->
277         let state_transitions = Hashtbl.find transitions state0 in
278         Hashtbl.find_opt state_transitions symbol);
279   }
280
281   (** Returns an automaton identical to a but with state and all transitions
282   leading to state removed.
283
284   This is primarily used to preemptively end executions of a if they get stuck
285   in a sinkhole. *)
286   let remove_state (state : 'state)
287     (a : ('symbol, 'state) deterministic_automaton) :
288     ('symbol, 'state) deterministic_automaton =
289     if a.initial_state = state then
290       failwith "Cannot remove the initial state of an automaton";
291
292     let filtered_states = Hashset.create () in
293     Hashset.iter
294       (fun s -> if s <> state then Hashset.add filtered_states s)
295       a.states;
296
297     let filtered_final_states = Hashset.create () in
298     Hashset.iter
299       (fun s -> if s <> state then Hashset.add filtered_final_states s)
300       a.final_states;
301
302   {

```


5 – Annexes • 5.6 Code

```

303     states = filtered_states;
304     initial_state = a.initial_state;
305     final_states = filtered_final_states;
306     transition =
307       (fun state0 symbol ->
308         match a.transition state0 symbol with
309         | None -> None
310         | Some state1 when state1 = state -> None
311         | Some state1 -> Some state1);
312   }
313
314   let print_automaton (string_of_symbol : 'symbol -> string)
315     (string_of_state : 'state -> string) (a : ('symbol, 'state) automaton) :
316     unit =
317     print_endline "Automaton:";
318     print_endline "Initial states:";
319     List.fold_left
320       (fun () state -> print_endline ("  " ^ string_of_state state))
321       () a.initial_states;
322     print_endline "Final states:";
323     List.fold_left
324       (fun () state -> print_endline ("  " ^ string_of_state state))
325       () a.final_states;
326     print_endline "Transitions:";
327     List.fold_left
328       (fun () (state0, symbol, state1) ->
329         print_endline
330           ("  " ^ string_of_state state0 ^ " -- " ^ string_of_symbol symbol
331            ^ " -> " ^ string_of_state state1))
332       () a.transitions

```

caml_light.ml

```
1  open Utils
2  open Lexer
3  open Regex_grammar
4  open Parser
5  open Grammar_grammar
6
7  (** The token rules and grammar rules for parsing Caml Light. *)
8  let caml_light_tokens, caml_light_grammar =
9      let caml_light_grammar_definition_fp = open_in "../caml_light.grammar" in
10     let caml_light_grammar_definition =
11         really_input_string caml_light_grammar_definition_fp
12         (in_channel_length caml_light_grammar_definition_fp)
13     in
14     close_in caml_light_grammar_definition_fp;
15     parse_grammar caml_light_grammar_definition
16
17  (** The LR(1) automaton built from the Caml Light grammar. *)
18  let caml_light_automaton =
19      construit_automate_LR1 caml_light_grammar "IMPLEMENTATION" "<eof>"
20
21  (** Collapses precedence rules for target in tree.
22
23      Parsing ambiguities in grammars are resolved by introducing precedence
24      levels that indicate which groupings should be preferred.
25
26      However, such a solution leads to the syntax tree containing many variants
27      of what is semantically only one rule.
28
29      This method collapses all occurrences of variants of a rule into a single
30      type.
31
32      The resulting type is defined by target. A is considered to be a variant of
33      target if its type is prefixed by target, *)
```

5 – Annexes • 5.6 Code

```

34 let rec collapse_precedence (target : string)
35   (tree : (string, string) syntax_tree) : (string, string) syntax_tree =
36   match tree with
37   | Node (nt1, [ Node (nt2, children) ])
38     when nt1 = target && String.starts_with ~prefix:target nt2 ->
39     collapse_precedence target (Node (target, children))
40   | Node (nt, children)
41     when nt <> target && String.starts_with ~prefix:target nt ->
42     collapse_precedence target (Node (target, children))
43   | Node (nt, children) ->
44     Node (nt, List.map (collapse_precedence target) children)
45   | _ -> tree
46
47 (** Parses Caml Light source into a syntax tree. *)
48 let parse_caml_light_syntax_tree (source : string) =
49   (* Additional token rules for separating syntax elements (whitespace) and
50    parsing comments (comment_start, comment_end, unrecognizable - which
51    is allowed within a comment). *)
52   let enhanced_token_rules =
53     List.rev_append caml_light_tokens
54     [
55       (parse_regex "(\\n|\\t)+", "<whitespace>");
56       (parse_regex "\\(<*", "<comment_start>");
57       (parse_regex "\\)*\\)", "<comment_end>");
58     ]
59   in
60   let tokens =
61     tokenize enhanced_token_rules "<eof>" "<unrecognizable>" source
62   in
63
64   let rec filter_tokens_from (tokens : string token list) (comment_level : int)
65     (res : string token list) =
66     if comment_level < 0 then failwith "Too many comment end tags"
67     else

```

5 – Annexes • 5.6 Code

```

68     match tokens with
69     | [] -> List.rev res
70     | x :: q -> (
71         match x.token_type with
72         | "<comment_start>" -> filter_tokens_from q (comment_level + 1) res
73         | "<comment_end>" -> filter_tokens_from q (comment_level - 1) res
74         | "<whitespace>" -> filter_tokens_from q comment_level res
75         | _ ->
76             if comment_level > 0 then filter_tokens_from q comment_level res
77             else filter_tokens_from q comment_level (x :: res))
78 in
79
80 let filtered_tokens = filter_tokens_from tokens 0 [] in
81 parse caml_light_automaton "<eof>" filtered_tokens "IMPLEMENTATION"
82 |> collapse_precedence "EXPR"
83 |> collapse_precedence "PATTERN"
84 |> collapse_precedence "TYP_EXPR"
85 |> collapse_precedence "IMPL_PHRASE"
86 |> collapse_precedence "IMPLEMENTATION"
87
88 type program = phrase list
89
90 and phrase =
91     | Expression of expression
92     | ValueDefinition of value_definition
93     | TypeDefinition of type_definition
94     | ExceptionDefinition of exception_definition
95
96 and parenthesis_style = Parenthesis | BeginEnd
97 and for_direction = Up | Down
98 and prefix_operation = Minus | MinusDot
99
100 and infix_operation =
101     | DoubleStar

```

5 – Annexes • 5.6 Code

```
102 | Mod
103 | Star
104 | StarDot
105 | Slash
106 | SlashDot
107 | Plus
108 | PlusDot
109 | Minus
110 | MinusDot
111 | DoubleColon
112 | At
113 | Caret
114 | Eq
115 | Neq
116 | DoubleEq
117 | BangEq
118 | Less
119 | Leq
120 | Greater
121 | Geq
122 | LessDot
123 | LeqDot
124 | GreaterDot
125 | GeqDot
126 | Ampersand
127 | Or
128 | DoubleAmpersand
129 | DoublePipe
130
131 and pattern_cases = (pattern list * expression) list
132 and function_literal_style = Fun | Function
133 and label = string
134 and lowercase_ident = string
135 and uppercase_ident = string
```

5 – Annexes • 5.6 Code

```
136 and variable = string
137 and type_constructor = string
138
139 and constructor =
140   | Named of uppercase_ident
141   | EmptyList
142   | Unit of parenthesis_style
143   | EmptyArray
144
145 and function_ = {
146   name : variable;
147   parameters : pattern list;
148   body : expression;
149   return_type : type_expression option;
150 }
151
152 and binding =
153   | Variable of { lhs : pattern; value : expression }
154   | Function of function_
155
156 and expression =
157   | Variable of variable
158   | Constant of constant
159   | Parenthesised of { inner : expression; style : parenthesis_style }
160   | TypeCoercion of { inner : expression; typ : type_expression }
161   | ListLiteral of expression list
162   | ArrayLiteral of expression list
163   | RecordLiteral of (label * expression) list
164   | WhileLoop of { condition : expression; body : expression }
165   | ForLoop of {
166     direction : for_direction;
167     variable : lowercase_ident;
168     start : expression;
169     finish : expression;
```

```
170     body : expression;
171   }
172 | Dereference of expression
173 | FieldAccess of { receiver : expression; target : label }
174 | ArrayAccess of { receiver : expression; target : expression }
175 | FunctionApplication of {
176     receiver : expression;
177     arguments : expression list;
178 }
179 | PrefixOperation of { receiver : expression; operation : prefix_operation }
180 | InfixOperation of {
181     lhs : expression;
182     rhs : expression;
183     operation : infix_operation;
184 }
185 | Negation of expression
186 | Tuple of expression list
187 | FieldAssignment of {
188     receiver : expression;
189     target : label;
190     value : expression;
191 }
192 | ArrayAssignment of {
193     receiver : expression;
194     target : expression;
195     value : expression;
196 }
197 | ReferenceAssignment of { receiver : expression; value : expression }
198 | If of {
199     condition : expression;
200     body : expression;
201     else_body : expression option;
202 }
203 | Sequence of expression list
```

5 – Annexes • 5.6 Code

```

204 | Match of { value : expression; cases : pattern_cases }
205 | Try of { value : expression; cases : pattern_cases }
206 | FunctionLiteral of { style : function_literal_style; cases : pattern_cases }
207 | LetBinding of { bindings : binding list; is_rec : bool; inner : expression }
208 | StringAccess of { receiver : expression; target : expression }
209 | StringAssignment of {
210 |     receiver : expression;
211 |     target : expression;
212 |     value : expression;
213 | }
214
215 and pattern =
216 | Ident of lowercase_ident
217 | Underscore
218 | Parenthesised of pattern
219 | TypeCoercion of { inner : pattern; typ : type_expression }
220 | Constant of constant
221 | Record of (label * pattern) list
222 | List of pattern list
223 | Construction of { constructor : constructor; argument : pattern }
224 | Concatenation of { head : pattern; tail : pattern }
225 | Tuple of pattern list
226 | Or of pattern list
227 | As of { inner : pattern; name : lowercase_ident }
228
229 and type_expression =
230 | Argument of lowercase_ident
231 | Parenthesised of type_expression
232 | Construction of {
233 |     constructor : type_constructor;
234 |     arguments : type_expression list;
235 | }
236 | Tuple of type_expression list
237 | Function of { argument : type_expression; result : type_expression }

```



```
238
239 and char_literal_style = Old | New
240
241 and constant =
242   | IntegerLiteral of int
243   | FloatLiteral of float
244   | CharacterLiteral of { style : char_literal_style; value : char }
245   | StringLiteral of string
246   | Construction of constructor
247   | True
248   | False
249
250 and value_definition = { bindings : binding list; is_rec : bool }
251 and type_definition = typedef list
252 and type_alias_style = DoubleEq | SingleEq
253
254 and field_declaration = {
255   name : lowercase_ident;
256   is_mutable : bool;
257   typ : type_expression;
258 }
259
260 and type_constructor_declaration = {
261   name : uppercase_ident;
262   parameter : type_expression option;
263 }
264
265 and typedef =
266   | Constructors of {
267     name : lowercase_ident;
268     parameters : lowercase_ident list;
269     parenthesised_parameters : bool;
270     constructors : type_constructor_declaration list;
271   }
```

5 – Annexes • 5.6 Code

```

272 | Record of {
273 |   name : lowercase_ident;
274 |   parameters : lowercase_ident list;
275 |   parenthesised_parameters : bool;
276 |   fields : field_declaration list;
277 | }
278 | Alias of {
279 |   name : lowercase_ident;
280 |   style : type_alias_style;
281 |   parameters : lowercase_ident list;
282 |   parenthesised_parameters : bool;
283 |   aliased : type_expression;
284 | }
285 | Anonymous of {
286 |   name : lowercase_ident;
287 |   parameters : lowercase_ident list;
288 |   parenthesised_parameters : bool;
289 | }
290
291 and exception_definition = { exceptions : type_constructor_declaration list }
292
293 (** Converts a Caml Light syntax_tree to a Caml Light AST. *)
294 let rec ast_of_syntax_tree (tree : (string, string) syntax_tree) : program =
295   let rec label (tree : (string, string) syntax_tree) : label =
296     match tree with
297     | Node ("LABEL", [ Leaf { token_type = "lowercase_ident"; value } ]) ->
298       value
299     | _ -> failwith "Not a label"
300   and variable (tree : (string, string) syntax_tree) : variable =
301     match tree with
302     | Node ("VARIABLE", [ Leaf { token_type = "lowercase_ident"; value } ]) ->
303       value
304     | _ -> failwith "Not a variable"
305   and constructor (tree : (string, string) syntax_tree) : constructor =

```

```

306 match tree with
307 | Node ("CONSTR", [ Leaf { token_type = "uppercase_ident"; value } ]) ->
308   Named value
309 | Node
310   ( "CONSTR",
311     [
312       Leaf { token_type = "open_bracket" };
313       Leaf { token_type = "close_bracket" };
314     ] ) ->
315   EmptyList
316 | Node
317   ( "CONSTR",
318     [
319       Leaf { token_type = "open_paren" };
320       Leaf { token_type = "close_paren" };
321     ] ) ->
322   Unit Parenthesis
323 | Node
324   ( "CONSTR",
325     [
326       Leaf { token_type = "open_bracket_bar" };
327       Leaf { token_type = "close_bracket_bar" };
328     ] ) ->
329   EmptyArray
330 | Node
331   ( "CONSTR",
332     [ Leaf { token_type = "begin" }; Leaf { token_type = "end" } ] ) ->
333   Unit BeginEnd
334 | _ -> failwith "Not a constructor"
335 and type_constructor (tree : (string, string) syntax_tree) : type_constructor
336 =
337 match tree with
338 | Node ("TYPE_CONSTR", [ Leaf { token_type = "lowercase_ident"; value } ])
339   ->

```

5 – Annexes • 5.6 Code

```

340     value
341     | _ -> failwith "Not a type constructor"
342 and more_type_constructor_arguments (tree : (string, string) syntax_tree) :
343     type_expression list =
344     match tree with
345     | Node ("MORE_TYP_CONSTR_ARGS", [ inner ]) -> [ type_expression inner ]
346     | Node
347       ("MORE_TYP_CONSTR_ARGS", [ inner; Leaf { token_type = "comma" }; rest ])
348     ->
349       type_expression inner :: more_type_constructor_arguments rest
350     | _ -> failwith "Not a type expression argument list"
351 and tuple_type_expression (tree : (string, string) syntax_tree) :
352     type_expression list =
353     match tree with
354     | Node ("TYP_EXPR", [ first; Leaf { token_type = "star" }; rest ]) ->
355       type_expression first :: tuple_type_expression rest
356     | _ -> [ type_expression tree ]
357 and type_expression (tree : (string, string) syntax_tree) : type_expression =
358     match tree with
359     | Node
360       ( "TYP_EXPR",
361         [
362           Leaf { token_type = "apostrophe" };
363           Leaf { token_type = "lowercase_ident"; value };
364         ] ) ->
365       Argument value
366     | Node
367       ( "TYP_EXPR",
368         [
369           Leaf { token_type = "open_paren" };
370           inner;
371           Leaf { token_type = "close_paren" };
372         ] ) ->
373       Parenthesised (type_expression inner)

```

```

374 | Node ("TYP_EXPR", [ (Node ("TYPE_CONSTR", _) as constructor) ]) ->
375 |   let constructor_node = type_constructor constructor in
376 |
377 |   Construction { constructor = constructor_node; arguments = [] }
378 | Node
379 |   ( "TYP_EXPR",
380 |     [
381 |       (Node ("TYP_EXPR", _) as argument);
382 |       (Node ("TYPE_CONSTR", _) as constructor);
383 |     ] ) ->
384 |   let constructor_node = type_constructor constructor in
385 |   let argument_node = type_expression argument in
386 |
387 |   Construction
388 |     { constructor = constructor_node; arguments = [ argument_node ] }
389 | Node
390 |   ( "TYP_EXPR",
391 |     [
392 |       Leaf { token_type = "open_paren" };
393 |       first_argument;
394 |       Leaf { token_type = "comma" };
395 |       more_arguments;
396 |       Leaf { token_type = "close_paren" };
397 |       constructor;
398 |     ] ) ->
399 |   let arguments =
400 |     type_expression first_argument
401 |     :: more_type_constructor_arguments more_arguments
402 |   in
403 |   let constructor_node = type_constructor constructor in
404 |   Construction { constructor = constructor_node; arguments }
405 | Node ("TYP_EXPR", [ _; Leaf { token_type = "star" }; _ ] ) ->
406 |   let inner_expression_nodes = tuple_type_expression tree in
407 |

```

5 – Annexes • 5.6 Code

```

408     Tuple inner_expression_nodes
409 | Node
410   ("TYP_EXPR", [ argument; Leaf { token_type = "right_arrow" }; result ])
411   ->
412     let argument_node = type_expression argument in
413     let result_node = type_expression result in
414     Function { argument = argument_node; result = result_node }
415 | _ -> failwith "Not a type expression"
416 and constant (tree : (string, string) syntax_tree) : constant =
417   match tree with
418 | Node ("CONSTANT", [ Leaf { token_type = "integer_literal"; value } ]) ->
419     IntegerLiteral (int_of_string value)
420 | Node ("CONSTANT", [ Leaf { token_type = "float_literal"; value } ]) ->
421     FloatLiteral (float_of_string value)
422 | Node ("CONSTANT", [ Leaf { token_type = "char_literal"; value } ]) ->
423     CharacterLiteral { value = value.[1]; style = Old }
424 | Node ("CONSTANT", [ Leaf { token_type = "string_literal"; value } ]) ->
425     StringLiteral (String.sub value 1 (String.length value - 2))
426 | Node ("CONSTANT", [ (Node "CONSTR", _) as constr ] ) ->
427     let constr_node = constructor constr in
428     Construction constr_node
429 | Node ("CONSTANT", [ Leaf { token_type = "modern_char_literal"; value } ])
430   ->
431     CharacterLiteral { value = value.[0]; style = New }
432 | Node ("CONSTANT", [ Leaf { token_type = "true" } ]) -> True
433 | Node ("CONSTANT", [ Leaf { token_type = "false" } ]) -> False
434 | _ -> failwith "Not a constant"
435 and field_pattern (tree : (string, string) syntax_tree) : label * pattern =
436   match tree with
437 | Node ("FIELD_PATTERN", [ lbl; Leaf { token_type = "eq" }; ptrn ]) ->
438     (label lbl, pattern ptrn)
439 | _ -> failwith "Not a field pattern"
440 and field_pattern_list (tree : (string, string) syntax_tree) :
441   (label * pattern) list =

```

```

442     match tree with
443     | Node ("FIELD_PATTERN_LIST", [ inner ]) -> [ field_pattern inner ]
444     | Node
445       ( "FIELD_PATTERN_LIST",
446         [ inner; Leaf { token_type = "semicolon" }; rest ] ) ->
447       field_pattern inner :: field_pattern_list rest
448     | _ -> failwith "Not a field pattern list"
449 and a_pattern_list (tree : (string, string) syntax_tree) : pattern list =
450 match tree with
451 | Node ("A_PATTERN_LIST", [ pttrn ]) -> [ pattern pttrn ]
452 | Node ("A_PATTERN_LIST", [ pttrn; Leaf { token_type = "semicolon" }; rest ])
453   ->
454     pattern pttrn :: a_pattern_list rest
455 | _ -> failwith "Not an A pattern list"
456 and pattern_tuple (tree : (string, string) syntax_tree) : pattern list =
457 match tree with
458 | Node ("PATTERN", [ rest; Leaf { token_type = "comma" }; last ]) ->
459   pattern_tuple rest @ [ pattern last ]
460 | _ -> [ pattern tree ]
461 and pattern_or (tree : (string, string) syntax_tree) : pattern list =
462 match tree with
463 | Node ("PATTERN", [ rest; Leaf { token_type = "pipe" }; last ]) ->
464   pattern_or rest @ [ pattern last ]
465 | _ -> [ pattern tree ]
466 and pattern (tree : (string, string) syntax_tree) : pattern =
467 match tree with
468 | Node ("PATTERN", [ Leaf { token_type = "lowercase_ident"; value } ]) ->
469   Ident value
470 | Node ("PATTERN", [ Leaf { token_type = "underscore" } ]) -> Underscore
471 | Node
472   ( "PATTERN",
473     [
474       Leaf { token_type = "open_paren" };
475       inner;

```

```

476         Leaf { token_type = "close_paren" };
477     ] ) ->
478     Parenthesised (pattern inner)
479 | Node
480   ( "PATTERN",
481     [
482       Leaf { token_type = "open_paren" };
483       inner;
484       Leaf { token_type = "colon" };
485       typ_expr;
486       Leaf { token_type = "close_paren" };
487     ] ) ->
488     TypeCoercion { inner = pattern inner; typ = type_expression typ_expr }
489 | Node ("PATTERN", [ (Node ("CONSTANT", _) as cst) ]) ->
490     let cst_node = constant cst in
491     Constant cst_node
492 | Node
493   ( "PATTERN",
494     [
495       Leaf { token_type = "open_brace" };
496       fields;
497       Leaf { token_type = "close_brace" };
498     ] ) ->
499     Record (field_pattern_list fields)
500 | Node
501   ( "PATTERN",
502     [
503       Leaf { token_type = "open_bracket" };
504       patterns;
505       Leaf { token_type = "close_bracket" };
506     ] ) ->
507     List (a_pattern_list patterns)
508 | Node ("PATTERN", [ constr; ptrn ]) ->
509     let constr_node = constructor constr in

```


5 – Annexes • 5.6 Code

```

510         let pptrn_node = pattern pptrn in
511
512         Construction { constructor = constr_node; argument = pptrn_node }
513 | Node ("PATTERN", [ first; Leaf { token_type = "double_colon" }; rest ]) ->
514     let first_node = pattern first in
515     let rest_node = pattern rest in
516     Concatenation { head = first_node; tail = rest_node }
517 | Node ("PATTERN", [ _; Leaf { token_type = "comma" }; _ ]) ->
518     let pattern_nodes = pattern_tuple tree in
519     Tuple pattern_nodes
520 | Node ("PATTERN", [ _; Leaf { token_type = "pipe" }; _ ]) ->
521     let pattern_nodes = pattern_or tree in
522     Or pattern_nodes
523 | Node
524     ( "PATTERN",
525     [
526         inner;
527         Leaf { token_type = "as" };
528         Leaf { token_type = "lowercase_ident"; value };
529     ] ) ->
530     let inner_node = pattern inner in
531     As { inner = inner_node; name = value }
532 | _ -> failwith "Not a pattern"
533 and simple_matching (tree : (string, string) syntax_tree) : pattern_cases =
534 match tree with
535 | Node
536     ("SIMPLE_MATCHING", [ pptrn; Leaf { token_type = "right_arrow" }; expr ])
537     ->
538     let pptrn_node = pattern pptrn in
539     let expr_node = expression expr in
540     [ ([ pptrn_node ], expr_node) ]
541 | Node
542     ( "SIMPLE_MATCHING",
543     [

```

5 – Annexes • 5.6 Code

```

544         pttrn;
545         Leaf { token_type = "right_arrow" };
546         expr;
547         Leaf { token_type = "pipe" };
548         other_cases;
549     ] ) ->
550     let other_cases_nodes = simple_matching other_cases in
551     let pttrn_node = pattern pttrn in
552     let expr_node = expression expr in
553     ([ pttrn_node ], expr_node) :: other_cases_nodes
554 | _ -> failwith "Not a simple matching"
and b_pattern_list (tree : (string, string) syntax_tree) : pattern list =
555 match tree with
556 | Node ("B_PATTERN_LIST", [ inner ]) -> [ pattern inner ]
557 | Node ("B_PATTERN_LIST", [ inner; rest ]) ->
558     pattern inner :: b_pattern_list rest
559 | _ -> failwith "Not a B pattern list"
and multiple_matching (tree : (string, string) syntax_tree) : pattern_cases =
560 match tree with
561 | Node
562     ( "MULTIPLE_MATCHING",
563       [ patterns; Leaf { token_type = "right_arrow" }; expr ] ) ->
564     let pattern_nodes = b_pattern_list patterns in
565     let expr_node = expression expr in
566     [ (pattern_nodes, expr_node) ]
567 | Node
568     ( "MULTIPLE_MATCHING",
569       [
570         patterns;
571         Leaf { token_type = "right_arrow" };
572         expr;
573         Leaf { token_type = "pipe" };
574         rest;
575       ] ) ->

```

5 – Annexes • 5.6 Code

```

578     let rest_node = multiple_matching rest in
579     let pattern_nodes = b_pattern_list patterns in
580     let expr_node = expression expr in
581     (pattern_nodes, expr_node) :: rest_node
582 | _ -> failwith "Not a multiple matching"
583 and let_binding (tree : (string, string) syntax_tree) : binding =
584   match tree with
585   | Node ("LET_BINDING", [ pptrn; Leaf { token_type = "eq" }; expr ]) ->
586     let pptrn_node = pattern pptrn in
587     let expr_node = expression expr in
588     Variable { lhs = pptrn_node; value = expr_node }
589   | Node ("LET_BINDING", [ name; arguments; Leaf { token_type = "eq" }; expr ])
590   ->
591     let name_node = variable name in
592     let argument_nodes = b_pattern_list arguments in
593     let expr_node = expression expr in
594
595     Function
596     {
597       name = name_node;
598       parameters = argument_nodes;
599       body = expr_node;
600       return_type = None;
601     }
602   | Node
603     ( "LET_BINDING",
604       [
605         name;
606         arguments;
607         Leaf { token_type = "colon" };
608         return_type;
609         Leaf { token_type = "eq" };
610         expr;
611       ] ) ->

```

5 – Annexes • 5.6 Code

```

612     let name_node = variable name in
613     let argument_nodes = b_pattern_list arguments in
614     let expr_node = expression expr in
615     let return_type_node = type_expression return_type in
616
617     Function
618     {
619         name = name_node;
620         parameters = argument_nodes;
621         body = expr_node;
622         return_type = Some return_type_node;
623     }
624 | _ -> failwith "Not a let binding"
625 and let_binding_list (tree : (string, string) syntax_tree) : binding list =
626     match tree with
627     | Node ("LET_BINDING_LIST", [ b ]) -> [ let_binding b ]
628     | Node ("LET_BINDING_LIST", [ b; Leaf { token_type = "and" }; rest ]) ->
629         let_binding b :: let_binding_list rest
630 | _ -> failwith "Not a binding list"
631 and expression_sequence (tree : (string, string) syntax_tree) :
632     expression list =
633     match tree with
634     | Node
635         ( "EXPR",
636         [
637             (Node ("EXPR", _) as e1);
638             Leaf { token_type = "semicolon" };
639             (Node ("EXPR", _) as e2);
640         ] ) ->
641         expression e1 :: expression_sequence e2
642     | Node
643         ("EXPR", [ (Node ("EXPR", _) as e1); Leaf { token_type = "semicolon" } ])
644     ->
645         [ expression e1 ]

```

5 – Annexes • 5.6 Code

```

646 | _ -> [ expression tree ]
647 and field_expression (tree : (string, string) syntax_tree) :
648   label * expression =
649   match tree with
650   | Node ("FIELD_EXPR", [ name; Leaf { token_type = "eq" }; value ]) ->
651     (label name, expression value)
652   | _ -> failwith "Not a field expression"
653 and field_expression_sequence (tree : (string, string) syntax_tree) :
654   (label * expression) list =
655   match tree with
656   | Node ("FIELD_EXPR_LIST", [ expr ]) -> [ field_expression expr ]
657   | Node ("FIELD_EXPR_LIST", [ expr; Leaf { token_type = "semicolon" } ]) ->
658     [ field_expression expr ]
659   | Node ("FIELD_EXPR_LIST", [ expr; Leaf { token_type = "semicolon" }; rest ]) ->
660     field_expression expr :: field_expression_sequence rest
661   | _ -> failwith "Not a field expression list"
662 and adjacent_expressions (tree : (string, string) syntax_tree) :
663   expression list =
664   match tree with
665   | Node ("EXPR", [ (Node ("EXPR", _) as e1); (Node ("EXPR", _) as e2) ]) ->
666     adjacent_expressions e1 @ [ expression e2 ]
667   | _ -> [ expression tree ]
668 and expression_tuple (tree : (string, string) syntax_tree) : expression list =
669   match tree with
670   | Node
671     ( "EXPR",
672       [
673         (Node ("EXPR", _) as e1);
674         Leaf { token_type = "comma" };
675         (Node ("EXPR", _) as e2);
676       ] ) ->
677     expression_tuple e1 @ [ expression e2 ]
678   | _ -> [ expression tree ]

```

5 – Annexes • 5.6 Code

```

680 and expression (tree : (string, string) syntax_tree) : expression =
681   match tree with
682   | Node ("EXPR", [ (Node ("VARIABLE", _) as var) ]) ->
683     let var_node = variable var in
684     Variable var_node
685   | Node ("EXPR", [ (Node ("CONSTANT", _) as cst) ]) ->
686     let cst_node = constant cst in
687     Constant cst_node
688   | Node
689     ( "EXPR",
690       [
691         Leaf { token_type = "open_paren" };
692         inner;
693         Leaf { token_type = "close_paren" };
694       ] ) ->
695     let inner_node = expression inner in
696     Parenthesised { style = Parenthesis; inner = inner_node }
697   | Node
698     ( "EXPR",
699       [ Leaf { token_type = "begin" }; inner; Leaf { token_type = "end" } ]
700     ) ->
701     let inner_node = expression inner in
702     Parenthesised { style = BeginEnd; inner = inner_node }
703   | Node
704     ( "EXPR",
705       [
706         Leaf { token_type = "open_paren" };
707         inner;
708         Leaf { token_type = "colon" };
709         typ_expr;
710         Leaf { token_type = "close_paren" };
711       ] ) ->
712     let inner_node = expression inner in
713     let typ_expr_node = type_expression typ_expr in

```

```

714     TypeCoercion { inner = inner_node; typ = typ_expr_node }
715 | Node
716   ( "EXPR",
717     [
718       Leaf { token_type = "open_bracket" };
719       inner;
720       Leaf { token_type = "close_bracket" };
721     ] ) ->
722   let inner_nodes = expression_sequence inner in
723   ListLiteral inner_nodes
724 | Node
725   ( "EXPR",
726     [
727       Leaf { token_type = "open_bracket_bar" };
728       inner;
729       Leaf { token_type = "close_bracket_bar" };
730     ] ) ->
731   let inner_nodes = expression_sequence inner in
732   ArrayLiteral inner_nodes
733 | Node
734   ( "EXPR",
735     [
736       Leaf { token_type = "open_brace" };
737       inner;
738       Leaf { token_type = "close_brace" };
739     ] ) ->
740   let inner_nodes = field_expression_sequence inner in
741   RecordLiteral inner_nodes
742 | Node
743   ( "EXPR",
744     [
745       Leaf { token_type = "while" };
746       condition;
747       Leaf { token_type = "do" };

```

5 – Annexes • 5.6 Code

```

748         body;
749         Leaf { token_type = "done" };
750     ] ) ->
751     let condition_node = expression condition in
752     let body_node = expression body in
753     WhileLoop { body = body_node; condition = condition_node }
754 | Node
755   ( "EXPR",
756     [
757         Leaf { token_type = "for" };
758         Leaf { value = var_name };
759         Leaf { token_type = "eq" };
760         start;
761         Leaf { token_type = "to" };
762         finish;
763         Leaf { token_type = "do" };
764         body;
765         Leaf { token_type = "done" };
766     ] ) ->
767     let start_node = expression start in
768     let finish_node = expression finish in
769     let body_node = expression body in
770
771     ForLoop
772     {
773         direction = Up;
774         body = body_node;
775         start = start_node;
776         finish = finish_node;
777         variable = var_name;
778     }
779 | Node
780   ( "EXPR",
781     [

```


5 – Annexes • 5.6 Code

```

782         Leaf { token_type = "for" };
783         Leaf { value = var_name };
784         Leaf { token_type = "eq" };
785         start;
786         Leaf { token_type = "downtn" };
787         finish;
788         Leaf { token_type = "do" };
789         body;
790         Leaf { token_type = "done" };
791     ] ) ->
792     let start_node = expression start in
793     let finish_node = expression start in
794     let body_node = expression body in
795
796     ForLoop
797     {
798         direction = Down;
799         body = body_node;
800         start = start_node;
801         finish = finish_node;
802         variable = var_name;
803     }
804 | Node ("EXPR", [ Leaf { token_type = "bang" }; inner ]) ->
805     let inner_node = expression inner in
806     Dereference inner_node
807 | Node ("EXPR", [ receiver; Leaf { token_type = "dot" }; target ]) ->
808     let receiver_node = expression receiver in
809     let target_node = label target in
810     FieldAccess { receiver = receiver_node; target = target_node }
811 | Node
812   ( "EXPR",
813     [
814         receiver;
815         Leaf { token_type = "dot_paren" };

```

5 – Annexes • 5.6 Code

```

816         target;
817         Leaf { token_type = "close_paren" };
818     ] ) ->
819     let receiver_node = expression receiver in
820     let target_node = expression target in
821     ArrayAccess { receiver = receiver_node; target = target_node }
822 | Node ("EXPR", [ Node ("EXPR", _); Node ("EXPR", _) ]) ->
823     let chain = adjacent_expressions tree in
824     let receiver = List.hd chain in
825     let arguments = List.tl chain in
826     FunctionApplication { receiver; arguments }
827 | Node
828     ( "EXPR",
829     [ Leaf { token_type = ("minus" | "minus_dot") as operation }; inner ]
830     ) ->
831     let inner_node = expression inner in
832
833     PrefixOperation
834     {
835         operation =
836         (match operation with
837         | "minus" -> Minus
838         | "minus_dot" -> MinusDot
839         | _ -> failwith ("Unmatched operation: " ^ operation));
840         receiver = inner_node;
841     }
842 | Node
843     ( "EXPR",
844     [
845         lhs;
846         Leaf
847         {
848             token_type =
849             ( "double_star" | "mod" | "star" | "star_dot" | "slash"

```

5 – Annexes • 5.6 Code

```

850         | "slash_dot" | "plus" | "plus_dot" | "minus" | "minus_dot"
851         | "double_colon" | "at" | "caret" | "eq" | "neq" | "double_eq"
852         | "bang_equals" | "less" | "leq" | "greater" | "geq"
853         | "less_dot" | "leq_dot" | "greater_dot" | "geq_dot"
854         | "ampersand" | "or" | "double_ampersand" | "double_pipe" ) as
855     operation;
856 };
857 rhs;
858 ] ) ->
859 let lhs_node = expression lhs in
860 let rhs_node = expression rhs in
861 InfixOperation
862 {
863     operation =
864     (match operation with
865     | "double_star" -> DoubleStar
866     | "mod" -> Mod
867     | "star" -> Star
868     | "star_dot" -> StarDot
869     | "slash" -> Slash
870     | "slash_dot" -> SlashDot
871     | "plus" -> Plus
872     | "plus_dot" -> PlusDot
873     | "minus" -> Minus
874     | "minus_dot" -> MinusDot
875     | "double_colon" -> DoubleColon
876     | "at" -> At
877     | "caret" -> Caret
878     | "eq" -> Eq
879     | "neq" -> Neq
880     | "double_eq" -> DoubleEq
881     | "bang_equals" -> BangEq
882     | "less" -> Less
883     | "leq" -> Leq

```

5 – Annexes • 5.6 Code

```

884         | "greater" -> Greater
885         | "geq" -> Geq
886         | "less_dot" -> LessDot
887         | "leq_dot" -> LeqDot
888         | "greater_dot" -> GreaterDot
889         | "geq_dot" -> GeqDot
890         | "ampersand" -> Ampersand
891         | "or" -> Or
892         | "double_ampersand" -> DoubleAmpersand
893         | "double_pipe" -> DoublePipe
894         | _ -> failwith ("Unmatched operation: " ^ operation));
895     lhs = lhs_node;
896     rhs = rhs_node;
897 }
898 | Node ("EXPR", [ Leaf { token_type = "not" }; inner ]) ->
899     let inner_node = expression inner in
900     Negation inner_node
901 | Node ("EXPR", [ _ ; Leaf { token_type = "comma" }; _ ]) ->
902     let expressions = expression_tuple tree in
903     Tuple expressions
904 | Node
905     ( "EXPR",
906     [
907         receiver;
908         Leaf { token_type = "dot" };
909         target;
910         Leaf { token_type = "left_arrow" };
911         value;
912     ] ) ->
913     let receiver_node = expression receiver in
914     let target_node = label target in
915     let value_node = expression value in
916
917     FieldAssignment

```

5 – Annexes • 5.6 Code

```

918         { value = value_node; target = target_node; receiver = receiver_node }
919 | Node
920   ( "EXPR",
921     [
922       receiver;
923       Leaf { token_type = "dot_paren" };
924       target;
925       Leaf { token_type = "close_paren" };
926       Leaf { token_type = "left_arrow" };
927       value;
928     ] ) ->
929   let receiver_node = expression receiver in
930   let target_node = expression target in
931   let value_node = expression value in
932
933   ArrayAssignment
934     { value = value_node; target = target_node; receiver = receiver_node }
935 | Node ("EXPR", [ receiver; Leaf { token_type = "colon_equals" }; value ])
936   ->
937   let receiver_node = expression receiver in
938   let value_node = expression value in
939
940   ReferenceAssignment { receiver = receiver_node; value = value_node }
941 | Node
942   ( "EXPR",
943     [
944       Leaf { token_type = "if" };
945       condition;
946       Leaf { token_type = "then" };
947       body;
948     ] ) ->
949   let condition_node = expression condition in
950   let body_node = expression body in
951

```

```

952     If { else_body = None; body = body_node; condition = condition_node }
953 | Node
954   ( "EXPR",
955     [
956       Leaf { token_type = "if" };
957       condition;
958       Leaf { token_type = "then" };
959       body;
960       Leaf { token_type = "else" };
961       else_body;
962     ] ) ->
963   let condition_node = expression condition in
964   let body_node = expression body in
965   let else_body_node = expression else_body in
966
967   If
968     {
969       else_body = Some else_body_node;
970       body = body_node;
971       condition = condition_node;
972     }
973 | Node ("EXPR", [ _; Leaf { token_type = "semicolon" }; _ ]) ->
974   let sequence = expression_sequence tree in
975   Sequence sequence
976 | Node
977   ( "EXPR",
978     [
979       Leaf { token_type = "let" };
980       bindings;
981       Leaf { token_type = "in" };
982       inner;
983     ] ) ->
984   let inner_node = expression inner in
985   let bindings_nodes = let_binding_list bindings in

```

```

986
987     LetBinding
988     { is_rec = false; inner = inner_node; bindings = bindings_nodes }
989 | Node
990   ( "EXPR",
991     [
992       Leaf { token_type = "let" };
993       Leaf { token_type = "rec" };
994       bindings;
995       Leaf { token_type = "in" };
996       inner;
997     ] ) ->
998   let inner_node = expression inner in
999   let bindings_nodes = let_binding_list bindings in
1000
1001   LetBinding
1002   { is_rec = true; inner = inner_node; bindings = bindings_nodes }
1003 | Node
1004   ( "EXPR",
1005     [
1006       Leaf { token_type = "match" };
1007       value;
1008       Leaf { token_type = "with" };
1009       pattern_cases;
1010     ] ) ->
1011   let value_node = expression value in
1012   let pattern_case_nodes = simple_matching pattern_cases in
1013   Match { value = value_node; cases = pattern_case_nodes }
1014 | Node
1015   ( "EXPR",
1016     [
1017       Leaf { token_type = "try" };
1018       value;
1019       Leaf { token_type = "with" };

```

```

1020         pattern_cases;
1021     ] ) ->
1022     let value_node = expression value in
1023     let pattern_case_nodes = simple_matching pattern_cases in
1024     Try { value = value_node; cases = pattern_case_nodes }
1025 | Node ("EXPR", [ Leaf { token_type = "fun" }; pattern_cases ]) ->
1026     let pattern_case_nodes = multiple_matching pattern_cases in
1027     FunctionLiteral { style = Fun; cases = pattern_case_nodes }
1028 | Node ("EXPR", [ Leaf { token_type = "function" }; pattern_cases ]) ->
1029     let pattern_case_nodes = multiple_matching pattern_cases in
1030
1031     FunctionLiteral { style = Function; cases = pattern_case_nodes }
1032 | Node
1033   ( "EXPR",
1034     [
1035       receiver;
1036       Leaf { token_type = "dot_open_bracket" };
1037       target;
1038       Leaf { token_type = "close_bracket" };
1039     ] ) ->
1040     let receiver_node = expression receiver in
1041     let target_node = expression target in
1042
1043     StringAccess { receiver = receiver_node; target = target_node }
1044 | Node
1045   ( "EXPR",
1046     [
1047       receiver;
1048       Leaf { token_type = "dot_open_bracket" };
1049       target;
1050       Leaf { token_type = "close_bracket" };
1051       Leaf { token_type = "left_arrow" };
1052       value;
1053     ] ) ->

```



```

1054     let receiver_node = expression receiver in
1055     let target_node = expression target in
1056     let value_node = expression value in
1057
1058     StringAssignment
1059     { value = value_node; target = target_node; receiver = receiver_node }
1060 | Node ("EXPR", [ _; Leaf { token_type = "semicolon" } ]) ->
1061     let sequence = expression_sequence tree in
1062     Sequence sequence
1063 | Node
1064     ( "EXPR",
1065     [
1066         Leaf { token_type = "match" };
1067         value;
1068         Leaf { token_type = "with" };
1069         Leaf { token_type = "pipe" };
1070         pattern_cases;
1071     ] ) ->
1072     let value_node = expression value in
1073     let pattern_case_nodes = simple_matching pattern_cases in
1074     Match { value = value_node; cases = pattern_case_nodes }
1075 | Node
1076     ( "EXPR",
1077     [
1078         Leaf { token_type = "try" };
1079         value;
1080         Leaf { token_type = "with" };
1081         Leaf { token_type = "pipe" };
1082         pattern_cases;
1083     ] ) ->
1084     let value_node = expression value in
1085     let pattern_case_nodes = simple_matching pattern_cases in
1086     Try { value = value_node; cases = pattern_case_nodes }
1087 | Node

```

```

1088     ( "EXPR",
1089     [
1090         Leaf { token_type = "fun" };
1091         Leaf { token_type = "pipe" };
1092         pattern_cases;
1093     ] ) ->
1094     let pattern_case_nodes = multiple_matching pattern_cases in
1095     FunctionLiteral { style = Fun; cases = pattern_case_nodes }
1096 | Node
1097     ( "EXPR",
1098     [
1099         Leaf { token_type = "function" };
1100         Leaf { token_type = "pipe" };
1101         pattern_cases;
1102     ] ) ->
1103     let pattern_case_nodes = multiple_matching pattern_cases in
1104
1105     FunctionLiteral { style = Function; cases = pattern_case_nodes }
1106 | _ -> failwith "Not an expression"
1107 and value_definition (tree : (string, string) syntax_tree) : value_definition
1108 =
1109 match tree with
1110 | Node ("VALUE_DEFINITION", [ Leaf { token_type = "let" }; bindings ]) ->
1111     let binding_nodes = let_binding_list bindings in
1112     { bindings = binding_nodes; is_rec = false }
1113 | Node
1114     ( "VALUE_DEFINITION",
1115     [ Leaf { token_type = "let" }; Leaf { token_type = "rec" }; bindings ]
1116     ) ->
1117     let binding_nodes = let_binding_list bindings in
1118     { bindings = binding_nodes; is_rec = true }
1119 | _ -> failwith "Not a value definition"
1120 and constructor_declaration (tree : (string, string) syntax_tree) :
1121     type_constructor_declaration =

```

```

1122     match tree with
1123     | Node ("CONSTR_DECL", [ Leaf { token_type = "uppercase_ident"; value } ])
1124     ->
1125         { name = value; parameter = None }
1126     | Node
1127         ( "CONSTR_DECL",
1128         [
1129             Leaf { token_type = "uppercase_ident"; value };
1130             Leaf { token_type = "of" };
1131             argument;
1132         ] ) ->
1133         let argument_node = type_expression argument in
1134         { name = value; parameter = Some argument_node }
1135     | _ -> failwith "Not a type constructor declaration"
1136 and type_constructor_declaration_list (tree : (string, string) syntax_tree) :
1137     type_constructor_declaration list =
1138     match tree with
1139     | Node ("TYPE_CONSTR_DECL_LIST", [ decl ]) ->
1140         [ constructor_declaration decl ]
1141     | Node
1142         ("TYPE_CONSTR_DECL_LIST", [ decl; Leaf { token_type = "pipe" }; rest ])
1143     ->
1144         constructor_declaration decl :: type_constructor_declaration_list rest
1145     | _ -> failwith "Not a type constructor declaration list"
1146 and label_declaration (tree : (string, string) syntax_tree) :
1147     field_declaration =
1148     match tree with
1149     | Node
1150         ( "LABEL_DECL",
1151         [
1152             Leaf { token_type = "lowercase_ident"; value };
1153             Leaf { token_type = "colon" };
1154             typ_expr;
1155         ] ) ->

```

```

1156     let typ_expr_node = type_expression typ_expr in
1157
1158     { name = value; typ = typ_expr_node; is_mutable = false }
1159 | Node
1160   ( "LABEL_DECL",
1161     [
1162       Leaf { token_type = "mutable" };
1163       Leaf { token_type = "lowercase_ident"; value };
1164       Leaf { token_type = "colon" };
1165       typ_expr;
1166     ] ) ->
1167   let typ_expr_node = type_expression typ_expr in
1168
1169   { name = value; typ = typ_expr_node; is_mutable = true }
1170 | _ -> failwith "Not a label declaration"
and label_declaration_list (tree : (string, string) syntax_tree) :
  field_declaration list =
1172   match tree with
1173   | Node ("LABEL_DECL_LIST", [ decl ]) -> [ label_declaration decl ]
1174   | Node ("LABEL_DECL_LIST", [ decl; Leaf { token_type = "semicolon" }; rest ])
1175     ->
1176       label_declaration decl :: label_declaration_list rest
1177   | _ -> failwith "Not a label declaration"
and type_parameter_list (tree : (string, string) syntax_tree) :
  lowercase_ident list =
1180   match tree with
1181   | Node
1182     ( "TYPE_PARAM_LIST",
1183       [
1184         Leaf { token_type = "apostrophe" };
1185         Leaf { token_type = "lowercase_ident"; value };
1186       ] ) ->
1187     [ value ]
1188   | Node

```

5 – Annexes • 5.6 Code

```

1190     ( "TYPE_PARAM_LIST",
1191     [
1192         Leaf { token_type = "apostrophe" };
1193         Leaf { token_type = "lowercase_ident"; value };
1194         Leaf { token_type = "comma" };
1195         rest;
1196     ] ) ->
1197     value :: type_parameter_list rest
1198 | _ -> failwith "Not a type parameter list"
and type_parameters (tree : (string, string) syntax_tree) :
1200     lowercase_ident list * bool =
1201     match tree with
1202     | Node
1203       ( "TYPE_PARAMS",
1204       [
1205         Leaf { token_type = "apostrophe" };
1206         Leaf { token_type = "lowercase_ident"; value };
1207       ] ) ->
1208       ([ value ], false)
1209     | Node
1210       ( "TYPE_PARAMS",
1211       [
1212         Leaf { token_type = "open_paren" };
1213         list;
1214         Leaf { token_type = "close_paren" };
1215       ] ) ->
1216       (type_parameter_list list, true)
1217 | _ -> failwith "Not a type parameter list"
and typedef (tree : (string, string) syntax_tree) : typedef =
1219     match tree with
1220     | Node
1221       ( "TYPEDEF",
1222       [
1223         Leaf { token_type = "lowercase_ident"; value };

```

```

1224         Leaf { token_type = "eq" };
1225         (Node ( "TYPE_CONSTR_DECL_LIST", _) as constructor_declarations);
1226     ] ) ->
1227     let constructor_declaration_nodes =
1228         type_constructor_declaration_list constructor_declarations
1229     in
1230
1231     Constructors
1232     {
1233         name = value;
1234         parameters = [];
1235         parenthesised_parameters = false;
1236         constructors = constructor_declaration_nodes;
1237     }
1238 | Node
1239     ( "TYPEDEF",
1240     [
1241         Leaf { token_type = "lowercase_ident"; value };
1242         Leaf { token_type = "eq" };
1243         Leaf { token_type = "open_brace" };
1244         label_decl_list;
1245         Leaf { token_type = "close_brace" };
1246     ] ) ->
1247     Record
1248     {
1249         name = value;
1250         parameters = [];
1251         parenthesised_parameters = false;
1252         fields = label_declaration_list label_decl_list;
1253     }
1254 | Node
1255     ( "TYPEDEF",
1256     [
1257         Leaf { token_type = "lowercase_ident"; value };

```

5 – Annexes • 5.6 Code

```

1258         Leaf { token_type = "double_eq" };
1259         typ_expr;
1260     ] ) ->
1261     Alias
1262     {
1263         name = value;
1264         parameters = [];
1265         parenthesised_parameters = false;
1266         style = DoubleEq;
1267         aliased = type_expression typ_expr;
1268     }
1269 | Node ("TYPEDEF", [ Leaf { token_type = "lowercase_ident"; value } ]) ->
1270     Anonymous
1271     { name = value; parameters = []; parenthesised_parameters = false }
1272 | Node
1273     ( "TYPEDEF",
1274     [
1275         parameters;
1276         Leaf { token_type = "lowercase_ident"; value };
1277         Leaf { token_type = "eq" };
1278         (Node ("TYPE_CONSTR_DECL_LIST", _) as constructor_declarations);
1279     ] ) ->
1280     let parameters, parenthesised_parameters = type_parameters parameters in
1281     let constructor_declaration_nodes =
1282         type_constructor_declaration_list constructor_declarations
1283     in
1284
1285     Constructors
1286     {
1287         name = value;
1288         parameters;
1289         parenthesised_parameters;
1290         constructors = constructor_declaration_nodes;
1291     }

```

```

1292 | Node
1293 | ( "TYPEDEF",
1294 | [
1295 |     parameters;
1296 |     Leaf { token_type = "lowercase_ident"; value };
1297 |     Leaf { token_type = "eq" };
1298 |     Leaf { token_type = "open_brace" };
1299 |     label_decl_list;
1300 |     Leaf { token_type = "close_brace" };
1301 | ] ) ->
1302 | let parameters, parenthesised_parameters = type_parameters parameters in
1303 |
1304 | Record
1305 | {
1306 |     name = value;
1307 |     parameters;
1308 |     parenthesised_parameters;
1309 |     fields = label_declaration_list label_decl_list;
1310 | }
1311 | Node
1312 | ( "TYPEDEF",
1313 | [
1314 |     parameters;
1315 |     Leaf { token_type = "lowercase_ident"; value };
1316 |     Leaf { token_type = "double_eq" };
1317 |     typ_expr;
1318 | ] ) ->
1319 | let parameters, parenthesised_parameters = type_parameters parameters in
1320 |
1321 | Alias
1322 | {
1323 |     name = value;
1324 |     parameters;
1325 |     parenthesised_parameters;

```


5 – Annexes • 5.6 Code

```

1326         style = DoubleEq;
1327         aliased = type_expression typ_expr;
1328     }
1329 | Node
1330   ( "TYPEDEF",
1331     [ parameters; Leaf { token_type = "lowercase_ident"; value } ] ) ->
1332   let parameters, parenthesised_parameters = type_parameters parameters in
1333
1334   Anonymous { name = value; parameters; parenthesised_parameters }
1335 | Node
1336   ( "TYPEDEF",
1337     [
1338       Leaf { token_type = "lowercase_ident"; value };
1339       Leaf { token_type = "eq" };
1340       typ_expr;
1341     ] ) ->
1342   Alias
1343   {
1344     name = value;
1345     parameters = [];
1346     parenthesised_parameters = false;
1347     style = SingleEq;
1348     aliased = type_expression typ_expr;
1349   }
1350 | Node
1351   ( "TYPEDEF",
1352     [
1353       parameters;
1354       Leaf { token_type = "lowercase_ident"; value };
1355       Leaf { token_type = "eq" };
1356       typ_expr;
1357     ] ) ->
1358   let parameters, parenthesised_parameters = type_parameters parameters in
1359

```

5 – Annexes • 5.6 Code

```

1360     Alias
1361     {
1362         name = value;
1363         parameters;
1364         parenthesised_parameters;
1365         style = SingleEq;
1366         aliased = type_expression typ_expr;
1367     }
1368 | _ -> failwith "Not a typedef"
1369 and typedef_list (tree : (string, string) syntax_tree) : typedef list =
1370     match tree with
1371     | Node ("TYPEDEF_LIST", [ typedef ]) -> [ typedef typedef ]
1372     | Node ("TYPEDEF_LIST", [ typedef; Leaf { token_type = "and" }; rest ]) ->
1373         typedef typedef :: typedef_list rest
1374     | _ -> failwith "Not a typedef list"
1375 and type_definition (tree : (string, string) syntax_tree) : type_definition =
1376     match tree with
1377     | Node ("TYPE_DEFINITION", [ Leaf { token_type = "type" }; typedef_list ]) ->
1378         typedef_list typedef_list
1379     | _ -> failwith "Not a type definition"
1380 and exception_constructor_declaration_list
1381     (tree : (string, string) syntax_tree) : type_constructor_declaration list
1382     =
1383     match tree with
1384     | Node ("EXCEPTION_CONSTR_DECL_LIST", [ decl ]) ->
1385         [ constructor_declaration decl ]
1386     | Node
1387         ( "EXCEPTION_CONSTR_DECL_LIST",
1388           [ decl; Leaf { token_type = "and" }; rest ] ) ->
1389         constructor_declaration decl
1390           :: exception_constructor_declaration_list rest
1391     | _ -> failwith "Not a type constructor declaration list"
1392 and exception_definition (tree : (string, string) syntax_tree) :
1393     exception_definition =

```

```

1394     match tree with
1395     | Node ("EXCEPTION_DEFINITION", [ Leaf { token_type = "exception" }; list ])
1396     ->
1397         let list_node_s = exception_constructor_declaration_list list in
1398         { exceptions = list_node_s }
1399     | _ -> failwith "Not an exception definition"
1400 in
1401
1402 let implementation_phrase (tree : (string, string) syntax_tree) : phrase =
1403     match tree with
1404     | Node ("IMPL_PHRASE", [ (Node ("EXPR", _) as expr) ]) ->
1405         let expr_node = expression expr in
1406         Expression expr_node
1407     | Node ("IMPL_PHRASE", [ (Node ("VALUE_DEFINITION", _) as val_def) ]) ->
1408         let val_def_node = value_definition val_def in
1409         ValueDefinition val_def_node
1410     | Node ("IMPL_PHRASE", [ (Node ("TYPE_DEFINITION", _) as typ_def) ]) ->
1411         let typ_def_node = type_definition typ_def in
1412         TypeDefinition typ_def_node
1413     | Node ("IMPL_PHRASE", [ (Node ("EXCEPTION_DEFINITION", _) as exc_def) ]) ->
1414         let exc_def_node = exception_definition exc_def in
1415         ExceptionDefinition exc_def_node
1416     | _ -> failwith "Not an implementation phrase"
1417 in
1418
1419 match tree with
1420 | Node ("IMPLEMENTATION", [ phrase; Leaf { token_type = "double_semicolon" } ])
1421 | Node ("IMPLEMENTATION", [ phrase ]) ->
1422     let phrase_node = implementation_phrase phrase in
1423     [ phrase_node ]
1424 | Node
1425     ( "IMPLEMENTATION",
1426       [ phrase; Leaf { token_type = "double_semicolon" }; remaining ] )
1427 | Node ("IMPLEMENTATION", [ phrase; remaining ]) ->

```

5 – Annexes • 5.6 Code

```

1428     let phrase_node = implementation_phrase phrase in
1429     let remaining_node = ast_of_syntax_tree remaining in
1430     phrase_node :: remaining_node
1431 | _ -> failwith "Not an implementation"
1432
1433 (** Converts a Caml Light AST back into Caml Light source. sink is called with
1434 string fragments which, when concatenated in the order they are passed to
1435 sink, form the program source. *)
1436 let stringify_ast_into (ast : program) (sink : string -> unit) : unit =
1437 (* Required so OCaml does not instantiate 'a. *)
1438 let rec iter_with_join : 'a. ('a -> unit) -> string -> 'a list -> unit =
1439     fun f s l ->
1440         match l with
1441         | [] -> ()
1442         | x :: [] -> f x
1443         | x :: q ->
1444             f x;
1445             sink s;
1446             iter_with_join f s q
1447 in
1448
1449 let rec handle_type_expression (e : type_expression) =
1450     match e with
1451     | Argument n ->
1452         sink "'";
1453         sink n
1454     | Parenthesised e ->
1455         sink "(";
1456         handle_type_expression e;
1457         sink ")"
1458     | Construction c ->
1459         (match c.arguments with
1460         | [] -> ()
1461         | [ e ] -> handle_type_expression e

```

5 – Annexes • 5.6 Code

```

1462 | _ ->
1463 |   sink "(";
1464 |   iter_with_join handle_type_expression ", " c.arguments;
1465 |   sink ")");
1466 |   sink " ";
1467 |   sink c.constructor
1468 | Tuple t -> iter_with_join handle_type_expression " * " t
1469 | Function f ->
1470 |   handle_type_expression f.argument;
1471 |   sink " -> ";
1472 |   handle_type_expression f.result
1473 in
1474
1475 let handle_constructor (c : constructor) =
1476 match c with
1477 | Named n -> sink n
1478 | EmptyList -> sink "[]"
1479 | Unit style -> sink (if style = Parenthesis then "()" else "begin end")
1480 | EmptyArray -> sink "[|]"
1481 in
1482
1483 let handle_constant (c : constant) =
1484 match c with
1485 | IntegerLiteral i -> sink (string_of_int i)
1486 | FloatLiteral f -> sink (string_of_float f)
1487 | CharacterLiteral c ->
1488   sink (if c.style = New then "" else "~");
1489   sink (string_of_char c.value);
1490   sink (if c.style = New then "" else "~")
1491 | StringLiteral s ->
1492   sink "\"";
1493   sink s;
1494   sink "\"";
1495 | Construction c -> handle_constructor c

```

5 – Annexes • 5.6 Code

```

1496 | True -> sink " true "
1497 | False -> sink " false "
1498 in
1499
1500 let rec handle_pattern (p : pattern) =
1501   match p with
1502   | Ident e -> sink e
1503   | Underscore -> sink "_"
1504   | Parenthesised p ->
1505     sink "(";
1506     handle_pattern p;
1507     sink ")"
1508   | TypeCoercion c ->
1509     sink "(";
1510     handle_pattern c.inner;
1511     sink " : ";
1512     handle_type_expression c.typ;
1513     sink ")"
1514   | Constant c -> handle_constant c
1515   | Record r ->
1516     sink "{";
1517     iter_with_join
1518       (fun (field, pattern) ->
1519         sink field;
1520         sink " = ";
1521         handle_pattern pattern)
1522     "; " r;
1523     sink "}"
1524   | List l ->
1525     sink "[";
1526     iter_with_join handle_pattern "; " l;
1527     sink "]"
1528   | Construction c ->
1529     handle_constructor c.constructor;

```

5 – Annexes • 5.6 Code

```

1530     sink " ";
1531     handle_pattern c.argument
1532 | Concatenation c ->
1533     handle_pattern c.head;
1534     sink " :: ";
1535     handle_pattern c.tail
1536 | Tuple t -> iter_with_join handle_pattern ", " t
1537 | Or o -> iter_with_join handle_pattern " | " o
1538 | As a ->
1539     handle_pattern a.inner;
1540     sink " as ";
1541     sink a.name
1542 in
1543
1544 let rec handle_binding (b : binding) =
1545     match b with
1546 | Variable v ->
1547     handle_pattern v.lhs;
1548     sink " = ";
1549     handle_expression v.value
1550 | Function f ->
1551     sink f.name;
1552     sink " ";
1553     iter_with_join handle_pattern " " f.parameters;
1554     (match f.return_type with
1555     | None -> ()
1556     | Some return_type ->
1557         sink ":";
1558         handle_type_expression return_type);
1559     sink " = ";
1560     handle_expression f.body
1561 and handle_expression (e : expression) =
1562     match e with
1563 | Variable v -> sink v

```

5 – Annexes • 5.6 Code

```

1564 | Constant c -> handle_constant c
1565 | Parenthesised e ->
1566 |   sink (if e.style = Parenthesis then "(" else " begin ");
1567 |   handle_expression e.inner;
1568 |   sink (if e.style = Parenthesis then ")" else " end ")
1569 | TypeCoercion e ->
1570 |   sink "(";
1571 |   handle_expression e.inner;
1572 |   sink " : ";
1573 |   handle_type_expression e.typ;
1574 |   sink ")"
1575 | ListLiteral l ->
1576 |   sink "[";
1577 |   iter_with_join handle_expression ";" l;
1578 |   sink "]"
1579 | ArrayLiteral l ->
1580 |   sink "[";
1581 |   iter_with_join handle_expression ";" l;
1582 |   sink "]"
1583 | RecordLiteral l ->
1584 |   sink "{";
1585 |   iter_with_join
1586 |     (fun (l, e) ->
1587 |       sink l;
1588 |       sink " = ";
1589 |       handle_expression e)
1590 |     ";" l;
1591 |   sink "}"
1592 | WhileLoop l ->
1593 |   sink "while ";
1594 |   handle_expression l.condition;
1595 |   sink " do ";
1596 |   handle_expression l.body;
1597 |   sink " done"

```



```

1598 | ForLoop l ->
1599 |     sink "for ";
1600 |     sink l.variable;
1601 |     sink " = ";
1602 |     handle_expression l.start;
1603 |     sink (if l.direction = Up then " to " else " downto ");
1604 |     handle_expression l.finish;
1605 |     sink " do ";
1606 |     handle_expression l.body;
1607 |     sink " done"
1608 | Dereference e ->
1609 |     sink "!";
1610 |     handle_expression e
1611 | FieldAccess a ->
1612 |     handle_expression a.receiver;
1613 |     sink ".";
1614 |     sink a.target
1615 | ArrayAccess a ->
1616 |     handle_expression a.receiver;
1617 |     sink " . (";
1618 |     handle_expression a.target;
1619 |     sink ")"
1620 | FunctionApplication f ->
1621 |     handle_expression f.receiver;
1622 |     sink " ";
1623 |     iter_with_join handle_expression " " f.arguments
1624 | PrefixOperation o ->
1625 |     sink (match o.operation with Minus -> "-" | MinusDot -> "-.");
1626 |     handle_expression o.receiver
1627 | InfixOperation o ->
1628 |     handle_expression o.lhs;
1629 |     sink
1630 |         (match o.operation with
1631 |         | DoubleStar -> " ** "

```

5 – Annexes • 5.6 Code

```

1632 | Mod -> " mod "
1633 | Star -> " * "
1634 | StarDot -> " *. "
1635 | Slash -> " / "
1636 | SlashDot -> " /. "
1637 | Plus -> " + "
1638 | PlusDot -> " +. "
1639 | Minus -> " - "
1640 | MinusDot -> " -. "
1641 | DoubleColon -> " :: "
1642 | At -> " @ "
1643 | Caret -> " ^ "
1644 | Eq -> " = "
1645 | Neq -> " <> "
1646 | DoubleEq -> " == "
1647 | BangEq -> " != "
1648 | Less -> " < "
1649 | Leq -> " <= "
1650 | Greater -> " > "
1651 | Geq -> " >= "
1652 | LessDot -> " <. "
1653 | LeqDot -> " <=. "
1654 | GreaterDot -> " >. "
1655 | GeqDot -> " >=. "
1656 | Ampersand -> " & "
1657 | Or -> " or "
1658 | DoubleAmpersand -> " && "
1659 | DoublePipe -> " || ");
1660 | handle_expression o.rhs
1661 | Negation n ->
1662 |   sink "not ";
1663 |   handle_expression n
1664 | | Tuple t -> iter_with_join handle_expression ", " t
1665 | | FieldAssignment a ->

```

5 – Annexes • 5.6 Code

```

1666         handle_expression a.receiver;
1667         sink ".";
1668         sink a.target;
1669         sink "<- ";
1670         handle_expression a.value
1671 | ArrayAssignment a ->
1672     handle_expression a.receiver;
1673     sink "(".
1674     handle_expression a.target;
1675     sink ")<- ";
1676     handle_expression a.value
1677 | ReferenceAssignment a ->
1678     handle_expression a.receiver;
1679     sink " := ";
1680     handle_expression a.value
1681 | If i -> (
1682     sink "if ";
1683     handle_expression i.condition;
1684     sink " then ";
1685     handle_expression i.body;
1686     match i.else_body with
1687     | None -> ()
1688     | Some b ->
1689         sink " else ";
1690         handle_expression b)
1691 | Sequence s -> iter_with_join handle_expression "; " s
1692 | Match m ->
1693     sink "match ";
1694     handle_expression m.value;
1695     sink " with ";
1696     iter_with_join
1697         (function
1698         | [ case ], expr ->
1699             handle_pattern case;

```

```

1700         sink " -> ";
1701         handle_expression expr
1702     | _ -> failwith "More than one case in match")
1703     "|" m.cases
1704 | Try t ->
1705     sink "try ";
1706     handle_expression t.value;
1707     sink " with ";
1708     iter_with_join
1709         (function
1710         | [ case ], expr ->
1711             handle_pattern case;
1712             sink " -> ";
1713             handle_expression expr
1714         | _ -> failwith "More than one case in match")
1715     "|" t.cases
1716 | FunctionLiteral f ->
1717     sink (if f.style = Fun then "fun " else "function ");
1718     iter_with_join
1719         (fun (cases, expr) ->
1720             iter_with_join handle_pattern " " cases;
1721             sink " -> ";
1722             handle_expression expr)
1723     "|" f.cases
1724 | LetBinding l ->
1725     sink "let ";
1726     if l.is_rec then sink "rec ";
1727     iter_with_join handle_binding " and " l.bindings;
1728     sink " in ";
1729     handle_expression l.inner
1730 | StringAccess s ->
1731     handle_expression s.target;
1732     sink ".";
1733     handle_expression s.target;

```

```

1734         sink "]"
1735     | StringAssignment a ->
1736         handle_expression a.target;
1737         sink "[";
1738         handle_expression a.target;
1739         sink "]" <- ";
1740         handle_expression a.value
1741 in
1742
1743 let handle_value_definition (v : value_definition) =
1744     sink "let ";
1745     if v.is_rec then sink "rec ";
1746     iter_with_join handle_binding " and " v.bindings
1747 in
1748
1749 let handle_type_parameters (parenthesised : bool) (p : lowercase_ident list) =
1750     if parenthesised then sink "(";
1751     iter_with_join sink ", " (List.map (fun n -> "\"" ^ n) p);
1752     if parenthesised then sink ")";
1753     sink " "
1754 in
1755
1756 let handle_constructor_declaration (c : type_constructor_declaration) =
1757     sink c.name;
1758     match c.parameter with
1759     | None -> ()
1760     | Some p ->
1761         sink " of ";
1762         handle_type_expression p
1763 in
1764
1765 let handle_field_declaration (f : field_declaration) =
1766     if f.is_mutable then sink "mutable ";
1767     sink f.name;

```

5 – Annexes • 5.6 Code

```

1768     sink " : ";
1769     handle_type_expression f.typ
1770 in
1771
1772 let handle_typedef (t : typedef) =
1773     match t with
1774     | Constructors c ->
1775         handle_type_parameters c.parenthesised_parameters c.parameters;
1776         sink c.name;
1777         sink " = ";
1778         iter_with_join handle_constructor_declaration " | " c.constructors
1779     | Record r ->
1780         handle_type_parameters r.parenthesised_parameters r.parameters;
1781         sink r.name;
1782         sink "{";
1783         iter_with_join handle_field_declaration "; " r.fields;
1784         sink "}"
1785     | Alias a ->
1786         handle_type_parameters a.parenthesised_parameters a.parameters;
1787         sink a.name;
1788         sink (if a.style = SingleEq then " = " else " == ");
1789         handle_type_expression a.aliased
1790     | Anonymous a ->
1791         handle_type_parameters a.parenthesised_parameters a.parameters;
1792         sink a.name
1793 in
1794
1795 let handle_type_definition (t : type_definition) =
1796     sink "type ";
1797     iter_with_join handle_typedef " and " t
1798 in
1799
1800 let handle_exception_definition (e : exception_definition) =
1801     sink "exception ";

```

```

1802     iter_with_join handle_constructor_declaration " and " e.exceptions
1803 in
1804
1805 let handle_phrase (phrase : phrase) =
1806   (match phrase with
1807    | Expression e -> handle_expression e
1808    | ValueDefinition v -> handle_value_definition v
1809    | TypeDefinition t -> handle_type_definition t
1810    | ExceptionDefinition e -> handle_exception_definition e);
1811   sink ";;"
1812 in
1813 List.iter handle_phrase ast
1814
1815 (** Converts a Caml Light AST to Caml Light source code. *)
1816 let string_of_ast (ast : program) =
1817   let chain = ref [] in
1818   stringify_ast_into ast (fun piece -> chain := piece :: !chain);
1819   String.concat "" (List.rev !chain)
1820
1821 (** Parses a Caml Light AST from Caml Light source code. *)
1822 let parse_caml_light_ast (source : string) : program =
1823   ast_of_syntax_tree (parse_caml_light_syntax_tree source)

```

grammar_grammar.ml

```
1  open Regex
2  open Lexer
3  open Parser
4  open Regex_grammar
5  open Utils
6
7  type grammar_token_type =
8    | Terminal_pattern
9    | Derivation_symbol
10   | Identifier
11   | Question
12   | Whitespace
13   | Newline
14   | Comment_start
15   | Unrecognizable
16   | Eof
17
18  type grammar_node_type =
19    | Grammar
20    | Grammar_entry
21    | Terminal_definition
22    | Rule_identifier_list
23    | Rule_identifier_list_entry
24    | NonTerminal_definition
25
26  let grammar_token_rules =
27    [
28      ( Concatenation
29        ( Concatenation (Symbol (u ':'), Symbol (u '=')),
30          Regex.Star (Empty |> add_character_range_c ' ' '-') ),
31        Terminal_pattern );
32      ( Concatenation
33        (Concatenation (Symbol (u '-'), Symbol (u '>')), Symbol (u ' ')),
```


5 – Annexes • 5.6 Code

```

34     Derivation_symbol );
35   ( Regex.Concatenation
36     ( Regex.Empty
37       |> add_character_range_c 'A' 'Z'
38       |> add_character_range_c 'a' 'z'
39       |> add_character_range_c '0' '9'
40       |> add_character_c '-',
41       Regex.Star
42         (Regex.Empty
43           |> add_character_range_c 'A' 'Z'
44           |> add_character_range_c 'a' 'z'
45           |> add_character_range_c '0' '9'
46           |> add_character_c '-') ),
47       Identifier );
48   (Concatenation (Symbol (u ' '), Star (Symbol (u ' '))), Whitespace);
49   (Symbol (u '\n'), Newline);
50   (Symbol (u '?'), Question);
51   (Symbol (u '#'), Comment_start);
52   ( Regex.Empty |> add_character_range_c (char_of_int 0) (char_of_int 255),
53     Unrecognizable );
54 ]
55
56 let grammar_rules =
57   grammar_of_rule_list
58   [
59     (Grammar, [ NonTerminal Grammar_entry ]);
60     (Grammar, [ NonTerminal Grammar; NonTerminal Grammar_entry ]);
61     (* Allow empty lines *)
62     (Grammar_entry, [ Terminal Newline ]);
63     (Grammar_entry, [ NonTerminal Terminal_definition; Terminal Newline ]);
64     (Grammar_entry, [ NonTerminal NonTerminal_definition; Terminal Newline ]);
65     (Terminal_definition, [ Terminal Identifier; Terminal Terminal_pattern ]);
66     ( NonTerminal_definition,
67       [

```

5 – Annexes • 5.6 Code

```

68         Terminal Identifier;
69         Terminal Derivation_symbol;
70         NonTerminal Rule_identifier_list;
71     ] );
72     (Rule_identifier_list, [ NonTerminal Rule_identifier_list_entry ]);
73     ( Rule_identifier_list,
74     [
75         NonTerminal Rule_identifier_list_entry;
76         NonTerminal Rule_identifier_list;
77     ] );
78     (Rule_identifier_list_entry, [ Terminal Identifier ]);
79     (Rule_identifier_list_entry, [ Terminal Identifier; Terminal Question ]);
80 ]
81
82 (** Converts a grammar syntax tree to a set of token rules and grammar rules. *)
83 let grammar_of_syntax_tree
84   (tree : (grammar_token_type, grammar_node_type) syntax_tree) :
85   (uchar regex * string) list * (string, string) grammar =
86   let strip_prefix_from_terminal_pattern (pattern : string) : string =
87     let rec non_space_index (i : int) =
88       if i = String.length pattern - 1 then i
89       else if pattern.[i] <> ' ' then i
90       else non_space_index (i + 1)
91     in
92     let start = non_space_index 2 in
93     String.sub pattern start (String.length pattern - start)
94   in
95
96   let build_terminal_definition
97     (definition : (grammar_token_type, grammar_node_type) syntax_tree) :
98     uchar regex * string =
99     match definition with
100     | Node
101     ( Terminal_definition,

```

```

102     [
103         Leaf { token_type = Identifier; value = name };
104         Leaf { token_type = Terminal_pattern; value = pattern };
105     ] ) -> (
106     let raw_pattern = strip_prefix_from_terminal_pattern pattern in
107     try (parse_regex raw_pattern, name)
108     with Failure s ->
109         failwith ("Failed to parse regex " ^ raw_pattern ^ ": " ^ s))
110 | _ -> failwith "Not a terminal definition"
111 in
112
113 let build_identifier_list
114     (tree : (grammar_token_type, grammar_node_type) syntax_tree) :
115     string list list =
116     let rec build_into_rev
117         (tree : (grammar_token_type, grammar_node_type) syntax_tree)
118         (res : string list list) =
119         match tree with
120         | Node
121             ( Rule_identifier_list,
122               [
123                 Node
124                     ( Rule_identifier_list_entry,
125                       [ Leaf { token_type = Identifier; value } ] )
126               ] ) ->
127             List.map (fun r -> value :: r) res
128         | Node
129             ( Rule_identifier_list,
130               [
131                 Node
132                     ( Rule_identifier_list_entry,
133                       [
134                         Leaf { token_type = Identifier; value };
135                         Leaf { token_type = Question };

```

5 – Annexes • 5.6 Code

```

136         ] );
137     ] ) ->
138     List.rev_append (List.map (fun r -> value :: r) res) res
139 | Node
140   ( Rule_identifier_list,
141     [
142       Node
143         ( Rule_identifier_list_entry,
144           [ Leaf { token_type = Identifier; value } ] );
145       remaining;
146     ] ) ->
147     build_into_rev remaining (List.map (fun r -> value :: r) res)
148 | Node
149   ( Rule_identifier_list,
150     [
151       Node
152         ( Rule_identifier_list_entry,
153           [
154             Leaf { token_type = Identifier; value };
155             Leaf { token_type = Question };
156           ] );
157       remaining;
158     ] ) ->
159     build_into_rev remaining
160     (List.rev_append (List.map (fun r -> value :: r) res) res)
161 | _ -> failwith "Not an identifier list"
162 in
163 List.map List.rev (build_into_rev tree [ [] ])
164 in
165
166 let build_non_terminal_definition
167   (definition : (grammar_token_type, grammar_node_type) syntax_tree) :
168   string * string list list =
169   match definition with

```

```

170 | Node
171 |   ( NonTerminal_definition,
172 |     [
173 |       Leaf { token_type = Identifier; value = name };
174 |       Leaf { token_type = Derivation_symbol };
175 |       (Node (Rule_identifier_list, _) as identifier_list);
176 |     ] ) ->
177 |   (name, build_identifier_list identifier_list)
178 | _ -> failwith "Not a nonterminal definition"
179 in
180
181 let process_entry
182   (entry : (grammar_token_type, grammar_node_type) syntax_tree)
183   (token_rules : (uchar regex * string) list)
184   (grammar_rules : (string * string list) list) =
185 match entry with
186 | Node (Grammar_entry, [ Leaf { token_type = Newline } ]) ->
187   (token_rules, grammar_rules)
188 | Node
189   ( Grammar_entry,
190     [
191       (Node (Terminal_definition, _) as definition);
192       Leaf { token_type = Newline };
193     ] ) ->
194   let terminal_definition = build_terminal_definition definition in
195   if
196     List.find_opt
197       (fun (_, name) -> name = snd terminal_definition)
198       token_rules
199     <> None
200   then
201     failwith ("Duplicate definition for token: " ^ snd terminal_definition)
202   else (terminal_definition :: token_rules, grammar_rules)
203 | Node

```

5 – Annexes • 5.6 Code

```

204     ( Grammar_entry,
205     [
206         (Node (NonTerminal_definition, _) as definition);
207         Leaf { token_type = Newline };
208     ] ) ->
209     let name, derivations = build_non_terminal_definition definition in
210
211     ( token_rules,
212     List.rev_append
213         (List.map (fun d -> (name, d)) derivations)
214         grammar_rules )
215 | _ -> failwith "Not a grammar entry"
216 in
217
218 let rec build_result_from
219     (tree : (grammar_token_type, grammar_node_type) syntax_tree)
220     (token_rules : (uchar regex * string) list)
221     (grammar_rules : (string * string list) list) =
222 match tree with
223 | Node (Grammar, [ entry ]) -> process_entry entry token_rules grammar_rules
224 | Node (Grammar, [ others; entry ]) ->
225     let token_rules', grammar_rules' =
226         process_entry entry token_rules grammar_rules
227     in
228     build_result_from others token_rules' grammar_rules'
229 | _ -> failwith "Not a grammar"
230 in
231
232 let token_rules, raw_grammar_rules = build_result_from tree [] [] in
233
234 let convert_identifier (identifier : string) : (string, string) grammar_entry
235     =
236 if List.find_opt (fun (_, name) -> name = identifier) token_rules <> None
237 then Terminal identifier

```

5 – Annexes • 5.6 Code

```

238     else if
239         List.find_opt (fun (name, _) -> name = identifieur) raw_grammar_rules
240         <> None
241     then NonTerminal identifieur
242     else failwith ("No rule matching: " ^ identifieur)
243 in
244
245 let grammar_rules =
246     List.map
247         (fun (name, raw_derivation) ->
248             (name, List.map convert_identifieur raw_derivation))
249     raw_grammar_rules
250 in
251
252 (token_rules, grammar_of_rule_list grammar_rules)
253
254 (** Parses a set of token rules and grammar rules from grammar source code. *)
255 let parse_grammar (s : string) =
256     let tokens = tokenize grammar_token_rules Eof Unrecognizable s in
257     let tokens_no_whitespace =
258         List.filter (fun token -> token.token_type <> Whitespace) tokens
259     in
260
261     let rec remove_comments (tokens : grammar_token_type token list)
262         (in_comment : bool) (res : grammar_token_type token list) :
263         grammar_token_type token list =
264     match tokens with
265     | [] -> res
266     | { token_type = Comment_start } :: q -> remove_comments q true res
267     | ({ token_type = Newline } as x) :: q -> remove_comments q false (x :: res)
268     | x :: q ->
269         let next_res = if in_comment then res else x :: res in
270         remove_comments q in_comment next_res
271 in

```

```
272 |  
273 |   let tokens_clean = List.rev (remove_comments tokens_no_whitespace false []) in  
274 |   let automaton = construit_automate_LR1 grammar_rules Grammar Eof in  
275 |   let syntax_tree = parse automaton Eof tokens_clean Grammar in  
276 |   grammar_of_syntax_tree syntax_tree
```


lexer.ml

```

1  open Regex
2  open Automaton
3  open Utils
4
5  type 'a token = { token_type : 'a; value : string; start : int; length : int }
6
7  exception Tokenize_failure of { current_pos : int }
8
9  (** Converts text to a list of tokens (including an EOF token) according to the
10 specified rules. *)
11 let tokenize (rules : (uchar regex * 'a) list) (tok_eof : 'a) (tok_unknown : 'a)
12   (text : string) : 'a token list =
13   if List.find_opt (fun (_, token_type) -> token_type = tok_eof) rules <> None
14   then failwith "EOF token had production rule"
15   else
16     let automaton =
17       List.map
18         (fun (regex, t) ->
19           (regex |> automaton_of_regex |> remove_epsilon_transitions
20            |> determinize |> remove_state [],
21             t ))
22       rules
23     in
24     let rec tokenize_from (global_offset : int) (remaining_text : uchar list)
25       (reversed_result : 'a token list) : 'a token list =
26       if remaining_text = [] then
27         { token_type = tok_eof; value = ""; start = global_offset; length = 0 }
28         :: reversed_result
29       else
30         let rec read_longest_token_from (start : int)
31           (alive_states : (uchar, _) execution_state list)
32           (current_result : ('a token * uchar list) option)
33           (current_lexeme : uchar list) (remaining_text : uchar list) :

```

5 – Annexes • 5.6 Code

```

34         ('a token * uchar list) option =
35     if List.length alive_states = 0 then current_result
36     else
37         let final_states = List.filter is_final_state alive_states in
38         let next_result =
39             match final_states with
40             | [] -> current_result
41             | e :: _ ->
42                 Some
43                     ( {
44                         token_type = List.assq e.automaton automations;
45                         value = implode_uchar (List.rev current_lexeme);
46                         start;
47                         length = List.length current_lexeme;
48                     },
49                     remaining_text )
50         in
51         match remaining_text with
52         | [] -> next_result
53         | c :: next_text ->
54             let next_states =
55                 alive_states
56                 |> List.filter_map (fun state -> next_state state c)
57             in
58             let next_lexeme = c :: current_lexeme in
59             read_longest_token_from start next_states next_result
60             next_lexeme next_text
61     in
62     let states = automations |> List.map fst |> List.map start_execution in
63     match
64         read_longest_token_from global_offset states None [] remaining_text
65     with
66     (* / None -> raise (Tokenize_failure { current_pos = global_offset }) *)
67     | None ->

```

```

68         tokenize_from (global_offset + 1) (List.tl remaining_text)
69         ({
70             start = global_offset;
71             token_type = tok_unknown;
72             value = string_of_uchar (List.hd remaining_text);
73             length = 1;
74         })
75         :: reversed_result)
76 | Some (token, remaining_text) ->
77     tokenize_from
78         (global_offset + token.length)
79         remaining_text (token :: reversed_result)
80 in
81 List.rev (tokenize_from 0 (uchar_list_of_string text) [])
82
83 let string_of_token (string_of_type : 'a -> string) (token : 'a token) =
84     "Token("
85     ^ string_of_type token.token_type
86     ^ ", " ^ token.value ^ ", start=" ^ string_of_int token.start ^ ")"

```

parser.ml

```

1  open Lexer
2  open Automaton
3  open Utils
4
5  exception SyntaxError of string * int
6  exception EmptyFile
7
8  type ('token_type, 'non_terminal) grammar_entry =
9    | NonTerminal of 'non_terminal
10   | Terminal of 'token_type
11
12  type ('token_type, 'non_terminal) non_terminal_or_token =
13    | NonTerminalRepr of 'non_terminal * int
14    | Token of 'token_type token
15
16  type ('token_type, 'non_terminal) derivation =
17    ('token_type, 'non_terminal) grammar_entry list
18
19  type ('token_type, 'non_terminal) rule =
20    'non_terminal * ('token_type, 'non_terminal) derivation
21
22  type ('token_type, 'non_terminal) grammar =
23    ('non_terminal, ('token_type, 'non_terminal) derivation list) Hashtbl.t
24
25  type ('token_type, 'non_terminal) syntax_tree =
26    | Node of 'non_terminal * ('token_type, 'non_terminal) syntax_tree list
27    | Leaf of 'token_type token
28
29  (* Représente les situations LR(0), du style ... ↑ ... *)
30  type ('token_type, 'non_terminal) lr0_situation =
31    ('token_type, 'non_terminal) rule * int
32
33  (* Représente les situations LR(1), du style ... ↑ ... ~ {b...b} *)

```

5 – Annexes • 5.6 Code

```

34 | type ('token_type, 'non_terminal) lr1_situation =
35 |   ('token_type, 'non_terminal) lr0_situation * 'token_type TerminalSet.t
36 |
37 | (* Dictionnaire de situations LR(0) à leurs ensembles premiers correspondants.
38 | * On assure de cette façon l'unicité de la situation pour une règle donnée
39 | * dans les états des automates LR(1). *)
40 | type ('token_type, 'non_terminal) lr1_automaton_state =
41 |   (('token_type, 'non_terminal) lr0_situation, 'token_type TerminalSet.t)
42 |   Hashtbl.t
43 |
44 | (* Représente un automate LR(1) *)
45 | type ('token_type, 'non_terminal) lr1_automaton =
46 |   ( ('token_type, 'non_terminal) grammar_entry,
47 |     ('token_type, 'non_terminal) lr1_automaton_state )
48 |   deterministic_automaton
49 |
50 | type ('token_type, 'non_terminal) lr1_transition =
51 |   ('token_type, 'non_terminal) lr1_automaton_state
52 |   * ('token_type, 'non_terminal) grammar_entry
53 |   * ('token_type, 'non_terminal) lr1_automaton_state
54 |
55 | let grammar_of_rule_list (l: ('token_type, 'non_terminal) rule list) :
56 |   ('token_type, 'non_terminal) grammar =
57 |   let res = Hashtbl.create 8 in
58 |   List.iter
59 |     (fun (nt, derivation) ->
60 |       match Hashtbl.find_opt res nt with
61 |       | Some existing_derivations ->
62 |         Hashtbl.replace res nt (derivation :: existing_derivations)
63 |       | None -> Hashtbl.add res nt [ derivation ])
64 |     l;
65 |   res
66 |
67 | (* Renvoie l'ensemble Premier_LL(1)(s) dans la grammaire g *)

```

5 – Annexes • 5.6 Code

```

68 let premier_LL1 (s : ('token_type, 'non_terminal) derivation)
69   (g : ('token_type, 'non_terminal) grammar)
70   (mapping : 'token_type TerminalSet.mapping)
71   (cache :
72     (('token_type, 'non_terminal) derivation, 'token_type TerminalSet.t)
73     Hashtbl.t) : 'token_type TerminalSet.t =
74   (* Crée un dictionnaire d tel que d[s] est l'ensemble des dérivations w tels
75    * que premier(s) est inclus dans premier(w).
76    * cf. 4.2.1.3 Théorème 32 (p 63). *)
77   let construire_inclusions () :
78     (('token_type, 'non_terminal) derivation,
79     ('token_type, 'non_terminal) derivation Hashset.t )
80     Hashtbl.t =
81     let inclusions = Hashtbl.create 2 in
82     let a_traiter = Hashset.create () in
83     Hashtbl.add inclusions s (Hashset.create ());
84     Hashset.add a_traiter s;
85
86     while not (Hashset.is_empty a_traiter) do
87       let derivation = Hashset.remove_one a_traiter in
88       if Hashtbl.find_opt cache derivation = None then
89         let derivation_contient =
90           match derivation with
91           | [ Terminal _ ] -> []
92           | [ NonTerminal n ] -> (
93             match Hashtbl.find_opt g n with
94             | Some derivations -> derivations
95             | None -> [])
96           | x :: q -> [ [ x ] ]
97           | [] -> failwith "Cannot compute premier_LL1 of an empty derivation"
98         in
99         derivation_contient
100         |> List.iter (
101           fun derivation_contenue ->

```

```

102         match Hashtbl.find_opt inclusions derivation_contenue with
103         | Some sur_derivations -> Hashset.add sur_derivations derivation
104         | None ->
105             let sur_derivations = Hashset.create () in
106             Hashset.add sur_derivations derivation;
107             Hashtbl.add inclusions derivation_contenue sur_derivations;
108             (* C'est une dérivation jusqu'ici inconnue. *)
109             Hashset.add a_traiter derivation_contenue
110         )
111     done;
112
113     inclusions
114 in
115
116     let inclusions = construire_inclusions () in
117     let valeurs = Hashtbl.create (Hashtbl.length inclusions) in
118
119     let a_traiter = Hashset.create () in
120
121     (* Initialisation *)
122     Hashtbl.iter
123     (fun k v ->
124         let premier_k = TerminalSet.create mapping in
125         Hashtbl.add valeurs k premier_k;
126         match k with
127         | [ Terminal a ] ->
128             TerminalSet.add premier_k a;
129             Hashset.add a_traiter k
130         | _ -> (
131             match Hashtbl.find_opt cache k with
132             | None -> ()
133             | Some premier ->
134                 (* On doit remplacer et non ajouter, car on itère plus tard *)
135                 Hashtbl.replace valeurs k premier;

```

```

136         Hashset.add a_traiter k))
137     inclusions;
138
139     (* Saturation *)
140     while not (Hashset.is_empty a_traiter) do
141         let derivation = Hashset.remove_one a_traiter in
142         let valeur_derivation = Hashtbl.find valeurs derivation in
143         let sur_derivations = Hashtbl.find inclusions derivation in
144         Hashset.iter
145             (fun sur_derivation ->
146                 let valeur_sur_derivation = Hashtbl.find valeurs sur_derivation in
147                 let longueur_init = TerminalSet.length valeur_sur_derivation in
148                 TerminalSet.iter (TerminalSet.add valeur_sur_derivation) valeur_derivation;
149
150                 (* Si l'ajout des éléments de valeur_derivation a changé la
151                  * valeur de valeur_sur_derivation, alors il faut re-traiter
152                  * sur_derivation. *)
153                 if longueur_init <> TerminalSet.length valeur_sur_derivation then
154                     Hashset.add a_traiter sur_derivation)
155             sur_derivations
156     done;
157
158     Hashtbl.iter (Hashtbl.replace cache) valeurs;
159
160     Hashtbl.find valeurs s
161
162     (* Renvoie l'ensemble Premier_LR(1)(s, ) dans la grammaire g *)
163     let premier_LR1 (s : ('token_type, 'non_terminal) derivation)
164         (sigma : 'token_type TerminalSet.t)
165         (g : ('token_type, 'non_terminal) grammar)
166         (mapping : 'token_type TerminalSet.mapping)
167         (cache :
168             (('token_type, 'non_terminal) derivation, 'token_type TerminalSet.t) Hashtbl.t)
169         : 'token_type TerminalSet.t =

```


5 – Annexes • 5.6 Code

```

170     match s with [] -> sigma | _ -> premier_LL1 s g mapping cache
171
172     (* Sature e jusqu'à que e soit une fermeture des situations LR(1) de e. *)
173     let fermer_situations_LR1 (e : ('token_type, 'non_terminal) lr1_automaton_state)
174       (g : ('token_type, 'non_terminal) grammar)
175       (mapping : 'token_type TerminalSet.mapping)
176       (premier_cache :
177         (('token_type, 'non_terminal) derivation, 'token_type TerminalSet.t )
178         Hashtbl.t)
179       : unit =
180       (* Si le non-terminal de règle est nt, renvoie la situation nt->~2.
181        * Renvoie None sinon *)
182       let nouvelle_situation_regle (sigma2 : 'token_type TerminalSet.t)
183         (regle : ('token_type, 'non_terminal) rule) :
184         ('token_type, 'non_terminal) lr1_situation =
185         ((regle, 0), TerminalSet.copy sigma2)
186     in
187
188     (* Sachant une situation s : N -> ~T~ avec T un non-terminal, renvoie
189      * la liste des situations T->~premierLR1(, ) pour T-> règle de g *)
190     let liste_nouvelles_situations
191       (((_, derivation), curseur) : ('token_type, 'non_terminal) lr0_situation)
192       (sigma : 'token_type TerminalSet.t) :
193       ('token_type, 'non_terminal) lr1_situation list =
194     let regle_fin = list_skip derivation curseur in
195     match regle_fin with
196     | NonTerminal nt :: beta -> (
197       let premier = premier_LR1 beta sigma g mapping premier_cache in
198       match Hashtbl.find_opt g nt with
199       | Some derivations ->
200         List.map
201           (fun derivation ->
202             nouvelle_situation_regle premier (nt, derivation))
203         derivations

```

5 – Annexes • 5.6 Code

```

204 | None -> []
205 | _ -> []
206 in
207
208 let a_traiter = Hashset.create () in
209 Hashtbl.iter (fun k _ -> Hashset.add a_traiter k) e;
210
211 while not (Hashset.is_empty a_traiter) do
212   let situation = Hashset.remove_one a_traiter in
213   let sigma = Hashtbl.find e situation in
214   let nouvelles_situations = liste_nouvelles_situations situation sigma in
215   List.iter
216     (fun (nouvelle_situation, nouveau_premier) ->
217       match Hashtbl.find_opt e nouvelle_situation with
218       | None ->
219         (* La derivation T -> ^gamma n'était pas présente dans e. *)
220         Hashtbl.add e nouvelle_situation nouveau_premier;
221         Hashset.add a_traiter nouvelle_situation
222       | Some premier ->
223         let longueur_init = TerminalSet.length premier in
224         TerminalSet.iter (TerminalSet.add premier) nouveau_premier;
225         if longueur_init <> TerminalSet.length premier then
226           (* On a ajouté des éléments à l'ensemble sigma de T -> ^gamma *)
227           Hashset.add a_traiter nouvelle_situation)
228     nouvelles_situations
229 done
230
231 let states_equal (a : ('token_type, 'non_terminal) lr1_automaton_state)
232   (b : ('token_type, 'non_terminal) lr1_automaton_state) =
233   a == b
234   ||
235   if Hashtbl.length a <> Hashtbl.length b then false
236   else
237     Hashtbl.to_seq a

```

5 – Annexes • 5.6 Code

```

238 |> Seq.for_all (fun (k, v) ->
239 |   match Hashtbl.find_opt b k with
240 |   | None -> false
241 |   | Some v' -> TerminalSet.equals v v')
242
243 (* Construit l'automate LR(1) de la grammaire g. eof_token désigne le lexème
244 * de fin de fichier, il ne doit pas être utilisé dans les règles de la
245 * grammaire (mais il devra figurer dans la sortie de l'analyseur lexical) *)
246 let construit_automate_LR1 (g : ('token_type, 'non_terminal) grammar)
247   (axiome : 'non_terminal) (lexeme_eof : 'token_type) :
248   ('token_type, 'non_terminal) lr1_automaton =
249   (* <herebedragons> *)
250   (* On veut pouvoir mettre des états de l'automate LR(1) dans un dictionnaire.
251   * Cela facilite non seulement la construction de l'automate mais rend aussi
252   * son exécution plus efficace.
253
254   * Malheureusement, les états LR(1) ne sont pas compatibles avec le module
255   * Hashtbl d'OCaml tel quels : ils contiennent des Hashset.t, dont l'égalité
256   * (i.e Hashset.equals) n'est pas l'identité structurelle (deux Hashtbl.t
257   * peuvent contenir les mêmes éléments mais ne pas avoir le même nombre de
258   * buckets, par exemple).
259
260   * Il faut donc nous munir d'un Hashtbl avec une fonction de hachage et
261   * d'égalité que nous contrôlons, ce qui est possible via le functor
262   * Hashtbl.Make.
263
264   * Deuxième malheur, nos états LR(1) sont génériques, et le type HashedType.t
265   * ne peut pas l'être. Nous devons donc masquer les paramètres de type de
266   * lr1_automaton_state dans notre Hashtbl.
267
268   * Cela est possible grâce à l'usage de GADT.
269
270   * Cela mène à deux problèmes de plus :
271   * - Le type de notre Hashtbl doit être un GADT. Il ne peut donc pas

```

5 – Annexes • 5.6 Code

```

272 * directement être un lr1_automaton_state, mais est plutôt un variant d'un
273 * type contenant un lr1_automaton_state.
274 * - Comme nous masquons les paramètres de type de toutes les clés de notre
275 * Hashtbl, OCaml n'apporte aucune garantie quant au type exact des objets
276 * extraits de notre Hashtbl : c'est à nous de s'assurer que nous
277 * n'insérons et n'extrayons que des objets du même type. Il faut alors
278 * utiliser Obj.magic pour éviter les erreurs de type.
279
280 * Ces modules sont restreints à la fonction construit_automate_LR1 pour ces
281 * raisons.
282 *)
283 let module AnyLR1State = struct
284   type t = Any : (_, _) lr1_automaton_state -> t
285
286   let equal (Any a) (Any b) = states_equal (Obj.magic a) (Obj.magic b)
287   let hash (Any a) = Hashtbl.fold (fun k _ acc -> acc lxor Hashtbl.hash k) a 0
288 end in
289 let module LR1StateMap = struct
290   include Hashtbl.Make (AnyLR1State)
291
292   let remove_one (t : 'a t) : ('non_terminal, 'token_type) lr1_automaton_state
293   =
294     match (to_seq_keys t) () with
295     | Nil -> raise (Invalid_argument "Cannot remove from an empty set")
296     | Cons (key, _) -> (
297       remove t key;
298       match key with Any a -> Obj.magic a)
299 end in
300 (* </herebedragons> *)
301 (* Étant donné une liste l de situations LR(1), renvoie la liste des terminaux
302 * et non terminaux pouvant être lus. *)
303 let rec liste_terminaux_et_non_terminaux_a_iterer
304   (s : ('token_type, 'non_terminal) lr1_automaton_state) :
305   ('token_type, 'non_terminal) grammar_entry Hashset.t =

```

5 – Annexes • 5.6 Code

```

306     let res = Hashset.create () in
307     Hashtbl.iter
308       (fun (_, regle), curseur) _ ->
309         if curseur <> List.length regle then
310           let alpha = List.nth regle curseur in
311           Hashset.add res alpha)
312     s;
313     res
314   in
315
316   let premier_cache = Hashtbl.create 2 in
317   let mapping =
318     TerminalSet.build_mapping
319       (lexeme_eof
320        :: (Hashtbl.to_seq_values g |> List.of_seq |> List.flatten
321           |> List.map
322             (List.filter_map (fun s ->
323               match s with NonTerminal _ -> None | Terminal t -> Some t))
324           |> List.flatten))
325   in
326
327   (* On ajoute toutes les règles pour l'axiome, puis on ferme l'ensemble pour
328      * obtenir l'état initial. *)
329   let etat_initial = Hashtbl.create 2 in
330   (match Hashtbl.find_opt g axiome with
331   | Some derivations ->
332     List.iter
333       (fun derivation ->
334         Hashtbl.add etat_initial
335           ((axiome, derivation), 0)
336           (TerminalSet.singleton mapping lexeme_eof))
337     derivations
338   | None -> ());
339   fermer_situations_LR1 etat_initial g mapping premier_cache;

```

5 – Annexes • 5.6 Code

```

340
341   (* transitions[state0][symbol] est l'état atteint en lisant symbol depuis
342   * state0. *)
343   let transitions = LR1StateMap.create 2 in
344   (* Collection des états créés. *)
345   let etats = Hashset.create () in
346
347   (* Ensemble des états à traiter, sous forme d'un dictionnaires d'états à
348   * unit. *)
349   let a_traiter = LR1StateMap.create 2 in
350   LR1StateMap.add a_traiter (AnyLR1State.Any etat_initial) ();
351
352   let fermeture_cache = LR1StateMap.create 2 in
353
354   while LR1StateMap.length a_traiter <> 0 do
355     let etat = LR1StateMap.remove_one a_traiter in
356     Hashset.add etats etat;
357     (* Les transitions sortantes de cet état. *)
358     let transitions_etat = Hashtbl.create 2 in
359     LR1StateMap.add transitions (AnyLR1State.Any etat) transitions_etat;
360     let tnt = liste_terminaux_et_non_terminaux_a_iterer etat in
361     Hashset.iter
362       (fun alpha ->
363         let nouvel_etat = Hashtbl.create 2 in
364
365         Hashtbl.iter
366           (fun ((nt, derivation), curseur) v ->
367             if
368               curseur <> List.length derivation
369               && List.nth derivation curseur = alpha
370             then
371               Hashtbl.add nouvel_etat
372                 ((nt, derivation), curseur + 1)
373                 (TerminalSet.copy v))

```

```

374         etat;
375
376     let fermeture_nouvel_etat =
377         match
378             LR1StateMap.find_opt fermeture_cache (AnyLR1State.Any nouvel_etat)
379         with
380         | Some fermeture -> fermeture
381         | None ->
382             let copy = Hashtbl.copy nouvel_etat in
383             fermer_situations_LR1 nouvel_etat g mapping premier_cache;
384             LR1StateMap.add
385                 fermeture_cache (AnyLR1State.Any copy) nouvel_etat;
386             if
387                 LR1StateMap.find_opt transitions (AnyLR1State.Any nouvel_etat)
388                 = None
389             then
390                 LR1StateMap.replace a_traiter (AnyLR1State.Any nouvel_etat) ();
391             nouvel_etat
392     in
393
394     Hashtbl.add transitions_etat alpha fermeture_nouvel_etat)
395   tnt
396 done;
397
398 {
399   states = etats;
400   initial_state = etat_initial;
401   final_states = Hashset.create ();
402   transition =
403     (fun state0 symbol ->
404       let transitions_state0 =
405         LR1StateMap.find transitions (AnyLR1State.Any state0)
406       in
407       Hashtbl.find_opt transitions_state0 symbol);

```

5 – Annexes • 5.6 Code

```

408     }
409
410     (* Renvoie None si `a` est sans conflits LR(1), et un état de `a` présentant un
411      * conflit sinon. *)
412     let trouve_conflits (a : ('token_type, 'non_terminal) lr1_automaton) :
413       ('token_type, 'non_terminal) lr1_automaton_state option =
414       let exception Conflit in
415       (* Renvoie true ssi `s` est un état LR(1) présentant un conflit. *)
416       let est_conflit (s : ('token_type, 'non_terminal) lr1_automaton_state) : bool
417         =
418         let mapping = (List.hd (Hashtbl.to_seq_values s |> List.of_seq)).mapping in
419
420         try
421           let suivants_a_reduire = TerminalSet.create mapping in
422           (* Recherche de conflits réduire/réduire *)
423           Hashtbl.iter
424             (fun ((nt, derivation), curseur) v ->
425               if curseur = List.length derivation then (
426                 if
427                   not (TerminalSet.is_empty (TerminalSet.intersection v suivants_a_reduire))
428                 then raise Conflit;
429                 TerminalSet.iter (TerminalSet.add suivants_a_reduire) v))
430             s;
431
432           (* Recherche de conflits lire/réduire *)
433           Hashtbl.iter
434             (fun ((_, derivation), curseur) _ ->
435               if curseur <> List.length derivation then
436                 match List.nth derivation curseur with
437                 | NonTerminal nt -> ()
438                 | Terminal t ->
439                   if TerminalSet.mem suivants_a_reduire t then raise Conflit)
440             s;
441

```


5 – Annexes • 5.6 Code

```

442     false
443     with Conflit -> true
444 in
445
446 let rec trouve_conflit
447   (remaining_states : ('token_type, 'non_terminal) lr1_automaton_state list)
448   : ('token_type, 'non_terminal) lr1_automaton_state option =
449   match remaining_states with
450   | [] -> None
451   | s :: q -> if est_conflit s then Some s else trouve_conflit q
452 in
453
454 trouve_conflit (HashSet.to_list a.states)
455
456 (* À partir d'un état e et d'un terminal t, renvoie une règle que l'on peut
457  * réduire (i.e. une situation  $N \rightarrow a_1 \dots a_n \sim$  où t ) ou None s'il n'y en
458  * n'a pas. *)
459 let trouve_reduction_a_faire
460   (e : ('token_type, 'non_terminal) lr1_automaton_state) (t : 'token_type) :
461   ('token_type, 'non_terminal) rule option =
462   (* Renvoie true si la situation s est une situation à réduction, false sinon
463   *)
464   let a_reduire (((_, rule), idx) : ('token_type, 'non_terminal) lr0_situation)
465     : bool =
466     idx = List.length rule
467 in
468
469 let situation_a_reduire =
470   Hashtbl.to_seq e
471   |> seq_find (fun (lr0_situation, sigma) ->
472     a_reduire lr0_situation && TerminalSet.mem sigma t)
473 in
474 match situation_a_reduire with
475 | None -> None

```

5 – Annexes • 5.6 Code

```

476 | Some ((rule, _), _) -> Some rule
477
478 let parse (a : ('token_type, 'non_terminal) lrl_automaton)
479   (eof_symbol : 'token_type) (texte : 'token_type token list)
480   (axiome : 'non_terminal) : ('token_type, 'non_terminal) syntax_tree =
481   if trouve_conflits a <> None then
482     failwith "L'automate présente des conflits - la grammaire n'est pas LR(1)";
483   let pile_arbres : ('token_type, 'non_terminal) syntax_tree * int = Stack.t =
484     Stack.create ()
485   in
486   let pile_etats = Stack.create () in
487
488   Stack.push a.initial_state pile_etats;
489   let etat_courant = ref a.initial_state in
490
491   let nouveau_texte = List.map (fun x -> Token x) texte in
492
493   let rec parse_a_partir
494     (text : ('token_type, 'non_terminal) non_terminal_or_token list) :
495     ('token_type, 'non_terminal) syntax_tree =
496     match text with
497     | [] -> raise (SyntaxError (
498       "Le token EOF n'aurait pas dû être lu (ou absence de token EOF)",
499       0
500     ))
501   | x :: q -> (
502     match x with
503     | NonTerminalRepr (nt, pos_nt) ->
504       let est_a_eof =
505         match q with
506         | [ Token { token_type } ] when token_type = eof_symbol -> true
507         | _ -> false
508       in
509       if

```

```

510         states_equal !etat_courant a.initial_state
511         && nt = axiome && est_a_eof
512     then
513         let racine_opt = Stack.pop_opt pile_arbres in
514         match racine_opt with
515         | None -> raise (SyntaxError (
516             "État invalide : absence de racine apres lecture du texte",
517             pos_nt
518         ))
519         | Some (racine, _) ->
520             if not (Stack.is_empty pile_arbres) then
521                 let pos_start = snd (Stack.pop pile_arbres) in
522                 raise (SyntaxError (
523                     "Axiome lu alors qu'il restait des arbres",
524                     pos_start
525                 ))
526             else racine
527     else
528         let nouvel_etat = a.transition !etat_courant (NonTerminal nt) in
529
530         (etat_courant :=
531             match nouvel_etat with
532             | Some e -> e
533             | None -> raise (SyntaxError ("Impossible de lire !", pos_nt))
534         );
535         Stack.push !etat_courant pile_etats;
536         parse_a_partir q
537 | Token t -> (
538     match trouve_reduction_a_faire !etat_courant t.token_type with
539     | None ->
540         let nouvel_etat =
541             a.transition !etat_courant (Terminal t.token_type)
542         in
543

```

5 – Annexes • 5.6 Code

```

544         (etat_courant :=
545             match nouvel_etat with
546             | Some e -> e
547             | _ ->
548                 if t.token_type <> eof_symbol then
549                     raise (SyntaxError (
550                         ("Impossible de lire " ^ t.value), t.start))
551                 else
552                     raise (SyntaxError (
553                         ("Lecture du texte terminée sans que l'analyse "
554                         ^ "syntaxique n'ait abouti"), t.start));
555             Stack.push !etat_courant pile_etats;
556             (* La ligne suivante ne figure pas dans l'algo du livre *)
557             Stack.push (Leaf t, t.start) pile_arbres;
558             parse_a_partir q
559         | Some (nt, regle) ->
560             let n = List.length regle in
561             let n_arbres_pos = pop_n pile_arbres n |> List.rev in
562             let pos_start = begin match n_arbres_pos with
563                 | (_, pos)::_ -> pos
564                 | _ -> 0
565             end in
566             let n_arbres = List.map fst n_arbres_pos in
567             Stack.push (Node (nt, n_arbres), pos_start) pile_arbres;
568             let _ = pop_n pile_etats n in
569             etat_courant := Stack.top pile_etats;
570             parse_a_partir ((NonTerminalRepr (nt, pos_start)) :: text)))
571     in
572     if nouveau_texte = [Token {token_type=eof_symbol; value=""; start=0; length=0}] then
573         raise EmptyFile
574     else
575         parse_a_partir nouveau_texte
576
577     (***** Fonctions d'affichage *****)

```

5 – Annexes • 5.6 Code

```

578 let string_of_symbol (s : (char, char) grammar_entry) : string =
579   match s with Terminal c | NonTerminal c -> string_of_char c
580
581 let string_of_situation
582   (((n, rule), idx), sigma) : (char, char) lr1_situation
583   : string =
584   let ns = string_of_char n in
585   let rule_chars =
586     List.map (fun x -> match x with Terminal c | NonTerminal c -> c) rule
587   in
588   let rule_beginning = implode (list_beginning rule_chars idx) in
589   let rule_end = implode (list_skip rule_chars idx) in
590   let sigma_s =
591     TerminalSet.to_seq sigma |> List.of_seq |> List.map string_of_char
592     |> String.concat ", "
593   in
594   ns ^ " -> " ^ rule_beginning ^ "~" ^ rule_end ^ " ~ {" ^ sigma_s ^ "}"
595
596 let string_of_state (s : (char, char) lr1_automaton_state) : string =
597   let strings =
598     Hashtbl.to_seq s |> List.of_seq |> List.map string_of_situation
599   in
600   "(" ^ String.concat ", " strings ^ ")"
601
602 let print_syntax_tree (t : ('token_type, 'non_terminal) syntax_tree)
603   (string_of_non_terminal : 'non_terminal -> string)
604   (string_of_token : 'token_type token -> string) =
605   let rec print_syntax_tree_indent (indent : string) =
606     (t : ('token_type, 'non_terminal) syntax_tree) =
607     match t with
608     | Leaf token -> print_endline (indent ^ string_of_token token)
609     | Node (nt, children) ->
610       print_endline (indent ^ string_of_non_terminal nt ^ ":");
611       List.iter

```

```
612 |         (fun child -> print_syntax_tree_indent (indent ^ " ") child)
613 |         children
614 |     in
615 |     print_syntax_tree_indent "" t
616 |     (*****)
```

rectify_helper.ml

```

1  open Caml_light
2  open Rectify
3
4  type rectify_result =
5    | CouldNotComputeRectifyingSet
6    | EmptyRectifyingSet
7    | NewBindings of value_definition list * recursive_call_info option
8
9  let rectify_bindings (bindings : binding list) : rectify_result =
10    let k = ref 0 in
11
12    (* Step 1: Linearize *)
13    let linearized_functions =
14      bindings
15      |> List.filter_map (fun (binding : binding) ->
16        match binding with
17        | Variable _ -> None
18        | Function { name; parameters; body; return_type } ->
19          let body_lin, k' = linearize body !k in
20            k := k';
21            Some
22              ( name,
23                FunctionLiteral
24                  {
25                    style = Fun;
26                    cases = [ (parameters, body_lin) ];
27                    return_type_for_delinearize = return_type;
28                  } ))
29    in
30
31    (* Step 2: Calculate rectifying set *)
32    match cloture_rectifiable linearized_functions with
33    | None -> CouldNotComputeRectifyingSet

```

5 – Annexes • 5.6 Code

```

34 | Some { rectifying_set = [] } -> EmptyRectifyingSet
35 | Some { rectifying_set = cloture_rect } ->
36 |   (* Step 3: Rectify *)
37 |   let rectified_functions =
38 |     rectify linearized_functions cloture_rect linearized_functions
39 |   in
40 |
41 |   (* Step 4: Replace continuations with accumulators (if possible) *)
42 |   let accumulator_functions, initial_accumulator_constant =
43 |     (* Step 4a: Extract all continuations *)
44 |     match find_continuations rectified_functions cloture_rect with
45 |     | None -> (rectified_functions, None)
46 |     | Some continuations -> (
47 |       (* Step 4b: Extract composition with previous continuation form each continuation
48 |         ↪ *)
49 |       let extracted_continuations =
50 |         continuations |> List.filter_map extract_continuation_composition
51 |       in
52 |       if List.length extracted_continuations <> List.length continuations
53 |       then (rectified_functions, None)
54 |       else
55 |         (* Step 4c: Find the initial constant to use *)
56 |         let initial_constant = find_initials rectified_functions in
57 |         match initial_constant with
58 |         | None -> (rectified_functions, None)
59 |         | Some (variables, initial) ->
60 |           (* Step 4d: Check continuations can commute *)
61 |           if commutes extracted_continuations && List.length initial = 1
62 |           then
63 |             ( rectified_functions
64 |               |> List.map (fun (name, body) ->
65 |                 replace_continuations_with_accumulator
66 |                   [ (name, body) ]

```


5 – Annexes • 5.6 Code

```

67         cloture_rect variables
68         |> List.hd),
69         Some (List.hd initial) )
70     else (rectified_functions, None))
71 in
72
73   (* Step 5: Rename updated functions *)
74   let new_name (n : string) = n ^ "_rectified" in
75
76   let renamed_functions =
77     rename_elements accumulator_functions cloture_rect new_name
78   in
79
80   let rectified_bindings =
81     bindings
82     |> List.map (fun (binding : binding) ->
83       match binding with
84       | Variable _ -> (binding, None)
85       | Function { name; parameters; body; return_type } ->
86         if not (List.mem name cloture_rect) then (binding, None)
87         else
88           let new_linearized_definition =
89             match List.assoc (new_name name) renamed_functions with
90             | FunctionLiteral f -> f
91             | _ -> failwith "Unable to find redefined definition"
92           in
93
94           let new_parameters, new_linearized_body =
95             match new_linearized_definition.cases with
96             | [ c ] -> c
97             | _ -> failwith "Bad function"
98           in
99
100     let new_definition =

```

```

101         (Function
102         {
103             name = new_name name;
104             parameters = new_parameters;
105             body = fst (delinearize new_linearized_body []);
106             (* The return type depends on the return type of the continuation
107             ↪ *)
108             return_type =
109                 new_linearized_definition
110                 .return_type_for_delinearize;
111         }
112         : Caml_light.binding)
113
114     in
115     let interceptor =
116         (Function
117         {
118             name;
119             parameters =
120                 parameters
121                 |> List.mapi (fun i parameter ->
122                     match get_name parameter with
123                     | Some _ -> parameter
124                     | None -> (
125                         match parameter with
126                         | Constant
127                           (Construction (Unit Parenthesis))
128                           ->
129                             parameter
130                         | _ -> Ident ("arg" ^ string_of_int i)));
131             body =
132                 FunctionApplication
133                 {
134                     receiver = Variable (new_name name);

```

```

134 arguments =
135   (parameters
136   |> List.mapi (fun i parameter ->
137     match get_name parameter with
138     | Some name ->
139       (Variable name : expression)
140     | None -> (
141       match parameter with
142       | Constant
143         (Construction
144           (Unit Parenthesis)) ->
145         Constant
146           (Construction
147             (Unit Parenthesis))
148       | _ ->
149         Variable
150           ("arg" ^ string_of_int i)))
151   )
152   @ [
153     Parenthesised
154     {
155       style = Parenthesis;
156       inner =
157         (match
158           initial_accumulator_constant
159         with
160         | None ->
161           FunctionLiteral
162           {
163             style = Fun;
164             cases =
165             [
166               ( [ Ident "res" ],
167                 Variable "res" );

```

5 – Annexes • 5.6 Code

```

168                                     ];
169                                     }
170                                     | Some cst -> Constant cst);
171                                     };
172                                     ];
173                                     };
174                                     return_type;
175                                     }
176                                     : Caml_light.binding)
177                                     in
178
179                                     (new_definition, Some interceptor))
180      (* Rassemble tous les bindings de fonctions mutuellement récursives
181       * puis ajoute les bindings d'intercepteurs juste après *)
182      |> List.fold_left
183      (fun l (new_def, interceptor) ->
184       match (l, interceptor) with
185       | [], Some i ->
186         [
187           { bindings = [ new_def ]; is_rec = true };
188           { bindings = [ i ]; is_rec = false };
189         ]
190       | [], None -> [ { bindings = [ new_def ]; is_rec = true } ]
191       | { bindings } :: q, Some i ->
192         { bindings = new_def :: bindings; is_rec = true }
193         :: { bindings = [ i ]; is_rec = false }
194         :: q
195       | { bindings } :: q, None ->
196         { bindings = new_def :: bindings; is_rec = true } :: q)
197      []
198      |> fun l ->
199      match l with
200      | [] -> []
201      | { bindings; is_rec } :: q ->

```

```

202         { bindings = List.rev bindings; is_rec } :: q
203     in
204     NewBindings (rectified_bindings, cloture_rectifiable renamed_functions)
205
206     (* La fonction transform bindings prend le bindings et le is_rec et renvoie
207      * les nouveaux bindings *)
208     let rec try_transform_bindings_deep (def : value_definition)
209       (transform_bindings : value_definition -> rectify_result) :
210       value_definition list =
211       match transform_bindings def with
212       | NewBindings (b, _) -> b
213       (* Only push deeper if we didn't fail to compute a rectifying set. If we did,
214        * then we might be hiding recursive calls when we dig deeper (if a local
215        * function calls a function defined in a higher scope). *)
216       | CouldNotComputeRectifyingSet -> [ def ]
217       | EmptyRectifyingSet ->
218         let transform_one_binding (binding : binding) =
219           match binding with
220           | Variable { lhs; value } ->
221             {
222               bindings =
223                 [
224                   Variable
225                     {
226                       lhs;
227                       value = transform_bindings_in value transform_bindings;
228                     };
229                 ];
230               is_rec = false;
231             }
232           | Function { name; parameters; body; return_type } ->
233             {
234               bindings =
235                 [

```

```

236         Function
237         {
238             name;
239             parameters;
240             body = transform_bindings_in body transform_bindings;
241             return_type;
242         };
243     ];
244     is_rec = def.is_rec;
245 }
246 in
247 List.map transform_one_binding def.bindings
248
249 and transform_bindings_in (e : expression)
250   (transform_bindings : value_definition -> rectify_result) : expression =
251   match e with
252   | Variable _ -> e
253   | Constant _ -> e
254   | Parenthesised { inner; style } ->
255       Parenthesised
256         { style; inner = transform_bindings_in inner transform_bindings }
257   | TypeCoercion { inner; typ } ->
258       TypeCoercion
259         { inner = transform_bindings_in inner transform_bindings; typ }
260   | ListLiteral l ->
261       ListLiteral
262         (List.map (fun l -> transform_bindings_in l transform_bindings) l)
263   | ArrayLiteral l ->
264       ArrayLiteral
265         (List.map (fun l -> transform_bindings_in l transform_bindings) l)
266   | RecordLiteral r ->
267       RecordLiteral
268         (List.map
269           (fun (n, e) -> (n, transform_bindings_in e transform_bindings))

```

```

270         r)
271     | WhileLoop { condition; body } ->
272         WhileLoop
273         {
274             condition = transform_bindings_in condition transform_bindings;
275             body = transform_bindings_in body transform_bindings;
276         }
277     | ForLoop { direction = direction'; variable; start; finish; body } ->
278         ForLoop
279         {
280             direction = direction';
281             variable;
282             start = transform_bindings_in start transform_bindings;
283             finish = transform_bindings_in finish transform_bindings;
284             body = transform_bindings_in body transform_bindings;
285         }
286     | Dereference e -> Dereference (transform_bindings_in e transform_bindings)
287     | FieldAccess { receiver; target } ->
288         FieldAccess
289         { receiver = transform_bindings_in receiver transform_bindings; target }
290     | ArrayAccess { receiver; target } ->
291         ArrayAccess
292         {
293             receiver = transform_bindings_in receiver transform_bindings;
294             target = transform_bindings_in target transform_bindings;
295         }
296     | FunctionApplication { receiver; arguments } ->
297         FunctionApplication
298         {
299             receiver = transform_bindings_in receiver transform_bindings;
300             arguments =
301                 List.map
302                 (fun arg -> transform_bindings_in arg transform_bindings)
303                 arguments;

```

```

304     }
305 | PrefixOperation { receiver; operation } ->
306   PrefixOperation
307   {
308     operation;
309     receiver = transform_bindings_in receiver transform_bindings;
310   }
311 | InfixOperation { lhs; rhs; operation } ->
312   InfixOperation
313   {
314     lhs = transform_bindings_in lhs transform_bindings;
315     rhs = transform_bindings_in rhs transform_bindings;
316     operation;
317   }
318 | Negation e -> Negation (transform_bindings_in e transform_bindings)
319 | Tuple t ->
320   Tuple (List.map (fun x -> transform_bindings_in x transform_bindings) t)
321 | FieldAssignment { receiver; target; value } ->
322   FieldAssignment
323   {
324     receiver = transform_bindings_in receiver transform_bindings;
325     target;
326     value = transform_bindings_in value transform_bindings;
327   }
328 | ArrayAssignment { receiver; target; value } ->
329   ArrayAssignment
330   {
331     receiver = transform_bindings_in receiver transform_bindings;
332     target = transform_bindings_in target transform_bindings;
333     value = transform_bindings_in value transform_bindings;
334   }
335 | ReferenceAssignment { receiver; value } ->
336   ReferenceAssignment
337   {

```



```

338         receiver = transform_bindings_in receiver transform_bindings;
339         value = transform_bindings_in value transform_bindings;
340     }
341 | If { condition; body; else_body } ->
342     If
343     {
344         condition = transform_bindings_in condition transform_bindings;
345         body = transform_bindings_in body transform_bindings;
346         else_body =
347             Option.map
348             (fun b -> transform_bindings_in b transform_bindings)
349             else_body;
350     }
351 | Sequence s ->
352     Sequence
353     (List.map (fun e -> transform_bindings_in e transform_bindings) s)
354 | Match { value; cases } ->
355     Match
356     {
357         value = transform_bindings_in value transform_bindings;
358         cases =
359             List.map
360             (fun (patterns, e) ->
361                 (patterns, transform_bindings_in e transform_bindings))
362             cases;
363     }
364 | Try { value; cases } ->
365     Try
366     {
367         value = transform_bindings_in value transform_bindings;
368         cases =
369             List.map
370             (fun (patterns, e) ->
371                 (patterns, transform_bindings_in e transform_bindings))

```

5 – Annexes • 5.6 Code

```

372         cases;
373     }
374 | FunctionLiteral { style; cases } ->
375     FunctionLiteral
376     {
377         style;
378         cases =
379             List.map
380             (fun (patterns, e) ->
381                 (patterns, transform_bindings_in e transform_bindings))
382             cases;
383     }
384 | LetBinding { bindings; is_rec; inner } ->
385     let new_bindings_list =
386         try_transform_bindings_deep { bindings; is_rec } transform_bindings
387     in
388     let rec flatten (inner_acc : expression)
389         (bindings_list : value_definition list) =
390         match bindings_list with
391         | [] -> inner_acc
392         | [ { bindings; is_rec } ] ->
393             LetBinding { bindings; is_rec; inner = inner_acc }
394         | { bindings } :: q ->
395             flatten
396                 (LetBinding { bindings; is_rec = false; inner = inner_acc })
397                 q
398     in
399     flatten
400         (transform_bindings_in inner transform_bindings)
401         (List.rev new_bindings_list)
402 | StringAccess { receiver; target } ->
403     StringAccess
404     {
405         receiver = transform_bindings_in receiver transform_bindings;

```

```
406         target = transform_bindings_in target transform_bindings;
407     }
408 | StringAssignment { receiver; target; value } ->
409   StringAssignment
410   {
411       receiver = transform_bindings_in receiver transform_bindings;
412       target = transform_bindings_in target transform_bindings;
413       value = transform_bindings_in value transform_bindings;
414   }
```

rectify.ml

```

1  open Caml_light
2
3  type linear_element =
4  | Variable of variable
5  | Constant of constant
6  | Parenthesised of { inner : linear_form; style : parenthesis_style }
7  | TypeCoercion of { inner : linear_form; typ : type_expression }
8  | ListLiteral of linear_form list
9  | ArrayLiteral of linear_form list
10 | RecordLiteral of (label * linear_form) list
11 | WhileLoop of { condition : linear_form; body : linear_form }
12 | ForLoop of {
13     direction : for_direction;
14     variable : lowercase_ident;
15     start : linear_form;
16     finish : linear_form;
17     body : linear_form;
18 }
19 | Dereference of linear_form
20 | FieldAccess of { receiver : linear_form; target : label }
21 | ArrayAccess of { receiver : linear_form; target : linear_form }
22 | FunctionApplication of {
23     receiver : linear_form;
24     arguments : linear_form list;
25 }
26 | PrefixOperation of { receiver : linear_form; operation : prefix_operation }
27 | InfixOperation of {
28     lhs : linear_form;
29     rhs : linear_form;
30     operation : infix_operation;
31 }
32 | Negation of linear_form
33 | Tuple of linear_form list

```

5 – Annexes • 5.6 Code

```
34 | FieldAssignment of {
35 |   receiver : linear_form;
36 |   target : label;
37 |   value : linear_form;
38 | }
39 | ArrayAssignment of {
40 |   receiver : linear_form;
41 |   target : linear_form;
42 |   value : linear_form;
43 | }
44 | ReferenceAssignment of { receiver : linear_form; value : linear_form }
45 | If of {
46 |   condition : linear_form;
47 |   body : linear_form;
48 |   else_body : linear_form option;
49 | }
50 | Sequence of linear_form list
51 | Match of { value : linear_form; cases : linear_pattern_cases }
52 | Try of { value : linear_form; cases : linear_pattern_cases }
53 | FunctionLiteral of linear_function_literal
54 | LetBinding of {
55 |   bindings : linear_binding list;
56 |   is_rec : bool;
57 |   inner : linear_form;
58 | }
59 | StringAccess of { receiver : linear_form; target : linear_form }
60 | StringAssignment of {
61 |   receiver : linear_form;
62 |   target : linear_form;
63 |   value : linear_form;
64 | }
65
66 | and linear_pattern_cases = (pattern list * linear_form) list
67
```

5 – Annexes • 5.6 Code

```

68 | and linear_function_literal = {
69 |   style : function_literal_style;
70 |   cases : linear_pattern_cases;
71 |   return_type_for_delinearize : type_expression option;
72 | }
73
74 | and linear_function_ = {
75 |   name : variable;
76 |   parameters : pattern list;
77 |   body : linear_form;
78 |   return_type : type_expression option;
79 | }
80
81 | and linear_binding =
82 |   | Variable of { lhs : pattern; value : linear_form }
83 |   | Function of linear_function_
84
85 | and linear_form = (variable * linear_element) list
86
87 | (** Given a linear form l, extracts the name associated with the last linear
88 |   element in l *)
89 | let rec last_var (l : linear_form) : variable =
90 |   match l with
91 |   | [] -> failwith "l was empty"
92 |   | (p, _) :: [] -> p
93 |   | x :: q -> last_var q
94
95 | (** Converts an OCaml expression e to a linear form, associating the generated
96 |   linear elements with the names a_k, a_(k+1), and so on.
97 |
98 |   Returns the linear form and the index after the last used name. *)
99 | let rec linearize (e : expression) (k : int) : linear_form * int =
100 |   let p (i : int) : variable = "a_" ^ string_of_int i in
101

```

```

102 match e with
103 | Variable v -> ([ (p k, Variable v) ], k + 1)
104 | Constant c -> ([ (p k, Constant c) ], k + 1)
105 | Parenthesised { inner; style } -> linearize inner k
106 | TypeCoercion { inner; typ } ->
107   let inner_lin, k = linearize inner k in
108   let inner_var = last_var inner_lin in
109   let e_elt =
110     TypeCoercion { inner = [ (p (k + 1), Variable inner_var) ]; typ }
111   in
112   (inner_lin @ [ (p k, e_elt) ], k + 2)
113 | ListLiteral l ->
114   let elt_lins, elt_names, k =
115     List.fold_left
116       (fun (lins, names, k) elt ->
117         let elt_lin, k = linearize elt k in
118         let elt_name = last_var elt_lin in
119         (elt_lin :: lins, elt_name :: names, k))
120       ([], [], k) l
121   in
122   let e_elt =
123     ListLiteral
124       (elt_names |> List.rev
125         |> List.mapi (fun i name -> [ (p (k + i + 1), Variable name) ]))
126   in
127   let elt_count = List.length elt_names in
128   ( (elt_lins |> List.rev |> List.concat) @ [ (p k, e_elt) ],
129     k + elt_count + 1 )
130 | ArrayLiteral l ->
131   let elt_lins, elt_names, k =
132     List.fold_left
133       (fun (lins, names, k) elt ->
134         let elt_lin, k = linearize elt k in
135         let elt_name = last_var elt_lin in

```

5 – Annexes • 5.6 Code

```

136         (elt_lin :: lins, elt_name :: names, k))
137     ([], [], k) 1
138 in
139 let e_elt =
140     ArrayLiteral
141       (elt_names |> List.rev
142         |> List.mapi (fun i name -> [ (p (k + i + 1), Variable name) ]))
143 in
144 let elt_count = List.length elt_names in
145 ( (elt_lins |> List.rev |> List.concat) @ [ (p k, e_elt) ],
146   k + elt_count + 1 )
147 | RecordLiteral l ->
148   let elt_lins, elt_names, k =
149     List.fold_left
150       (fun (lins, names, k) (field, elt) ->
151         let elt_lin, k = linearize elt k in
152         let elt_name = last_var elt_lin in
153         (elt_lin :: lins, (field, elt_name) :: names, k))
154       ([], [], k) 1
155 in
156 let e_elt =
157     RecordLiteral
158       (elt_names |> List.rev
159         |> List.mapi (fun i (field, name) ->
160           (field, [ (p (k + i + 1), Variable name) ])))
161 in
162 let elt_count = List.length elt_names in
163 ( (elt_lins |> List.rev |> List.concat) @ [ (p k, e_elt) ],
164   k + elt_count + 1 )
165 | WhileLoop _ -> failwith "Cannot linearize while loops"
166 | ForLoop _ -> failwith "Cannot linearize for loops"
167 | Dereference inner ->
168   let inner_lin, k = linearize inner k in
169   let inner_var = last_var inner_lin in

```


5 – Annexes • 5.6 Code

```

170     let e_elt = Dereference [ (p (k + 1), Variable inner_var) ] in
171     (inner_lin @ [ (p k, e_elt) ], k + 2)
172 | FieldAccess { target; receiver } -> (
173     match receiver with
174     | Constant c ->
175         (* This is a module access *)
176         ( [
177             ( p k,
178               FieldAccess { receiver = [ (p (k + 1), Constant c) ]; target }
179             );
180         ],
181         k + 2 )
182 | _ ->
183     let receiver_lin, k = linearize receiver k in
184     let receiver_var = last_var receiver_lin in
185     let e_elt =
186         FieldAccess
187         { target; receiver = [ (p (k + 1), Variable receiver_var) ] }
188     in
189     (receiver_lin @ [ (p k, e_elt) ], k + 2))
190 | ArrayAccess { target; receiver } ->
191     let receiver_lin, k = linearize receiver k in
192     let receiver_var = last_var receiver_lin in
193     let target_lin, k = linearize target k in
194     let target_var = last_var target_lin in
195     let e_elt =
196         ArrayAccess
197         {
198             receiver = [ (p (k + 1), Variable receiver_var) ];
199             target = [ (p (k + 2), Variable target_var) ];
200         }
201     in
202     (receiver_lin @ target_lin @ [ (p k, e_elt) ], k + 3)
203 | FunctionApplication { receiver; arguments } -> (

```

5 – Annexes • 5.6 Code

```

204     match receiver with
205     | Constant c ->
206         (* This is a type constructor. Technically we deviate from the definition of a
           ↪ correct
           linear form here (the argument may be a parenthesised tuple with depth > 1). *)
207         let arguments_lins, argument_names, k =
208             match arguments with
209             | [ Parenthesised { inner = Tuple actual_arguments } ] ->
210                 List.fold_left
211                     (fun (lins, names, k) elt ->
212                         let elt_lin, k = linearize elt k in
213                         let elt_name = last_var elt_lin in
214                         (elt_lin :: lins, elt_name :: names, k))
215                     ([], [], k) actual_arguments
216             | [ argument ] ->
217                 let argument_lin, k = linearize argument k in
218                 let argument_name = last_var argument_lin in
219                 ([ argument_lin ], [ argument_name ], k)
220             | _ -> failwith "Type constructor had more than one argument"
221         in
222         let e_elt =
223             match argument_names with
224             | [ argument ] ->
225                 FunctionApplication
226                     {
227                         receiver = [ (p (k + 1), Constant c) ];
228                         arguments = [ [ (p (k + 2), Variable argument) ] ];
229                     }
230             | _ ->
231                 FunctionApplication
232                     {
233                         receiver = [ (p (k + 1), Constant c) ];
234                         arguments =
235                             [
236

```

5 – Annexes • 5.6 Code

```

237         [
238             ( p (k + 2),
239               Parenthesised
240               {
241                 style = Parenthesis;
242                 inner =
243                 [
244                     ( p (k + 3),
245                       Tuple
246                       (argument_names |> List.rev
247                       |> List.mapi (fun i arg ->
248                         [ (p (k + i + 4), Variable arg) ])
249                       ) );
250                 ];
251             } );
252         ];
253     ];
254 }
255
256 let argument_count = List.length argument_names in
257 ( (arguments_lins |> List.rev |> List.concat) @ [ (p k, e_elt) ],
258   k + argument_count + 4 )
259 | - ->
260 let receiver_lin, k = linearize receiver k in
261 let receiver_var = last_var receiver_lin in
262 let arg_lins, arg_names, k =
263   List.fold_left
264     (fun (lins, names, k) elt ->
265       let elt_lin, k = linearize elt k in
266       let elt_name = last_var elt_lin in
267       (elt_lin :: lins, elt_name :: names, k))
268     ([], [], k) arguments
269 in
270 let e_elt =

```

```

271     FunctionApplication
272     {
273         receiver = [ (p (k + 1), Variable receiver_var) ];
274         arguments =
275             arg_names |> List.rev
276             |> List.mapi (fun i name ->
277                 [ (p (k + i + 2), Variable name) ] );
278     }
279     in
280     let argument_count = List.length arguments in
281     ( receiver_lin
282       @ (arg_lins |> List.rev |> List.concat)
283       @ [ (p k, e_elt) ],
284       k + argument_count + 2 ))
285 | PrefixOperation { operation; receiver } ->
286     let receiver_lin, k = linearize receiver k in
287     let receiver_var = last_var receiver_lin in
288     let e_elt =
289         PrefixOperation
290         { operation; receiver = [ (p (k + 1), Variable receiver_var) ] }
291     in
292     (receiver_lin @ [ (p k, e_elt) ], k + 2)
293 | InfixOperation { lhs; rhs; operation } ->
294     let lhs_lin, k = linearize lhs k in
295     let lhs_var = last_var lhs_lin in
296     let rhs_lin, k = linearize rhs k in
297     let rhs_var = last_var rhs_lin in
298     let e_elt =
299         InfixOperation
300         {
301             operation;
302             lhs = [ (p (k + 1), Variable lhs_var) ];
303             rhs = [ (p (k + 2), Variable rhs_var) ];
304         }

```

5 – Annexes • 5.6 Code

```

305     in
306     (lhs_lin @ rhs_lin @ [ (p k, e_elt) ], k + 3)
307 | Negation inner ->
308     let inner_lin, k = linearize inner k in
309     let inner_var = last_var inner_lin in
310     let e_elt = Negation [ (p (k + 1), Variable inner_var) ] in
311     (inner_lin @ [ (p k, e_elt) ], k + 2)
312 | Tuple t ->
313     let elt_lins, elt_names, k =
314         List.fold_left
315             (fun (lins, names, k) elt ->
316                 let elt_lin, k = linearize elt k in
317                 let elt_name = last_var elt_lin in
318                 (elt_lin :: lins, elt_name :: names, k))
319             ([], [], k) t
320     in
321     let e_elt =
322         Tuple
323             (elt_names |> List.rev
324              |> List.mapi (fun i name -> [ (p (k + i + 1), Variable name) ]))
325     in
326     let elt_count = List.length elt_names in
327     ( (elt_lins |> List.rev |> List.concat) @ [ (p k, e_elt) ],
328       k + elt_count + 1 )
329 | FieldAssignment { receiver; target; value } ->
330     let receiver_lin, k = linearize receiver k in
331     let receiver_var = last_var receiver_lin in
332     let value_lin, k = linearize value k in
333     let value_var = last_var value_lin in
334     let e_elt =
335         FieldAssignment
336             {
337                 receiver = [ (p (k + 1), Variable receiver_var) ];
338                 target;

```

```

339         value = [ (p (k + 2), Variable value_var) ];
340     }
341     in
342     (receiver_lin @ value_lin @ [ (p k, e_elt) ], k + 3)
343 | ArrayAssignment { receiver; target; value } ->
344     let receiver_lin, k = linearize receiver k in
345     let receiver_var = last_var receiver_lin in
346     let target_lin, k = linearize target k in
347     let target_var = last_var target_lin in
348     let value_lin, k = linearize value k in
349     let value_var = last_var value_lin in
350     let e_elt =
351         ArrayAssignment
352         {
353             receiver = [ (p (k + 1), Variable receiver_var) ];
354             target = [ (p (k + 2), Variable target_var) ];
355             value = [ (p (k + 3), Variable value_var) ];
356         }
357     in
358     (receiver_lin @ target_lin @ value_lin @ [ (p k, e_elt) ], k + 4)
359 | ReferenceAssignment { receiver; value } ->
360     let receiver_lin, k = linearize receiver k in
361     let receiver_var = last_var receiver_lin in
362     let value_lin, k = linearize value k in
363     let value_var = last_var value_lin in
364     let e_elt =
365         ReferenceAssignment
366         {
367             receiver = [ (p (k + 1), Variable receiver_var) ];
368             value = [ (p (k + 2), Variable value_var) ];
369         }
370     in
371     (receiver_lin @ value_lin @ [ (p k, e_elt) ], k + 3)
372 | If { condition; body; else_body } ->

```

5 – Annexes • 5.6 Code

```

373     let condition_lin, k = linearize condition k in
374     let condition_var = last_var condition_lin in
375     let body_lin, k = linearize body k in
376     let else_body_lin, k =
377       match else_body with
378       | None -> (None, k)
379       | Some else_body ->
380         let else_body_lin, k = linearize else_body k in
381         (Some else_body_lin, k)
382   in
383   let e_elt =
384     If
385     {
386       condition = [ (p (k + 2), Variable condition_var) ];
387       body = body_lin;
388       else_body = else_body_lin;
389     }
390   in
391   (condition_lin @ [ (p k, e_elt) ], k + 3)
392 | Sequence s ->
393   let elt_lins, k =
394     List.fold_left
395     (fun (lins, k) elt ->
396       let elt_lin, k = linearize elt k in
397       (elt_lin :: lins, k))
398     ([], k) s
399   in
400   (elt_lins |> List.rev |> List.concat, k)
401 | Match { value; cases } ->
402   let value_lin, k = linearize value k in
403   let value_var = last_var value_lin in
404   let orig_k = k in
405   let cases_lins, k =
406     List.fold_left

```

5 – Annexes • 5.6 Code

```

407     (fun (lins, k) (pattern, body) ->
408       let body_lin, k = linearize body k in
409       ((pattern, body_lin) :: lins, k))
410     ([], orig_k + 2)
411     cases
412   in
413   let e_elt =
414     Match
415     {
416       value = [ (p (orig_k + 1), Variable value_var) ];
417       cases = List.rev cases_lins;
418     }
419   in
420   (value_lin @ [ (p orig_k, e_elt) ], k)
421 | Try _ -> failwith "Cannot linearize try expressions"
422 | FunctionLiteral { style; cases } ->
423   let orig_k = k in
424   let cases_lins, k =
425     List.fold_left
426       (fun (lins, k) (pattern, body) ->
427         let body_lin, k = linearize body k in
428         ((pattern, body_lin) :: lins, k))
429       ([], orig_k + 1)
430     cases
431   in
432   let e_elt =
433     FunctionLiteral
434     {
435       style;
436       cases = List.rev cases_lins;
437       return_type_for_delinearize = None;
438     }
439   in
440   ([ (p orig_k, e_elt) ], k)

```


5 – Annexes • 5.6 Code

```

441 | LetBinding { bindings; is_rec; inner } ->
442 |   let variable_bindings_lins = ref [] in
443 |   let inner_lin, k = linearize inner k in
444 |   let k_ref = ref k in
445 |   let e_elt =
446 |     LetBinding
447 |     {
448 |       bindings =
449 |         bindings
450 |         |> List.map (fun (binding : binding) ->
451 |           match binding with
452 |           | Variable { lhs; value } ->
453 |             let value_lin, k = linearize value !k_ref in
454 |             k_ref := k + 1;
455 |             variable_bindings_lins :=
456 |               value_lin :: !variable_bindings_lins;
457 |             (Variable
458 |               {
459 |                 lhs;
460 |                 value =
461 |                   [ (p (k + 1), Variable (last_var value_lin)) ];
462 |               }
463 |               : linear_binding)
464 |           | Function { name; parameters; body; return_type } ->
465 |             let body_lin, k = linearize body !k_ref in
466 |             k_ref := k;
467 |             Function
468 |             { name; parameters; body = body_lin; return_type });
469 |     is_rec;
470 |     inner = inner_lin;
471 |   }
472 | in
473 | ([ (p !k_ref, e_elt) ] :: !variable_bindings_lins
474 |   |> List.rev |> List.concat,

```

5 – Annexes • 5.6 Code

```

475     !k_ref + 1 )
476   | StringAccess { target; receiver } ->
477     let receiver_lin, k = linearize receiver k in
478     let receiver_var = last_var receiver_lin in
479     let target_lin, k = linearize target k in
480     let target_var = last_var target_lin in
481     let e_elt =
482       ArrayAccess
483       {
484         receiver = [ (p (k + 1), Variable receiver_var) ];
485         target = [ (p (k + 2), Variable target_var) ];
486       }
487     in
488     (receiver_lin @ target_lin @ [ (p k, e_elt) ], k + 3)
489   | StringAssignment { receiver; target; value } ->
490     let receiver_lin, k = linearize receiver k in
491     let receiver_var = last_var receiver_lin in
492     let target_lin, k = linearize target k in
493     let target_var = last_var target_lin in
494     let value_lin, k = linearize value k in
495     let value_var = last_var value_lin in
496     let e_elt =
497       ArrayAssignment
498       {
499         receiver = [ (p (k + 1), Variable receiver_var) ];
500         target = [ (p (k + 2), Variable target_var) ];
501         value = [ (p (k + 3), Variable value_var) ];
502       }
503     in
504     (receiver_lin @ target_lin @ value_lin @ [ (p k, e_elt) ], k + 4)
505
506 let rec element_contains_reference (f : variable) (e : linear_element) : bool =
507   match e with
508   | Variable v -> v = f

```

```

509 | Constant _ -> false
510 | Parenthesised { inner } | TypeCoercion { inner } ->
511   contains_reference f inner
512 | ListLiteral l | ArrayLiteral l -> List.exists (contains_reference f) l
513 | RecordLiteral l -> l |> List.map snd |> List.exists (contains_reference f)
514 | WhileLoop _ -> failwith "Found linearized while loop"
515 | ForLoop _ -> failwith "Found linearized for loop"
516 | Dereference inner -> contains_reference f inner
517 | FieldAccess { receiver } -> contains_reference f receiver
518 | ArrayAccess { receiver; target } ->
519   contains_reference f receiver || contains_reference f target
520 | FunctionApplication { receiver; arguments } ->
521   contains_reference f receiver
522   || List.exists (contains_reference f) arguments
523 | PrefixOperation { receiver } -> contains_reference f receiver
524 | InfixOperation { lhs; rhs } ->
525   contains_reference f lhs || contains_reference f rhs
526 | Negation inner -> contains_reference f inner
527 | Tuple l -> List.exists (contains_reference f) l
528 | FieldAssignment { receiver; value } ->
529   contains_reference f receiver || contains_reference f value
530 | ArrayAssignment { receiver; target; value } ->
531   contains_reference f receiver
532   || contains_reference f target
533   || contains_reference f value
534 | ReferenceAssignment { receiver; value } ->
535   contains_reference f receiver || contains_reference f value
536 | If { condition; body; else_body } -> (
537   contains_reference f condition
538   || contains_reference f body
539   || match else_body with None -> false | Some b -> contains_reference f b)
540 | Sequence s -> List.exists (contains_reference f) s
541 | Match { value; cases } ->
542   contains_reference f value

```

5 – Annexes • 5.6 Code

```

543     || cases |> List.map snd |> List.exists (contains_reference f)
544 | Try _ -> failwith "Found linearized try"
545 | FunctionLiteral { cases } ->
546     List.exists (fun (_, body) -> contains_reference f body) cases
547 | LetBinding { bindings; inner } ->
548     contains_reference f inner
549     || bindings
550     |> List.exists (fun (binding : linear_binding) ->
551         match binding with
552         | Variable { value } -> contains_reference f value
553         | Function { body } -> contains_reference f body)
554 | StringAccess { receiver; target } ->
555     contains_reference f receiver || contains_reference f target
556 | StringAssignment { receiver; target; value } ->
557     contains_reference f receiver
558     || contains_reference f target
559     || contains_reference f value
560
561 (** contains_reference f l is true iff l contains `Variable f` *)
562 and contains_reference (f : variable) (l : linear_form) : bool =
563     match l with
564     | [] -> false
565     | (p, e) :: q -> element_contains_reference f e || contains_reference f q
566
567 (** Given an element and the definitions of variables that may appear in the
568 element, returns an expression equivalent to the element and list of
569 variables that were inlined from their definitions. *)
570 let rec delinearize_element (e : linear_element) (prev_vars : linear_form) :
571     expression * variable list =
572     match e with
573     | Variable v -> (
574         match List.assoc_opt v prev_vars with
575         | None -> (Variable v, [])
576         | Some definition ->

```

5 – Annexes • 5.6 Code

```

577     let inlined_elt, inlined_vars =
578         delinearize_element definition
579         (List.filter (fun (name, _) -> name <> v) prev_vars)
580     in
581     (* Add parenthesis to prevent any precedence issues. *)
582     ( Parenthesised { inner = inlined_elt; style = Parenthesis },
583       v :: inlined_vars ))
584 | Constant c -> (Constant c, [])
585 | Parenthesised { style; inner } ->
586     let inner_inlined, inlined_vars = delinearize inner prev_vars in
587     (Parenthesised { style; inner = inner_inlined }, inlined_vars)
588 | TypeCoercion { typ; inner } ->
589     let inner_inlined, inlined_vars = delinearize inner prev_vars in
590     (TypeCoercion { typ; inner = inner_inlined }, inlined_vars)
591 | ListLiteral l ->
592     let terms, inlined_vars =
593         List.fold_right
594             (fun elt (terms, inlined_vars) ->
595                 let elt_inlined, inlined_vars' = delinearize elt prev_vars in
596                 (elt_inlined :: terms, inlined_vars' @ inlined_vars))
597             l ([], [])
598     in
599     (ListLiteral terms, inlined_vars)
600 | ArrayLiteral l ->
601     let terms, inlined_vars =
602         List.fold_right
603             (fun elt (terms, inlined_vars) ->
604                 let elt_inlined, inlined_vars' = delinearize elt prev_vars in
605                 (elt_inlined :: terms, inlined_vars' @ inlined_vars))
606             l ([], [])
607     in
608     (ArrayLiteral terms, inlined_vars)

```

```

611 | RecordLiteral l ->
612 |   let terms, inlined_vars =
613 |     List.fold_right
614 |       (fun (name, elt) (terms, inlined_vars) ->
615 |         let elt_inlined, inlined_vars' = delinearize elt prev_vars in
616 |         ((name, elt_inlined) :: terms, inlined_vars' @ inlined_vars))
617 |     1 ([], [])
618 |   in
619 |
620 |   (RecordLiteral terms, inlined_vars)
621 | WhileLoop _ -> failwith "Found linearized while loop"
622 | ForLoop _ -> failwith "Found linearized for loop"
623 | Dereference inner ->
624 |   let inner_inlined, inlined_vars = delinearize inner prev_vars in
625 |   (Dereference inner_inlined, inlined_vars)
626 | FieldAccess { receiver; target } ->
627 |   let receiver_inlined, inlined_vars = delinearize receiver prev_vars in
628 |   (FieldAccess { receiver = receiver_inlined; target }, inlined_vars)
629 | ArrayAccess { receiver; target } ->
630 |   let receiver_inlined, inlined_vars = delinearize receiver prev_vars in
631 |   let target_inlined, inlined_vars_2 = delinearize target prev_vars in
632 |   ( ArrayAccess { receiver = receiver_inlined; target = target_inlined },
633 |     inlined_vars @ inlined_vars_2 )
634 | FunctionApplication { receiver; arguments } ->
635 |   let receiver_inlined, inlined_vars = delinearize receiver prev_vars in
636 |   let arguments_inlined, inlined_vars_2 =
637 |     List.fold_right
638 |       (fun elt (terms, inlined_vars) ->
639 |         let elt_inlined, inlined_vars' = delinearize elt prev_vars in
640 |         (elt_inlined :: terms, inlined_vars' @ inlined_vars))
641 |       arguments ([], [])
642 |   in
643 |   ( FunctionApplication
644 |     { receiver = receiver_inlined; arguments = arguments_inlined },

```

5 – Annexes • 5.6 Code

```

645         inlined_vars @ inlined_vars_2 )
646 | PrefixOperation { operation; receiver } ->
647   let receiver_inlined, inlined_vars = delinearize receiver prev_vars in
648   (PrefixOperation { operation; receiver = receiver_inlined }, inlined_vars)
649 | InfixOperation { lhs; operation; rhs } ->
650   let lhs_inlined, inlined_vars = delinearize lhs prev_vars in
651   let rhs_inlined, inlined_vars2 = delinearize rhs prev_vars in
652   ( InfixOperation { lhs = lhs_inlined; operation; rhs = rhs_inlined },
653     inlined_vars @ inlined_vars2 )
654 | Negation inner ->
655   let inner_inlined, inlined_vars = delinearize inner prev_vars in
656   (Negation inner_inlined, inlined_vars)
657 | Tuple 1 ->
658   let terms, inlined_vars =
659     List.fold_right
660       (fun elt (terms, inlined_vars) ->
661         let elt_inlined, inlined_vars' = delinearize elt prev_vars in
662         (elt_inlined :: terms, inlined_vars' @ inlined_vars))
663       1 ([], [])
664   in
665
666   (Tuple terms, inlined_vars)
667 | FieldAssignment { receiver; target; value } ->
668   let receiver_inlined, inlined_vars = delinearize receiver prev_vars in
669   let value_inlined, inlined_vars_2 = delinearize value prev_vars in
670   ( FieldAssignment
671     { receiver = receiver_inlined; target; value = value_inlined },
672     inlined_vars @ inlined_vars_2 )
673 | ArrayAssignment { receiver; target; value } ->
674   let receiver_inlined, inlined_vars = delinearize receiver prev_vars in
675   let value_inlined, inlined_vars_2 = delinearize value prev_vars in
676   let target_inlined, inlined_vars_3 = delinearize value prev_vars in
677   ( ArrayAssignment
678     {

```

5 – Annexes • 5.6 Code

```

679         receiver = receiver_inlined;
680         target = target_inlined;
681         value = value_inlined;
682     },
683     inlined_vars @ inlined_vars_2 @ inlined_vars_3 )
684 | ReferenceAssignment { receiver; value } ->
685     let receiver_inlined, inlined_vars = delinearize receiver prev_vars in
686     let value_inlined, inlined_vars_2 = delinearize value prev_vars in
687     ( ReferenceAssignment
688       { receiver = receiver_inlined; value = value_inlined },
689       inlined_vars @ inlined_vars_2 )
690 | If { condition; body; else_body } ->
691     let condition_inlined, inlined_vars = delinearize condition prev_vars in
692     let body_inlined, inlined_vars_2 = delinearize body prev_vars in
693     let else_inlined, inlined_vars_3 =
694         match else_body with
695         | None -> (None, [])
696         | Some b ->
697             let b_inlined, inlined_vars = delinearize b prev_vars in
698             (Some b_inlined, inlined_vars)
699     in
700     ( If
701       {
702         condition = condition_inlined;
703         body = body_inlined;
704         else_body = else_inlined;
705       },
706       inlined_vars @ inlined_vars_2 @ inlined_vars_3 )
707 | Sequence s ->
708     let terms, inlined_vars =
709         List.fold_right
710           (fun elt (terms, inlined_vars) ->
711             let elt_inlined, inlined_vars' = delinearize elt prev_vars in
712             (elt_inlined :: terms, inlined_vars' @ inlined_vars))

```


5 – Annexes • 5.6 Code

```

713     s ([], [])
714   in
715
716   (Sequence terms, inlined_vars)
717 | Match { value; cases } ->
718   let value_inlined, inlined_vars = delinearize value prev_vars in
719   let cases_inlined, inlined_vars_2 =
720     List.fold_right
721       (fun (pattern, elt) (terms, inlined_vars) ->
722         let elt_inlined, inlined_vars' = delinearize elt prev_vars in
723         ((pattern, elt_inlined) :: terms, inlined_vars' @ inlined_vars))
724     cases ([], [])
725   in
726
727   ( Match { value = value_inlined; cases = cases_inlined },
728     inlined_vars @ inlined_vars_2 )
729 | Try _ -> failwith "Found linearized try"
730 | FunctionLiteral { style; cases } ->
731   let cases_inlined, inlined_vars =
732     List.fold_right
733       (fun (pattern, elt) (terms, inlined_vars) ->
734         let elt_inlined, inlined_vars' = delinearize elt prev_vars in
735         ((pattern, elt_inlined) :: terms, inlined_vars' @ inlined_vars))
736     cases ([], [])
737   in
738   (FunctionLiteral { style; cases = cases_inlined }, inlined_vars)
739 | LetBinding { bindings; is_rec; inner } ->
740   let bindings_inlined, inlined_vars =
741     List.fold_right
742       (fun binding (terms, inlined_vars) ->
743         match binding with
744         | (Variable { lhs; value } : linear_binding) ->
745           let value_inlined, inlined_vars' =
746             delinearize value prev_vars

```

5 – Annexes • 5.6 Code

```

747         in
748         ( (Variable { lhs; value = value_inlined } : binding) :: terms,
749           inlined_vars' @ inlined_vars )
750     | Function { name; parameters; body; return_type } ->
751       let body_inlined, inlined_vars' = delinearize body prev_vars in
752       ( Function { name; parameters; body = body_inlined; return_type }
753         :: terms,
754         inlined_vars' @ inlined_vars ))
755     bindings ([], [])
756   in
757   let inner_inlined, inlined_vars_2 = delinearize inner prev_vars in
758   ( LetBinding { bindings = bindings_inlined; is_rec; inner = inner_inlined },
759     inlined_vars @ inlined_vars_2 )
760 | StringAccess { receiver; target } ->
761   let receiver_inlined, inlined_vars = delinearize receiver prev_vars in
762   let target_inlined, inlined_vars_2 = delinearize target prev_vars in
763   ( StringAccess { receiver = receiver_inlined; target = target_inlined },
764     inlined_vars @ inlined_vars_2 )
765 | StringAssignment { receiver; target; value } ->
766   let receiver_inlined, inlined_vars = delinearize receiver prev_vars in
767   let value_inlined, inlined_vars_2 = delinearize value prev_vars in
768   let target_inlined, inlined_vars_3 = delinearize target prev_vars in
769   ( StringAssignment
770     {
771       receiver = receiver_inlined;
772       target = target_inlined;
773       value = value_inlined;
774     },
775     inlined_vars @ inlined_vars_2 @ inlined_vars_3 )
776
777 (** delinearize l prev_vars converts a linear form l to an OCaml expression.)
778
779 In addition to the algorithm detailed in the paper, three additional
780 modifications are performed to improve legibility:

```

5 – Annexes • 5.6 Code

```

781
782   - Use of function bindings: Instead of generating bindings of the form `let
783     x = fun arg1 ... argn -> body`, generate `let x arg1 ... argn = body`
784
785   - Inlining: If a variable is encountered and prev_vars contains a definition
786     for that variable, replace the occurrence of the variable with the
787     definition. This helps to undo the "flattening" introduced by linearize.
788
789   - Use of sequences: Instead of generating `let x = value in e`, if `x` does
790     not appear in `e`, generate `value; e`.
791
792   This function returns the generated expression as well as a list of
793   variables that were inlined. *)
794 and delinearize (l : linear_form) (prev_vars : linear_form) :
795   expression * variable list =
796   match l with
797   | [] -> failwith "Empty linear form"
798   | (p, e) :: [] -> delinearize_element e prev_vars
799   (* Special case for formatting functions nicely *)
800   | ( p,
801       FunctionLiteral
802         { style; cases = [ (args, body) ]; return_type_for_delinearize } )
803   :: q ->
804     let q_inlined, inlined_vars = delinearize q prev_vars in
805     let body_inlined, inlined_vars_2 = delinearize body prev_vars in
806     ( LetBinding
807       {
808         is_rec = false;
809         bindings =
810           [
811             Function
812             {
813               name = p;
814               parameters = args;

```

5 – Annexes • 5.6 Code

```

815         body = body_inlined;
816         return_type = return_type_for_delinearize;
817     };
818 ];
819     inner = q_inlined;
820 },
821 inlined_vars @ inlined_vars_2 )
822 | (p, e) :: q ->
823   let q_delinearized, inlined_vars = delinearize q ((p, e) :: prev_vars) in
824   if List.mem p inlined_vars then (q_delinearized, inlined_vars)
825   else
826     let e_inlined, inlined_vars_2 = delinearize_element e prev_vars in
827     if contains_reference p q then
828       ( LetBinding
829         {
830           is_rec = false;
831           bindings = [ Variable { lhs = Ident p; value = e_inlined } ];
832           inner = q_delinearized;
833         },
834         inlined_vars @ inlined_vars_2 )
835     else
836       ( Parenthesised
837         {
838           inner = Sequence [ e_inlined; q_delinearized ];
839           style = Parenthesis;
840         },
841         inlined_vars @ inlined_vars_2 )
842
843 let rec element_contains_application (f : variable) (e : linear_element) : bool
844 =
845   match e with
846   | Variable _ -> false
847   | Constant _ -> false
848   | Parenthesised { inner } | TypeCoercion { inner } ->

```

```

849     contains_application f inner
850 | ListLiteral l | ArrayLiteral l -> List.exists (contains_application f) l
851 | RecordLiteral l -> l |> List.map snd |> List.exists (contains_application f)
852 | WhileLoop _ -> failwith "Found linearized while loop"
853 | ForLoop _ -> failwith "Found linearized for loop"
854 | Dereference inner -> contains_application f inner
855 | FieldAccess { receiver } -> contains_application f receiver
856 | ArrayAccess { receiver; target } ->
857     contains_application f receiver || contains_application f target
858 | FunctionApplication { receiver; arguments } ->
859     (match receiver with [ (_, Variable f') ] -> f' = f | _ -> false)
860     || contains_application f receiver
861     || List.exists (contains_application f) arguments
862 | PrefixOperation { receiver } -> contains_application f receiver
863 | InfixOperation { lhs; rhs } ->
864     contains_application f lhs || contains_application f rhs
865 | Negation inner -> contains_application f inner
866 | Tuple l -> List.exists (contains_application f) l
867 | FieldAssignment { receiver; value } ->
868     contains_application f receiver || contains_application f value
869 | ArrayAssignment { receiver; target; value } ->
870     contains_application f receiver
871     || contains_application f target
872     || contains_application f value
873 | ReferenceAssignment { receiver; value } ->
874     contains_application f receiver || contains_application f value
875 | If { condition; body; else_body } -> (
876     contains_application f condition
877     || contains_application f body
878     ||
879     match else_body with None -> false | Some b -> contains_application f b)
880 | Sequence s -> List.exists (contains_application f) s
881 | Match { value; cases } ->
882     contains_application f value

```

5 – Annexes • 5.6 Code

```

883     || cases |> List.map snd |> List.exists (contains_application f)
884 | Try _ -> failwith "Found linearized try"
885 (* TODO: Formalize "contains" to better explain this. *)
886 | FunctionLiteral { cases } -> false
887 | LetBinding { bindings; inner } ->
888     contains_application f inner
889     || bindings
890     |> List.exists (fun (binding : linear_binding) ->
891         match binding with
892         | Variable { value } -> contains_application f value
893         | Function { body } -> false)
894 | StringAccess { receiver; target } ->
895     contains_application f receiver || contains_application f target
896 | StringAssignment { receiver; target; value } ->
897     contains_application f receiver
898     || contains_application f target
899     || contains_application f value
900
901 (** contains_application f l is true iff l contains a FunctionApplication whose
902 receiver is f **)
903 and contains_application (f : variable) (l : linear_form) : bool =
904     match l with
905     | [] -> false
906     | (p, e) :: q -> element_contains_application f e || contains_application f q
907
908 (** map_locally_terminal_children applies a transformation only to the locally
909 terminal children of f **)
910 let map_locally_terminal_children (f : linear_form -> linear_form)
911     (e : linear_element) : linear_element =
912     match (e : linear_element) with
913     | If { condition; body; else_body } ->
914         If
915         {
916             condition;

```

5 – Annexes • 5.6 Code

```

917         body = f body;
918     else_body =
919         (match else_body with None -> None | Some b -> Some (f b));
920     }
921 | Match { value; cases } ->
922     Match
923     {
924         value;
925         cases = List.map (fun (pattern, body) -> (pattern, f body)) cases;
926     }
927 | LetBinding { is_rec; bindings; inner } ->
928     LetBinding { is_rec; bindings; inner = f inner }
929 | Parenthesised { style; inner } -> Parenthesised { style; inner = f inner }
930 | TypeCoercion { inner; typ } -> TypeCoercion { inner = f inner; typ }
931 (* No locally terminal children *)
932 | _ -> e
933
934 let rec get_name (p : pattern) : variable option =
935     match p with
936     | Ident n -> Some n
937     | Underscore -> None
938     | Parenthesised inner -> get_name inner
939     | TypeCoercion { inner } -> get_name inner
940     | Constant _ -> None
941     | Record _ -> None
942     | List _ -> None
943     | Construction _ -> None
944     | Concatenation _ -> None
945     | Tuple _ -> None
946     | Or _ -> None
947     | As { name } -> Some name
948
949 let rec find_definitions_in_element (name : variable) (e : linear_element) :
950     linear_element list =

```

```

951 match e with
952 | Variable _ | Constant _ -> []
953 | Parenthesised { inner } | TypeCoercion { inner } | Dereference inner ->
954   find_definitions name inner
955 | ListLiteral l | ArrayLiteral l | Tuple l | Sequence l ->
956   l |> List.map (fun elt -> find_definitions name elt) |> List.concat
957 | RecordLiteral r ->
958   r |> List.map (fun (_, elt) -> find_definitions name elt) |> List.concat
959 | WhileLoop _ | ForLoop _ -> failwith "Found linearized loop"
960 | FieldAccess { receiver } -> find_definitions name receiver
961 | ArrayAccess { receiver; target } | StringAccess { receiver; target } ->
962   find_definitions name receiver @ find_definitions name target
963 | FunctionApplication { receiver; arguments } ->
964   find_definitions name receiver
965   @ (arguments
966     |> List.map (fun arg -> find_definitions name arg)
967     |> List.concat)
968 | PrefixOperation { receiver } -> find_definitions name receiver
969 | InfixOperation { lhs; rhs } ->
970   find_definitions name lhs @ find_definitions name rhs
971 | Negation receiver -> find_definitions name receiver
972 | FieldAssignment { receiver; value } ->
973   find_definitions name receiver @ find_definitions name value
974 | ArrayAssignment { receiver; target; value }
975 | StringAssignment { receiver; target; value } ->
976   find_definitions name receiver
977   @ find_definitions name target
978   @ find_definitions name value
979 | ReferenceAssignment { receiver; value } ->
980   find_definitions name receiver @ find_definitions name value
981 | If { condition; body; else_body } -> (
982   find_definitions name condition
983   @ find_definitions name body
984   @ match else_body with None -> [] | Some b -> find_definitions name b)

```


5 – Annexes • 5.6 Code

```

985 | Match { value; cases } ->
986   find_definitions name value
987   @ (cases
988     |> List.map (fun (_, body) -> find_definitions name body)
989     |> List.concat)
990 | Try _ -> failwith "Found linearized try"
991 | FunctionLiteral { cases } ->
992   List.map (fun (_, body) -> find_definitions name body) cases
993   |> List.concat
994 | LetBinding { bindings; inner } ->
995   find_definitions name inner
996   @ (bindings
997     |> List.map (fun (binding : linear_binding) ->
998       match binding with
999       | Variable { lhs; value } ->
1000         (if get_name lhs = Some name then
1001           [ Variable (last_var value) ]
1002           else [])
1003         @ find_definitions name value
1004       | Function { name = name'; parameters; body; return_type } ->
1005         (* It's OK to create a synthetic element here so long as we
1006            don't expect to find it in a later AST search. As it
1007            currently stands, that is indeed the case. *)
1008         (if name' = name then
1009           [
1010             FunctionLiteral
1011             {
1012               style = Fun;
1013               cases = [ (parameters, body) ];
1014               return_type_for_delinearize = return_type;
1015             }
1016           ]
1017           else [])
1018     @ List.filter_map

```

```

1019 (fun parameter ->
1020   let rec create_synthetic_function_from (p : pattern) :
1021     linear_element option =
1022     match p with
1023     | Parenthesised inner ->
1024       create_synthetic_function_from inner
1025     | TypeCoercion { inner; typ } -> (
1026       match get_name inner with
1027       | Some name' when name = name' -> (
1028         let rec extract_parameters_and_return_type
1029           (e : type_expression) :
1030             type_expression list * type_expression =
1031           match e with
1032           | Argument _ -> ([], e)
1033           | Parenthesised _ -> ([], e)
1034           | Construction _ -> ([], e)
1035           | Tuple _ -> ([], e)
1036           | Function { argument; result } ->
1037             let inner_params, inner_ret =
1038               extract_parameters_and_return_type
1039                 result
1040             in
1041             (argument :: inner_params, inner_ret)
1042         in
1043         match
1044           extract_parameters_and_return_type typ
1045         with
1046         | [], _ -> None
1047         | parameter_types, return_type ->
1048           Some
1049             (FunctionLiteral
1050               {
1051                 style = Fun;
1052                 cases =

```

```

1053 [
1054   ( List.mapi
1055     (fun i parameter_type ->
1056       (Parenthesised
1057         (TypeCoercion
1058           {
1059             inner =
1060               Ident
1061                 ("synthetic_arg_"
1062                  ^ string_of_int
1063                    i);
1064             typ =
1065               parameter_type;
1066           })
1067         : pattern))
1068     parameter_types,
1069     [
1070       ( "SYNTHETIC_BODY",
1071         Constant
1072           (Construction
1073             (Unit Parenthesis))
1074       );
1075     ] );
1076   ];
1077   return_type_for_delinearize =
1078     Some return_type;
1079   )))
1080   | _ -> None)
1081   | _ -> None
1082   in
1083     create_synthetic_function_from parameter)
1084   parameters
1085   @ find_definitions name body)
1086   |> List.concat)

```

```

1087
1088 and find_definitions (name : variable) (l : linear_form) : linear_element list =
1089   let child_definitions =
1090     1
1091     |> List.map (fun (_, e) -> find_definitions_in_element name e)
1092     |> List.flatten
1093   in
1094   let self_definitions =
1095     1
1096     |> List.filter_map (fun (variable, elt) ->
1097       if variable = name then Some elt else None)
1098   in
1099
1100   child_definitions @ self_definitions
1101
1102   (** find_definition fns name finds the unique definition of the linear element
1103     with the name name in fns.
1104
1105     Returns None if zero or multiple definitions are found. *)
1106 and find_definition (fns : (variable * linear_element) list) (name : variable) :
1107   linear_element option =
1108   let root_definitions =
1109     fns
1110     |> List.filter_map (fun (name', def) ->
1111       if name' = name then Some def else None)
1112   in
1113   let inner_definitions =
1114     fns |> List.map snd
1115     |> List.map (find_definitions_in_element name)
1116     |> List.concat
1117   in
1118   let definitions = root_definitions @ inner_definitions in
1119   match definitions with [ def ] -> Some def | _ -> None
1120

```

5 – Annexes • 5.6 Code

```

1121  (** rectify l c returns a linear form equivalent to l in which all calls to
1122  functions in c are made terminal. *)
1123  let rec rectify (l : linear_form) (cloture_rect : variable list)
1124  (fns : linear_form) : linear_form =
1125  match l with
1126  | [] -> failwith "Empty linear form (rectify)"
1127  | (name, e) :: q -> (
1128    (* If e is a let-binding, update any recursive functions defined in e. *)
1129    let e =
1130      match e with
1131      | LetBinding { bindings; inner; is_rec } ->
1132        LetBinding
1133          {
1134            bindings =
1135              List.map
1136                (fun binding ->
1137                  match binding with
1138                  | Function { name; parameters; body; return_type } ->
1139                    let updated_parameters =
1140                      List.map
1141                        (fun parameter ->
1142                          match get_name parameter with
1143                          | Some name when List.mem name cloture_rect ->
1144                            let rec update_type_expr
1145                              (t : type_expression) : type_expression
1146                              =
1147                                match (t : type_expression) with
1148                                | Function { argument; result } ->
1149                                  Function
1150                                    {
1151                                      argument;
1152                                      result = update_type_expr result;
1153                                    }
1154                                | _ ->

```

```

1155         Function
1156     {
1157         argument =
1158             Parenthesised
1159             (Function
1160             {
1161                 argument =
1162                     Parenthesised t;
1163                 result = Argument "ret";
1164             });
1165         result = Argument "ret";
1166     }
1167 in
1168 let rec update_parameter_type (p : pattern)
1169     : pattern =
1170     match (p : pattern) with
1171     | Parenthesised inner ->
1172         Parenthesised
1173         (update_parameter_type inner)
1174     | TypeCoercion { inner; typ } ->
1175         TypeCoercion
1176         {
1177             inner;
1178             typ = update_type_expr typ;
1179         }
1180     | _ -> p
1181 in
1182     update_parameter_type parameter
1183 | _ -> parameter)
1184 parameters
1185 in
1186 if List.mem name cloture_rect then
1187     Function
1188     {

```

```

1189         name;
1190         parameters =
1191             updated_parameters
1192         @ [
1193             (match return_type with
1194             | None -> Ident "cont"
1195             | Some typ ->
1196                 TypeCoercion
1197                 {
1198                     inner = Ident "cont";
1199                     typ =
1200                         Function
1201                         {
1202                             argument = typ;
1203                             result = Argument "ret";
1204                         };
1205                     });
1206             ];
1207         body = rectify body cloture_rect fns;
1208         return_type =
1209             (match return_type with
1210             | None -> None
1211             | Some _ -> Some (Argument "ret"));
1212     }
1213 else
1214     Function
1215     {
1216         name;
1217         parameters = updated_parameters;
1218         body;
1219         return_type;
1220     }
1221 | _ -> binding)
1222 bindings;

```

5 – Annexes • 5.6 Code

```

1223         inner;
1224         is_rec;
1225     }
1226     | _ -> e
1227 in
1228
1229 if List.mem name cloture_rect then
1230     (* This is a definition of a function that needs rectifying. *)
1231     (* Assumption: these definitions do not invoke anything recursively. *)
1232     let q_rect =
1233         match q with [] -> [] | _ -> rectify q cloture_rect fns
1234     in
1235     match e with
1236     | Variable v -> (name, e) :: q_rect
1237     | FunctionLiteral { style; cases; return_type_for_delinearize } ->
1238         let e_updated =
1239             FunctionLiteral
1240             {
1241                 style;
1242                 cases =
1243                     List.map
1244                     (fun (patterns, body) ->
1245                         ( patterns
1246                             @ [
1247                                 (match return_type_for_delinearize with
1248                                 | None -> Ident "cont"
1249                                 | Some typ ->
1250                                     TypeCoercion
1251                                     {
1252                                         inner = Ident "cont";
1253                                         typ =
1254                                             Function
1255                                             {
1256

```



```

1257         result = Argument "ret";
1258     };
1259     });
1260 ],
1261     rectify body cloture_rect fns ))
1262     cases;
1263     return_type_for_delinearize =
1264     (match return_type_for_delinearize with
1265     | None -> None
1266     | Some _ -> Some (Argument "ret"));
1267 }
1268 in
1269     (name, e_updated) :: q_rect
1270 | _ -> failwith "Unknown definition type for rectified function"
1271 else if
1272     List.exists (fun fn -> element_contains_application fn e) cloture_rect
1273 then
1274     let e_rect =
1275         match e with
1276         | FunctionApplication { receiver; arguments } ->
1277             FunctionApplication
1278             {
1279                 receiver;
1280                 arguments = arguments @ [ [ ("cont_arg", Variable "cont") ] ];
1281             }
1282         | _ ->
1283             map_locally_terminal_children
1284             (fun child -> rectify child cloture_rect fns)
1285             e
1286     in
1287     match q with
1288     | [] -> [ (name, e_rect) ]
1289     | _ ->
1290         let e_return_type =

```

```

1291         match e_rect with
1292         | FunctionApplication { receiver = [ (p, _) ] } ->
1293             let rec find_return_type (v : variable) :
1294                 type_expression option =
1295                 match find_definition fns v with
1296                 | Some (FunctionLiteral { return_type_for_delinearize }) ->
1297                     return_type_for_delinearize
1298                 | Some (Variable v2) -> find_return_type v2
1299                 | _ -> None
1300             in
1301             find_return_type p
1302         | _ -> None
1303     in
1304
1305     let q_rect = rectify q cloture_rect fns in
1306     let new_cont =
1307         FunctionLiteral
1308         {
1309             style = Fun;
1310             cases =
1311             [
1312                 ( [
1313                     (match e_return_type with
1314                     | None -> Ident name
1315                     | Some typ -> TypeCoercion { inner = Ident name; typ });
1316                 ],
1317                 q_rect );
1318             ];
1319             return_type_for_delinearize = Some (Argument "ret");
1320         }
1321     in
1322     [
1323         ("new_cont", new_cont);
1324         ("cont", Variable "new_cont");

```

5 – Annexes • 5.6 Code

```

1325         ("rec_call", e_rect);
1326     ]
1327     else
1328         match q with
1329         | [] ->
1330             [
1331                 (name, e);
1332                 ( "cont_call",
1333                   FunctionApplication
1334                     {
1335                         receiver = [ ("cont_ref", Variable "cont") ];
1336                         arguments = [ [ ("res_ref", Variable name) ] ];
1337                     } );
1338             ]
1339         | _ -> (name, e) :: rectify q cloture_rect fns)
1340
1341 let rec rename_elements_in (e : linear_element) (cloture_rect : variable list)
1342   (new_name : variable -> variable) =
1343 let rec rename_elements_in_pattern (p : pattern) : pattern =
1344   match (p : pattern) with
1345   | Ident name ->
1346       if List.mem name cloture_rect then Ident (new_name name) else p
1347   | Underscore -> p
1348   | Parenthesised inner -> Parenthesised (rename_elements_in_pattern inner)
1349   | TypeCoercion { inner; typ } ->
1350       TypeCoercion { inner = rename_elements_in_pattern inner; typ }
1351   | Constant _ -> p
1352   | Record l ->
1353       Record
1354         (List.map
1355          (fun (label, inner) -> (label, rename_elements_in_pattern inner))
1356          l)
1357   | List l -> List (List.map rename_elements_in_pattern l)
1358   | Construction { constructor; argument } ->

```

```

1359         Construction
1360         { constructor; argument = rename_elements_in_pattern argument }
1361     | Concatenation { head; tail } ->
1362         Concatenation
1363         {
1364             head = rename_elements_in_pattern head;
1365             tail = rename_elements_in_pattern tail;
1366         }
1367     | Tuple l -> Tuple (List.map rename_elements_in_pattern l)
1368     | Or l -> Or (List.map rename_elements_in_pattern l)
1369     | As { inner; name } ->
1370         As
1371         {
1372             inner = rename_elements_in_pattern inner;
1373             name = (if List.mem name cloture_rect then new_name name else name);
1374         }
1375 in
1376 match e with
1377 | Variable v ->
1378     if List.mem v cloture_rect then Variable (new_name v) else Variable v
1379 | Constant _ -> e
1380 | Parenthesised { inner; style } ->
1381     Parenthesised
1382     { style; inner = rename_elements inner cloture_rect new_name }
1383 | TypeCoercion { inner; typ } ->
1384     TypeCoercion { inner = rename_elements inner cloture_rect new_name; typ }
1385 | ListLiteral l ->
1386     ListLiteral
1387     (List.map (fun l -> rename_elements l cloture_rect new_name) l)
1388 | ArrayLiteral l ->
1389     ArrayLiteral
1390     (List.map (fun l -> rename_elements l cloture_rect new_name) l)
1391 | RecordLiteral r ->
1392     RecordLiteral

```

5 – Annexes • 5.6 Code

```

1393         (List.map
1394           (fun (n, e) -> (n, rename_elements e cloture_rect new_name))
1395           r)
1396 | WhileLoop { condition; body } ->
1397   WhileLoop
1398   {
1399     condition = rename_elements condition cloture_rect new_name;
1400     body = rename_elements body cloture_rect new_name;
1401   }
1402 | ForLoop { direction = direction'; variable; start; finish; body } ->
1403   ForLoop
1404   {
1405     direction = direction';
1406     variable;
1407     start = rename_elements start cloture_rect new_name;
1408     finish = rename_elements finish cloture_rect new_name;
1409     body = rename_elements body cloture_rect new_name;
1410   }
1411 | Dereference e -> Dereference (rename_elements e cloture_rect new_name)
1412 | FieldAccess { receiver; target } ->
1413   FieldAccess
1414   { receiver = rename_elements receiver cloture_rect new_name; target }
1415 | ArrayAccess { receiver; target } ->
1416   ArrayAccess
1417   {
1418     receiver = rename_elements receiver cloture_rect new_name;
1419     target = rename_elements target cloture_rect new_name;
1420   }
1421 | FunctionApplication { receiver; arguments } ->
1422   FunctionApplication
1423   {
1424     receiver = rename_elements receiver cloture_rect new_name;
1425     arguments =
1426       List.map

```

```

1427         (fun arg -> rename_elements arg cloture_rect new_name)
1428         arguments;
1429     }
1430 | PrefixOperation { receiver; operation } ->
1431     PrefixOperation
1432     { operation; receiver = rename_elements receiver cloture_rect new_name }
1433 | InfixOperation { lhs; rhs; operation } ->
1434     InfixOperation
1435     {
1436         lhs = rename_elements lhs cloture_rect new_name;
1437         rhs = rename_elements rhs cloture_rect new_name;
1438         operation;
1439     }
1440 | Negation e -> Negation (rename_elements e cloture_rect new_name)
1441 | Tuple t ->
1442     Tuple (List.map (fun l -> rename_elements l cloture_rect new_name) t)
1443 | FieldAssignment { receiver; target; value } ->
1444     FieldAssignment
1445     {
1446         receiver = rename_elements receiver cloture_rect new_name;
1447         target;
1448         value = rename_elements value cloture_rect new_name;
1449     }
1450 | ArrayAssignment { receiver; target; value } ->
1451     ArrayAssignment
1452     {
1453         receiver = rename_elements receiver cloture_rect new_name;
1454         target = rename_elements target cloture_rect new_name;
1455         value = rename_elements value cloture_rect new_name;
1456     }
1457 | ReferenceAssignment { receiver; value } ->
1458     ReferenceAssignment
1459     {
1460         receiver = rename_elements receiver cloture_rect new_name;

```

```

1461         value = rename_elements value cloture_rect new_name;
1462     }
1463 | If { condition; body; else_body } ->
1464   If
1465   {
1466     condition = rename_elements condition cloture_rect new_name;
1467     body = rename_elements body cloture_rect new_name;
1468     else_body =
1469       Option.map
1470         (fun b -> rename_elements b cloture_rect new_name)
1471         else_body;
1472   }
1473 | Sequence s ->
1474   Sequence (List.map (fun e -> rename_elements e cloture_rect new_name) s)
1475 | Match { value; cases } ->
1476   Match
1477   {
1478     value = rename_elements value cloture_rect new_name;
1479     cases =
1480       List.map
1481         (fun (patterns, e) ->
1482           ( List.map rename_elements_in_pattern patterns,
1483             rename_elements e cloture_rect new_name ))
1484         cases;
1485   }
1486 | Try { value; cases } ->
1487   Try
1488   {
1489     value = rename_elements value cloture_rect new_name;
1490     cases =
1491       List.map
1492         (fun (patterns, e) ->
1493           ( List.map rename_elements_in_pattern patterns,
1494             rename_elements e cloture_rect new_name ))

```

```

1495         cases;
1496     }
1497 | FunctionLiteral { style; cases; return_type_for_delinearize } ->
1498     FunctionLiteral
1499     {
1500         style;
1501         cases =
1502             List.map
1503             (fun (patterns, e) ->
1504                 ( List.map rename_elements_in_pattern patterns,
1505                   rename_elements e cloture_rect new_name ))
1506             cases;
1507         return_type_for_delinearize;
1508     }
1509 | LetBinding { bindings; is_rec; inner } ->
1510     LetBinding
1511     {
1512         bindings =
1513             List.map
1514             (fun (binding : linear_binding) ->
1515                 match binding with
1516                 | Variable { lhs; value } ->
1517                     (Variable
1518                     {
1519                         lhs = rename_elements_in_pattern lhs;
1520                         value = rename_elements value cloture_rect new_name;
1521                     }
1522                     : linear_binding)
1523                 | Function { name; parameters; body; return_type } ->
1524                     Function
1525                     {
1526                         name =
1527                             (if List.mem name cloture_rect then new_name name
1528                              else name);

```



```

1529         parameters =
1530             List.map rename_elements_in_pattern parameters;
1531         body = rename_elements body cloture_rect new_name;
1532         return_type;
1533     })
1534     bindings;
1535     is_rec;
1536     inner = rename_elements inner cloture_rect new_name;
1537 }
1538 | StringAccess { receiver; target } ->
1539     StringAccess
1540     {
1541         receiver = rename_elements receiver cloture_rect new_name;
1542         target = rename_elements target cloture_rect new_name;
1543     }
1544 | StringAssignment { receiver; target; value } ->
1545     StringAssignment
1546     {
1547         receiver = rename_elements receiver cloture_rect new_name;
1548         target = rename_elements target cloture_rect new_name;
1549         value = rename_elements value cloture_rect new_name;
1550     }
1551
1552 (** rename_elements l c updates the definition and any references to variables
1553 in cloture_rect to their new names as indicated by new_name *)
1554 and rename_elements (l : linear_form) (cloture_rect : variable list)
1555 (new_name : variable -> variable) =
1556     1
1557     |> List.map (fun (name, elt) ->
1558         let new_elt = rename_elements_in elt cloture_rect new_name in
1559         if List.mem name cloture_rect then (new_name name, new_elt)
1560         else (name, new_elt))
1561
1562 type recursive_call_info = {

```

5 – Annexes • 5.6 Code

```

1563   rectifying_set : variable list;
1564   recursively_defined_functions : variable list;
1565   all_recursive_functions_are_toplevel : bool;
1566 }
1567
1568 (** clature_rectifiable fns computes a set that rectifies fns.
1569
1570   Returns None if such a set could not be computed. *)
1571 let clature_rectifiable (fns : (variable * linear_element) list) :
1572   recursive_call_info option =
1573   let rec get_argument_count (fn : variable) : int option =
1574     match find_definition fns fn with
1575     | Some def -> (
1576       match def with
1577       | FunctionLiteral { cases } -> (
1578         match cases with
1579         | (arguments, _) :: _ -> Some (List.length arguments)
1580         | _ -> None)
1581     | Variable v -> get_argument_count v
1582     | _ -> None)
1583   | None -> None
1584 in
1585
1586 let rec get_nth_argument_names (fn : variable) (n : int) :
1587   variable list option =
1588   match find_definition fns fn with
1589   | Some def -> (
1590     match def with
1591     | FunctionLiteral { cases } ->
1592       let nth_names =
1593         List.map
1594           (fun (patterns, _) ->
1595             let pattern = List.nth_opt patterns n in
1596             match pattern with Some p -> get_name p | None -> None)

```

```

1597         cases
1598     in
1599     if List.for_all (fun name -> name <> None) nth_names then
1600         Some (nth_names |> List.map Option.get |> List.sort_uniq compare)
1601     else None
1602 | Variable v -> get_nth_argument_names v n
1603 | FunctionApplication { receiver; arguments } ->
1604     let receiver_name = last_var receiver in
1605     get_nth_argument_names receiver_name (n + List.length arguments)
1606 | _ -> None
1607 | None -> None
1608 in
1609
1610 let cloture =
1611     ref
1612     (fns
1613     |> List.filter_map (fun (name, definition) ->
1614         let contains_other_reference =
1615             fns |> List.map fst
1616             |> List.exists (fun other_name ->
1617                 element_contains_reference other_name definition)
1618         in
1619         if contains_other_reference then Some name else None))
1620 in
1621 let a_traiter = ref !cloture in
1622
1623 let all_recursive_functions_are_toplevel = ref true in
1624 let recursively_defined_functions = ref [] in
1625
1626 let add_fn (fn : variable) : unit =
1627     if not (List.mem fn !cloture) then (
1628         cloture := fn :: !cloture;
1629         a_traiter := fn :: !a_traiter)
1630 in

```

```

1631
1632 let exception Exit of string in
1633 try
1634   while !a_traiter <> [] do
1635     let current = List.hd !a_traiter in
1636     a_traiter := List.tl !a_traiter;
1637
1638     let current_argument_count =
1639       match get_argument_count current with
1640       | Some n -> n
1641       | None -> raise (Exit ("Unable to get argument count for " ^ current))
1642   in
1643
1644   let rec propagate_from_element (e_name : variable) (e : linear_element)
1645     (enclosing_function : variable) (can_outer_leak : bool) : unit =
1646     match e with
1647     | Variable _ | Constant _ -> ()
1648     | Parenthesised { inner } | TypeCoercion { inner } ->
1649       propagate_from inner enclosing_function can_outer_leak
1650     | Dereference inner -> propagate_from inner enclosing_function false
1651     | ListLiteral l | ArrayLiteral l | Tuple l | Sequence l ->
1652       l
1653       |> List.iter (fun elt ->
1654         propagate_from elt enclosing_function false)
1655     | RecordLiteral r ->
1656       r
1657       |> List.iter (fun (_, elt) ->
1658         propagate_from elt enclosing_function false)
1659     | WhileLoop _ | ForLoop _ -> failwith "Found linearized loop"
1660     | FieldAccess { receiver } ->
1661       propagate_from receiver enclosing_function false
1662     | ArrayAccess { receiver; target } | StringAccess { receiver; target }
1663     ->
1664       propagate_from receiver enclosing_function false;

```

```

1665     propagate_from target enclosing_function false
1666 | FunctionApplication { receiver; arguments } ->
1667     propagate_from receiver enclosing_function true;
1668     arguments
1669     |> List.iter (fun arg -> propagate_from arg enclosing_function true)
1670 | PrefixOperation { receiver } ->
1671     propagate_from receiver enclosing_function false
1672 | InfixOperation { lhs; rhs } ->
1673     propagate_from lhs enclosing_function false;
1674     propagate_from rhs enclosing_function false
1675 | Negation receiver -> propagate_from receiver enclosing_function false
1676 | FieldAssignment { receiver; value } ->
1677     propagate_from receiver enclosing_function false;
1678     propagate_from value enclosing_function false
1679 | ArrayAssignment { receiver; target; value }
1680 | StringAssignment { receiver; target; value } ->
1681     propagate_from receiver enclosing_function false;
1682     propagate_from target enclosing_function false;
1683     propagate_from value enclosing_function false
1684 | ReferenceAssignment { receiver; value } ->
1685     propagate_from receiver enclosing_function false;
1686     propagate_from value enclosing_function false
1687 | If { condition; body; else_body } -> (
1688     propagate_from condition enclosing_function false;
1689     propagate_from body enclosing_function can_outer_leak;
1690     match else_body with
1691     | None -> ()
1692     | Some b -> propagate_from b enclosing_function can_outer_leak)
1693 | Match { value; cases } ->
1694     propagate_from value enclosing_function false;
1695     cases
1696     |> List.iter (fun (_, body) ->
1697         propagate_from body enclosing_function can_outer_leak)
1698 | Try _ -> failwith "Found linearized try"

```

```

1699 | FunctionLiteral { cases } ->
1700 |   List.iter (fun (_, body) -> propagate_from body e_name false) cases
1701 | LetBinding { bindings; inner } ->
1702 |   propagate_from inner enclosing_function can_outer_leak;
1703 |   bindings
1704 |   > List.iter (fun (binding : linear_binding) ->
1705 |       match binding with
1706 |       | Variable { lhs; value } ->
1707 |         (if last_var value = current then
1708 |           match get_name lhs with
1709 |           | None ->
1710 |             raise
1711 |             (Exit
1712 |              "Unable to get unique name for variable LHS")
1713 |             | Some var -> add_fn var);
1714 |         (* We can leak from value since this binding will then be in the
1715 |            ↪ rectifying set. *)
1716 |         propagate_from value enclosing_function true
1717 |         | Function { name; body } -> propagate_from body name false)
1718 |   and propagate_from (l : linear_form) (enclosing_function : variable)
1719 |   (can_leak : bool) : unit =
1720 |   if last_var l = current && not can_leak then
1721 |     raise (Exit ("Reference to " ^ current ^ " was leaked"));
1722 |   1
1723 |   > List.iter (fun (name, element) ->
1724 |       match element with
1725 |       | Variable n -> if n = current then add_fn name
1726 |       | FunctionApplication { receiver; arguments } ->
1727 |         let receiver = last_var receiver in
1728 |         if receiver = current then
1729 |           if List.length arguments = current_argument_count then (
1730 |             add_fn enclosing_function;
1731 |             all_recursive_functions_are_toplevel :=

```

```

1732         && List.assoc_opt enclosing_function fns <> None;
1733     if
1734     not
1735         (List.mem enclosing_function
1736          !recursively_defined_functions)
1737     then
1738         recursively_defined_functions :=
1739             enclosing_function :: !recursively_defined_functions)
1740     else raise (Exit ("Call to " ^ current ^ " was partial"))
1741 else
1742     arguments
1743     |> List.iteri (fun i argument ->
1744                 if last_var argument = current then
1745                     let ith_names =
1746                         get_nth_argument_names receiver i
1747                     in
1748                         match ith_names with
1749                         | Some names -> List.iter add_fn names
1750                         | None ->
1751                             raise
1752                                 (Exit
1753                                  ("Unable to get argument names for "
1754                                   ^ receiver)))
1755     | _ -> ());
1756
1757     1
1758     |> List.iter (fun (name, element) ->
1759                 propagate_from_element name element enclosing_function can_leak)
1760 in
1761
1762     fns
1763     |> List.iter (fun (name, def) ->
1764                 propagate_from_element name def "UNKNOWN_GLOBAL_ENCLOSURE" false)
1765 done;

```

```

1766     Some
1767     {
1768         rectifying_set = !cloture;
1769         all_recursive_functions_are_toplevel =
1770             !all_recursive_functions_are_toplevel;
1771         recursively_defined_functions = !recursively_defined_functions;
1772     }
1773 with Exit s -> None
1774
1775 (** find_continuations collects the definitions of all continuations defined in
1776 fns.
1777
1778 The continuations are assumed to have been generated by `rectify`; that is,
1779 they are assumed to be named "new_cont". *)
1780 let find_continuations (fns : (variable * linear_element) list)
1781     (cloture_rect : variable list) : linear_function_literal list option =
1782     let rec find_continuations_in_element (e : linear_element) :
1783         linear_function_literal list =
1784         match e with
1785         | Variable _ | Constant _ -> []
1786         | Parenthesised { inner } | TypeCoercion { inner } | Dereference inner ->
1787             find_continuations_in inner
1788         | ListLiteral l | ArrayLiteral l | Tuple l | Sequence l ->
1789             l |> List.map (fun elt -> find_continuations_in elt) |> List.concat
1790         | RecordLiteral r ->
1791             r |> List.map (fun (_, elt) -> find_continuations_in elt) |> List.concat
1792         | WhileLoop _ | ForLoop _ -> failwith "Found linearized loop"
1793         | FieldAccess { receiver } -> find_continuations_in receiver
1794         | ArrayAccess { receiver; target } | StringAccess { receiver; target } ->
1795             find_continuations_in receiver @ find_continuations_in target
1796         | FunctionApplication { receiver; arguments } ->
1797             find_continuations_in receiver
1798             @ (arguments
1799                 |> List.map (fun arg -> find_continuations_in arg)

```



```

1800         |> List.concat)
1801 | PrefixOperation { receiver } -> find_continuations_in receiver
1802 | InfixOperation { lhs; rhs } ->
1803     find_continuations_in lhs @ find_continuations_in rhs
1804 | Negation receiver -> find_continuations_in receiver
1805 | FieldAssignment { receiver; value } ->
1806     find_continuations_in receiver @ find_continuations_in value
1807 | ArrayAssignment { receiver; target; value }
1808 | StringAssignment { receiver; target; value } ->
1809     find_continuations_in receiver
1810     @ find_continuations_in target
1811     @ find_continuations_in value
1812 | ReferenceAssignment { receiver; value } ->
1813     find_continuations_in receiver @ find_continuations_in value
1814 | If { condition; body; else_body } -> (
1815     find_continuations_in condition
1816     @ find_continuations_in body
1817     @ match else_body with None -> [] | Some b -> find_continuations_in b)
1818 | Match { value; cases } ->
1819     find_continuations_in value
1820     @ (cases
1821         |> List.map (fun (_, body) -> find_continuations_in body)
1822         |> List.concat)
1823 | Try _ -> failwith "Found linearized try"
1824 | FunctionLiteral { cases } ->
1825     List.map (fun (_, body) -> find_continuations_in body) cases
1826     |> List.concat
1827 | LetBinding { bindings; inner } ->
1828     find_continuations_in inner
1829     @ (bindings
1830         |> List.map (fun (binding : linear_binding) ->
1831             match binding with
1832             | Variable { value } -> find_continuations_in value
1833             | Function { name; body } -> find_continuations_in body)

```

```

1834         |> List.concat)
1835 and find_continuations_in (l : linear_form) : linear_function_literal list =
1836   match l with
1837   | [] -> []
1838   | (name, x) :: q ->
1839     let x_def =
1840       if name = "new_cont" then
1841         match x with
1842         | FunctionLiteral f -> [ f ]
1843         | _ -> failwith "Not a continuation, but named new_cont"
1844       else []
1845     in
1846       x_def @ find_continuations_in_element x @ find_continuations_in q
1847 in
1848 Some
1849 (List.fold_left
1850  (fun prev (name, body) -> prev @ find_continuations_in_element body)
1851  [] fns
1852  |> List.sort_uniq compare)
1853
1854
1855 (** find_initials fns returns the set of all values passed as base cases to the
1856 continuations of fns.
1857
1858 Returns None if a base case was not a constant or if all base cases could
1859 not be computed. *)
1860 let find_initials (fns : (variable * linear_element) list) :
1861   (variable list * constant list) option =
1862   let exception Exit in
1863   try
1864     let rec find_initials_in_element (e : linear_element) :
1865       (variable * constant) list =
1866       match e with
1867       | Variable _ | Constant _ -> []

```

```

1868 | Parenthesised { inner } | TypeCoercion { inner } | Dereference inner ->
1869 | find_initials_in inner
1870 | ListLiteral l | ArrayLiteral l | Tuple l | Sequence l ->
1871 | l |> List.map (fun elt -> find_initials_in elt) |> List.concat
1872 | RecordLiteral r ->
1873 | r |> List.map (fun (_, elt) -> find_initials_in elt) |> List.concat
1874 | WhileLoop _ | ForLoop _ -> failwith "Found linearized loop"
1875 | FieldAccess { receiver } -> find_initials_in receiver
1876 | ArrayAccess { receiver; target } | StringAccess { receiver; target } ->
1877 | find_initials_in receiver @ find_initials_in target
1878 | FunctionApplication { receiver; arguments } ->
1879 | find_initials_in receiver
1880 | @ (arguments
1881 | |> List.map (fun arg -> find_initials_in arg)
1882 | |> List.concat)
1883 | PrefixOperation { receiver } -> find_initials_in receiver
1884 | InfixOperation { lhs; rhs } ->
1885 | find_initials_in lhs @ find_initials_in rhs
1886 | Negation receiver -> find_initials_in receiver
1887 | FieldAssignment { receiver; value } ->
1888 | find_initials_in receiver @ find_initials_in value
1889 | ArrayAssignment { receiver; target; value }
1890 | StringAssignment { receiver; target; value } ->
1891 | find_initials_in receiver @ find_initials_in target
1892 | @ find_initials_in value
1893 | ReferenceAssignment { receiver; value } ->
1894 | find_initials_in receiver @ find_initials_in value
1895 | If { condition; body; else_body } -> (
1896 | find_initials_in condition @ find_initials_in body
1897 | @ match else_body with None -> [] | Some b -> find_initials_in b)
1898 | Match { value; cases } ->
1899 | find_initials_in value
1900 | @ (cases
1901 | |> List.map (fun (_, body) -> find_initials_in body)

```

```

1902         |> List.concat)
1903   | Try _ -> failwith "Found linearized try"
1904   | FunctionLiteral { cases } ->
1905     List.map (fun (_, body) -> find_initials_in body) cases |> List.concat
1906   | LetBinding { bindings; inner } ->
1907     find_initials_in inner
1908   @ (bindings
1909     |> List.map (fun (binding : linear_binding) ->
1910       match binding with
1911       | Variable { value } -> find_initials_in value
1912       | Function { name; body } -> find_initials_in body)
1913     |> List.concat)
1914 and find_initials_in (l : linear_form) : (variable * constant) list =
1915   match l with
1916   | [] -> []
1917   | [
1918     (_,
1919       FunctionApplication
1920       {
1921         receiver = [ (_, Variable "cont") ];
1922         arguments = [ [ (_, Variable arg) ] ];
1923       }
1924     ) ->
1925     let rec find_constant_definition (arg : string) :
1926       (variable * constant) list =
1927       match find_definition fns arg with
1928       | Some (Variable v) ->
1929         let res = find_constant_definition v in
1930         (arg, snd (List.hd res)) :: res
1931       | Some (Constant c) -> [ (arg, c) ]
1932       | _ -> raise Exit
1933     in
1934     find_constant_definition arg
1935   | (name, x) :: q ->

```

5 – Annexes • 5.6 Code

```

1936         if name <> "new_cont" then
1937             find_initials_in_element x @ find_initials_in q
1938         else find_initials_in q
1939     in
1940     let pairs =
1941         List.fold_left
1942             (fun prev (name, body) -> prev @ find_initials_in_element body)
1943             [] fns
1944     in
1945     let names = List.map fst pairs in
1946     let constants = List.map snd pairs in
1947
1948     Some (names, constants |> List.sort_uniq compare)
1949 with Exit -> None
1950
1951 (** Given a computation, if it is semantically the composition of a previous
1952 continuation with some new function, return that new function. Else return
1953 None.
1954
1955 For example, the function `fun x -> if y then cont z else cont w` is
1956 semantically equivalent to `cont @@ (func x -> if y then z else w)`. *)
1957 let extract_continuation_composition (cont : linear_function_literal) :
1958     linear_function_literal option =
1959     let exception NotSimple in
1960     let rec extract_without_continuation (l : linear_form) : linear_form =
1961         match l with
1962         | [] -> raise NotSimple
1963         | [
1964             (
1965                 FunctionApplication
1966                 { receiver = [ (_, Variable "cont") ]; arguments = [ single_arg ] }
1967             ) ->
1968             single_arg
1969         (* We only get terminally recursive elements here *)

```

```

1970 | e :: q -> e :: extract_without_continuation q
1971 in
1972 try
1973   let res =
1974     {
1975       style = cont.style;
1976       cases =
1977         cont.cases
1978         |> List.map (fun (parameters, body) ->
1979           (parameters, extract_without_continuation body));
1980       return_type_for_delinearize =
1981         (match List.hd cont.cases with
1982         | [ TypeCoercion { typ } ], _ -> Some typ
1983         | _ -> None);
1984     }
1985   in
1986   Some res
1987 with NotSimple -> None
1988
1989 (** Returns true only if all the functions of continuations commute.
1990
1991 For now the check is that:
1992 - The value returned is an operator involving the argument that commutes,
1993 for example: arg + constant, not arg, -arg
1994 - The argument is not used in any other expressions in the continuation
1995 - None of the other expressions in the continuation have side effects. For
1996 this we just disallow function calls altogether. *)
1997 let commutes (continuations : linear_function_literal list) : bool =
1998 let check_continuation (continuation : linear_function_literal) : bool =
1999   let arguments, body = List.hd continuation.cases in
2000   let argument = Option.get (get_name (List.hd arguments)) in
2001
2002   (* If v is defined to be equal to argument, return a list of all the
2003     variables in the definition chain between v and argument, else return

```

5 – Annexes • 5.6 Code

```

2004     None. *)
2005 let rec ensure_is_argument (v : variable) : variable list option =
2006   if v = argument then Some [ v ]
2007   else
2008     match find_definition body v with
2009     | Some (Variable v2) ->
2010       ensure_is_argument v2 |> Option.map (fun vx -> v :: vx)
2011     | _ -> None
2012 in
2013
2014   (* If returned_var is defined in a valid way (see the doc comment for
2015      commutes), return a list of all the variables on the definition chain
2016      between the argument and the returned value, else return None. *)
2017 let rec check_return_from (returned_var : variable) : variable list option =
2018   if returned_var = argument then Some [ returned_var ]
2019   else
2020     let vx =
2021       match find_definition body returned_var with
2022       | None -> None
2023       | Some (Variable v) -> check_return_from v
2024       | Some (Negation l) -> ensure_is_argument (last_var l)
2025       | Some (PrefixOperation { receiver; operation = Minus | MinusDot }) ->
2026         ensure_is_argument (last_var receiver)
2027       | Some
2028         (InfixOperation
2029          { lhs; operation = Plus | PlusDot | Star | StarDot; rhs }) -> (
2030         match
2031           ( ensure_is_argument (last_var lhs),
2032             ensure_is_argument (last_var rhs) )
2033         with
2034         | Some v, None | None, Some v -> Some v
2035         | _ -> None)
2036     | _ -> None
2037 in

```

```

2038     match vx with Some vars -> Some (returned_var :: vars) | None -> None
2039 in
2040
2041 match check_return_from (last_var body) with
2042 | None -> false
2043 | Some vars_referencing_argument ->
2044     let rec check_remaining_element (e : linear_element) =
2045         match e with
2046         | Variable _ | Constant _ -> true
2047         | Parenthesised { inner } | TypeCoercion { inner } ->
2048             check_remaining inner
2049         | ListLiteral l | ArrayLiteral l -> List.for_all check_remaining l
2050         | RecordLiteral r ->
2051             List.for_all (fun (_, value) -> check_remaining value) r
2052         | WhileLoop _ | ForLoop _ -> failwith "Found linearized loop"
2053         | Dereference inner -> check_remaining inner
2054         | FieldAccess { receiver } -> check_remaining receiver
2055         | ArrayAccess { receiver; target } ->
2056             check_remaining receiver && check_remaining target
2057         (* No function applications are allowed *)
2058         | FunctionApplication _ -> false
2059         | PrefixOperation { receiver } -> check_remaining receiver
2060         | InfixOperation { lhs; rhs } ->
2061             check_remaining lhs && check_remaining rhs
2062         | Negation inner -> check_remaining inner
2063         | Tuple t -> List.for_all check_remaining t
2064         (* Side effects *)
2065         | FieldAssignment _ | ArrayAssignment _ | ReferenceAssignment _ ->
2066             false
2067         | If { condition; body; else_body } -> (
2068             check_remaining condition && check_remaining body
2069             &&
2070             match else_body with None -> true | Some b -> check_remaining b)
2071         | Sequence s -> List.for_all check_remaining s

```



```

2072 | Match { value; cases } ->
2073 |   check_remaining value
2074 |   && List.for_all (fun (_, body) -> check_remaining body) cases
2075 | Try _ -> failwith "Found linearized try"
2076 | FunctionLiteral _ -> true (* We can't call it anyway *)
2077 | LetBinding { bindings; inner } ->
2078 |   check_remaining inner
2079 |   && List.for_all
2080 |     (fun (binding : linear_binding) ->
2081 |       match binding with
2082 |       | Function _ -> true (* We can't call it anyway *)
2083 |       | Variable { value } -> check_remaining value)
2084 |     bindings
2085 | StringAccess { receiver; target } ->
2086 |   check_remaining receiver && check_remaining target
2087 |   (* Side effects *)
2088 | StringAssignment _ -> false
2089 | and check_remaining (l : linear_form) =
2090 |   List.for_all
2091 |     (fun (name, element) ->
2092 |       (match element with
2093 |       (* If the element references the argument then we must too *)
2094 |       | Variable v when List.mem v vars_referencing_argument ->
2095 |         List.mem name vars_referencing_argument
2096 |       | _ -> true)
2097 |       && check_remaining_element element)
2098 |     l
2099 |   in
2100 |   check_remaining body
2101 | in
2102 | List.for_all check_continuation continuations
2103 |
2104 | let rec replace_element_continuations_with_accumulator (e : linear_element)
2105 |   (cloture_rect : variable list) (vars_to_replace : variable list) =

```

```

2106 match e with
2107 | Variable _ -> e
2108 | Constant _ -> e
2109 | Parenthesised { inner; style } ->
2110   Parenthesised
2111     {
2112       style;
2113       inner =
2114         replace_continuations_with_accumulator inner cloture_rect
2115           vars_to_replace;
2116     }
2117 | TypeCoercion { inner; typ } ->
2118   TypeCoercion
2119     {
2120       inner =
2121         replace_continuations_with_accumulator inner cloture_rect
2122           vars_to_replace;
2123       typ;
2124     }
2125 | ListLiteral l ->
2126   ListLiteral
2127     (List.map
2128       (fun l ->
2129         replace_continuations_with_accumulator l cloture_rect
2130           vars_to_replace)
2131       l)
2132 | ArrayLiteral l ->
2133   ArrayLiteral
2134     (List.map
2135       (fun l ->
2136         replace_continuations_with_accumulator l cloture_rect
2137           vars_to_replace)
2138       l)
2139 | RecordLiteral r ->

```

```

2140     RecordLiteral
2141     (List.map
2142       (fun (n, e) ->
2143         ( n,
2144           replace_continuations_with_accumulator e cloture_rect
2145             vars_to_replace ))
2146       r)
2147 | WhileLoop { condition; body } ->
2148   WhileLoop
2149   {
2150     condition =
2151       replace_continuations_with_accumulator condition cloture_rect
2152         vars_to_replace;
2153     body =
2154       replace_continuations_with_accumulator body cloture_rect
2155         vars_to_replace;
2156   }
2157 | ForLoop { direction = direction'; variable; start; finish; body } ->
2158   ForLoop
2159   {
2160     direction = direction';
2161     variable;
2162     start =
2163       replace_continuations_with_accumulator start cloture_rect
2164         vars_to_replace;
2165     finish =
2166       replace_continuations_with_accumulator finish cloture_rect
2167         vars_to_replace;
2168     body =
2169       replace_continuations_with_accumulator body cloture_rect
2170         vars_to_replace;
2171   }
2172 | Dereference e ->
2173   Dereference

```

```

2174         (replace_continuations_with_accumulator e cloture_rect vars_to_replace)
2175 | FieldAccess { receiver; target } ->
2176   FieldAccess
2177   {
2178     receiver =
2179       replace_continuations_with_accumulator receiver cloture_rect
2180       vars_to_replace;
2181     target;
2182   }
2183 | ArrayAccess { receiver; target } ->
2184   ArrayAccess
2185   {
2186     receiver =
2187       replace_continuations_with_accumulator receiver cloture_rect
2188       vars_to_replace;
2189     target =
2190       replace_continuations_with_accumulator target cloture_rect
2191       vars_to_replace;
2192   }
2193 | FunctionApplication { receiver = [ (_, Variable "cont") ] } ->
2194   Variable "acc"
2195 | FunctionApplication { receiver; arguments } ->
2196   FunctionApplication
2197   {
2198     receiver =
2199       replace_continuations_with_accumulator receiver cloture_rect
2200       vars_to_replace;
2201     arguments =
2202       List.map
2203         (fun arg ->
2204           replace_continuations_with_accumulator arg cloture_rect
2205             vars_to_replace)
2206         arguments;
2207   }

```

```

2208 | PrefixOperation { receiver; operation } ->
2209 | PrefixOperation
2210 | {
2211 |     operation;
2212 |     receiver =
2213 |         replace_continuations_with_accumulator receiver cloture_rect
2214 |         vars_to_replace;
2215 | }
2216 | InfixOperation { lhs; rhs; operation } ->
2217 | InfixOperation
2218 | {
2219 |     lhs =
2220 |         replace_continuations_with_accumulator lhs cloture_rect
2221 |         vars_to_replace;
2222 |     rhs =
2223 |         replace_continuations_with_accumulator rhs cloture_rect
2224 |         vars_to_replace;
2225 |     operation;
2226 | }
2227 | Negation e ->
2228 | Negation
2229 | (replace_continuations_with_accumulator e cloture_rect vars_to_replace)
2230 | Tuple t ->
2231 | Tuple
2232 | (List.map
2233 |  (fun l ->
2234 |      replace_continuations_with_accumulator l cloture_rect
2235 |      vars_to_replace)
2236 |  t)
2237 | FieldAssignment { receiver; target; value } ->
2238 | FieldAssignment
2239 | {
2240 |     receiver =
2241 |         replace_continuations_with_accumulator receiver cloture_rect

```

```

2242         vars_to_replace;
2243     target;
2244     value =
2245         replace_continuations_with_accumulator value cloture_rect
2246         vars_to_replace;
2247 }
2248 | ArrayAssignment { receiver; target; value } ->
2249     ArrayAssignment
2250     {
2251         receiver =
2252             replace_continuations_with_accumulator receiver cloture_rect
2253             vars_to_replace;
2254         target =
2255             replace_continuations_with_accumulator target cloture_rect
2256             vars_to_replace;
2257         value =
2258             replace_continuations_with_accumulator value cloture_rect
2259             vars_to_replace;
2260     }
2261 | ReferenceAssignment { receiver; value } ->
2262     ReferenceAssignment
2263     {
2264         receiver =
2265             replace_continuations_with_accumulator receiver cloture_rect
2266             vars_to_replace;
2267         value =
2268             replace_continuations_with_accumulator value cloture_rect
2269             vars_to_replace;
2270     }
2271 | If { condition; body; else_body } ->
2272     If
2273     {
2274         condition =
2275             replace_continuations_with_accumulator condition cloture_rect

```

```

2276         vars_to_replace;
2277     body =
2278         replace_continuations_with_accumulator body cloture_rect
2279         vars_to_replace;
2280     else_body =
2281         Option.map
2282             (fun b ->
2283                 replace_continuations_with_accumulator b cloture_rect
2284                 vars_to_replace)
2285             else_body;
2286     }
2287 | Sequence s ->
2288     Sequence
2289     (List.map
2290         (fun e ->
2291             replace_continuations_with_accumulator e cloture_rect
2292             vars_to_replace)
2293         s)
2294 | Match { value; cases } ->
2295     Match
2296     {
2297         value =
2298             replace_continuations_with_accumulator value cloture_rect
2299             vars_to_replace;
2300         cases =
2301             List.map
2302                 (fun (patterns, e) ->
2303                     ( patterns,
2304                       replace_continuations_with_accumulator e cloture_rect
2305                       vars_to_replace ))
2306                 cases;
2307     }
2308 | Try { value; cases } ->
2309     Try

```

```

2310     {
2311       value =
2312         replace_continuations_with_accumulator value cloture_rect
2313         vars_to_replace;
2314       cases =
2315         List.map
2316           (fun (patterns, e) ->
2317             ( patterns,
2318               replace_continuations_with_accumulator e cloture_rect
2319                 vars_to_replace ))
2320         cases;
2321     }
2322 | FunctionLiteral { style; cases; return_type_for_delinearize } ->
2323   FunctionLiteral
2324   {
2325     style;
2326     cases =
2327       List.map
2328         (fun (patterns, e) ->
2329           ( patterns,
2330             replace_continuations_with_accumulator e cloture_rect
2331               vars_to_replace ))
2332         cases;
2333     return_type_for_delinearize;
2334   }
2335 | LetBinding { bindings; is_rec; inner } ->
2336   LetBinding
2337   {
2338     bindings =
2339       List.map
2340         (fun (binding : linear_binding) ->
2341           match binding with
2342           | Variable { lhs; value } ->
2343             (Variable

```



```

2344         {
2345             lhs;
2346             value =
2347                 replace_continuations_with_accumulator value
2348                 closure_rect vars_to_replace;
2349         }
2350         : linear_binding)
2351 | Function { name; parameters; body; return_type } ->
2352     Function
2353     {
2354         name;
2355         parameters;
2356         body =
2357             replace_continuations_with_accumulator body
2358             closure_rect vars_to_replace;
2359         return_type;
2360     })
2361     bindings;
2362     is_rec;
2363     inner =
2364         replace_continuations_with_accumulator inner closure_rect
2365         vars_to_replace;
2366     }
2367 | StringAccess { receiver; target } ->
2368     StringAccess
2369     {
2370         receiver =
2371             replace_continuations_with_accumulator receiver closure_rect
2372             vars_to_replace;
2373         target =
2374             replace_continuations_with_accumulator target closure_rect
2375             vars_to_replace;
2376     }
2377 | StringAssignment { receiver; target; value } ->

```

```

2378   StringAssignment
2379   {
2380       receiver =
2381           replace_continuations_with_accumulator receiver cloture_rect
2382           vars_to_replace;
2383       target =
2384           replace_continuations_with_accumulator target cloture_rect
2385           vars_to_replace;
2386       value =
2387           replace_continuations_with_accumulator value cloture_rect
2388           vars_to_replace;
2389   }
2390
2391   (** Given a linear form l that is terminally recursive for a rectifying set
2392   cloture_rect, replace CPS with the use of an accumulator. The continuations
2393   are assumed to commute.
2394
2395   Also removes the elements corresponding to the names in vars_to_remove. This
2396   allows the variables defining the initial values passed to the continuations
2397   to be removed. *)
2398   and replace_continuations_with_accumulator (l : linear_form)
2399       (cloture_rect : variable list) (vars_to_remove : variable list) =
2400       let rec get_accumulator_type_from_continuation (l : pattern list) :
2401           type_expression option =
2402           match l with
2403           | [] -> None
2404           | [ x ] -> (
2405               match x with
2406               | TypeCoercion { typ = Function { argument } } -> Some argument
2407               | _ -> None)
2408           | _ :: q -> get_accumulator_type_from_continuation q
2409       in
2410
2411       let rec replace_last_parameter (l : pattern list) =

```

5 – Annexes • 5.6 Code

```

2412     match l with
2413     | [] -> []
2414     | [ x ] -> (
2415         match x with
2416         | TypeCoercion { typ = Function { argument } } ->
2417           [ (TypeCoercion { inner = Ident "acc"; typ = argument } : pattern) ]
2418         | _ -> [ Ident "acc" ] )
2419     | x :: q -> x :: replace_last_parameter q
2420 in
2421
2422 1
2423 |> List.filter_map (fun (name, elt) ->
2424     if List.mem name vars_to_remove then None
2425     else
2426       Some
2427         (if elt = Variable "cont" then (name, Variable "acc")
2428          else if name = "new_cont" then
2429            match elt with
2430            | FunctionLiteral f ->
2431              ( "new_acc",
2432                (* extract_continuation works since we don't call this function unless
2433                  ↪ it worked for all continuations *)
2434                FunctionLiteral
2435                  (extract_continuation_composition f |> Option.get) )
2436            | _ -> failwith "Not a new continuation"
2437          else if name = "cont" then
2438            ( "acc",
2439              FunctionApplication
2440                {
2441                  receiver = [ ("new_acc_ref", Variable "new_acc") ];
2442                  arguments = [ [ ("acc_ref", Variable "acc") ] ];
2443                } )
2444          else
2445            let new_elt =

```

```

2445         replace_element_continuations_with_accumulator elt
2446         cloture_rect vars_to_remove
2447     in
2448     if List.mem name cloture_rect then
2449         let new_new_elt =
2450             match new_elt with
2451             | Variable v -> new_elt
2452             | FunctionLiteral { style; cases } ->
2453                 FunctionLiteral
2454                     {
2455                         style;
2456                         cases =
2457                             List.map
2458                                 (fun (arguments, body) ->
2459                                     (replace_last_parameter arguments, body))
2460                                 cases;
2461                         return_type_for_delinearize =
2462                             get_accumulator_type_from_continuation
2463                                 (fst (List.hd cases));
2464                     }
2465         | _ -> failwith ("Unable to update definition of " ^ name)
2466     in
2467     (name, new_new_elt)
2468 else (name, new_elt)))

```

regex_grammar.ml

```
1  open Regex
2  open Lexer
3  open Parser
4  open Utils
5
6  type regex_token_type =
7      | Open_paren
8      | Close_paren
9      | Open_bracket
10     | Close_bracket
11     | Or
12     | Plus
13     | Star
14     | Question
15     | Escape
16     | Character
17     | Eof
18     | Unknown
19
20 let add_character (c : uchar) (r : uchar regex) : uchar regex =
21     Regex.Union (r, Regex.Symbol c)
22
23 let add_character_c (c : char) (r : uchar regex) : uchar regex =
24     add_character (u c) r
25
26 let rec add_character_range (start : uchar) (last : uchar) (r : uchar regex) =
27     if start > last then failwith "Invalid range";
28     if start = last then add_character start r
29     else
30         add_character_range
31             (Uchar.of_int (Uchar.to_int start + 1))
32             last (add_character start r)
33
```

5 – Annexes • 5.6 Code

```

34 let add_character_range_c (start : char) (last : char) (r : uchar regex) =
35     add_character_range (u start) (u last) r
36
37 let regex_token_rules =
38     [
39         (Regex.Symbol (u '('), Open_paren);
40         (Regex.Symbol (u ')'), Close_paren);
41         (Regex.Symbol (u '['), Open_bracket);
42         (Regex.Symbol (u ']'), Close_bracket);
43         (Regex.Symbol (u '|'), Or);
44         (Regex.Symbol (u '*'), Star);
45         (Regex.Symbol (u '+'), Plus);
46         (Regex.Symbol (u '?'), Question);
47         ( Regex.Concatenation
48             ( Regex.Symbol (u '\\'),
49               Regex.Union
50                 ( Regex.Empty |> add_character_c '(' |> add_character_c ')'
51                   |> add_character_c '|' |> add_character_c '*'
52                   |> add_character_c '\\\\' |> add_character_c '\n'
53                   |> add_character_c 't' |> add_character_c 'r'
54                   |> add_character_c '[' |> add_character_c ']'
55                   |> add_character_c '+' |> add_character_c '?',
56                 Regex.Concatenation
57                   ( Regex.Symbol (u 'u'),
58                     Regex.Concatenation
59                       ( Regex.Concatenation
60                           ( Regex.Empty
61                               |> add_character_range_c '0' '9'
62                               |> add_character_range_c 'a' 'f',
63                             Regex.Empty
64                               |> add_character_range_c '0' '9'
65                               |> add_character_range_c 'a' 'f' ),
66                           Regex.Concatenation
67                             ( Regex.Empty

```

5 – Annexes • 5.6 Code

```

68         |> add_character_range_c '0' '9'
69         |> add_character_range_c 'a' 'f',
70         Regex.Empty
71         |> add_character_range_c '0' '9'
72         |> add_character_range_c 'a' 'f' ) ) ) ) ),
73     Escape );
74 ( Regex.Empty
75   |> add_character_range_c ' ' '\ '
76   (* (, ), * *)
77   |> add_character_range_c '+' '{'
78   (* / *)
79   |> add_character_range_c '}' '~',
80   Character );
81 ]
82
83 type regex_rule =
84 | Regex_union
85 | Regex_concatenation
86 | Regex_primitive
87 | Character_class
88 | Character_class_entry
89
90 let regex_grammar =
91   grammar_of_rule_list
92   [
93     (Regex_union, [ NonTerminal Regex_concatenation ]);
94     ( Regex_union,
95       [
96         NonTerminal Regex_concatenation; Terminal Or; NonTerminal Regex_union;
97       ] );
98     (Regex_concatenation, [ NonTerminal Regex_primitive ]);
99     ( Regex_concatenation,
100       [ NonTerminal Regex_primitive; NonTerminal Regex_concatenation ] );
101     (Regex_primitive, [ NonTerminal Regex_primitive; Terminal Star ]);

```

5 – Annexes • 5.6 Code

```

102     (Regex_primitive, [ NonTerminal Regex_primitive; Terminal Plus ]);
103     (Regex_primitive, [ NonTerminal Regex_primitive; Terminal Question ]);
104     (Regex_primitive, [ Terminal Character ]);
105     (Regex_primitive, [ Terminal Escape ]);
106     ( Regex_primitive,
107       [ Terminal Open_paren; NonTerminal Regex_union; Terminal Close_paren ]
108     );
109     ( Regex_primitive,
110       [
111         Terminal Open_bracket;
112         NonTerminal Character_class;
113         Terminal Close_bracket;
114       ] );
115     (Character_class, [ NonTerminal Character_class_entry ]);
116     ( Character_class,
117       [ NonTerminal Character_class_entry; NonTerminal Character_class ] );
118     (Character_class_entry, [ Terminal Character ]);
119     (Character_class_entry, [ Terminal Escape ]);
120   ]
121
122   let rec regex_of_regex_syntax_tree
123     (node : (regex_token_type, regex_rule) syntax_tree) : uchar regex =
124   let expand_escape (value : string) : uchar =
125     let escaped_char = value.[1] in
126     match escaped_char with
127     | 'n' -> u '\n'
128     | 't' -> u '\t'
129     | 'r' -> u '\r'
130     | 'u' ->
131       let code = int_of_string ("0x" ^ String.sub value 2 4) in
132       Uchar.of_int code
133     | _ -> u escaped_char
134   in
135

```


5 – Annexes • 5.6 Code

```

136 let flatten_character_class
137   (node : (regex_token_type, regex_rule) syntax_tree) : uchar list =
138   let extract_character (node : (regex_token_type, regex_rule) syntax_tree) :
139     uchar =
140     match node with
141     | Node (Character_class_entry, [ Leaf { token_type = Character; value } ])
142       ->
143         u value.[0]
144     | Node (Character_class_entry, [ Leaf { token_type = Escape; value } ]) ->
145       expand_escape value
146     | _ -> failwith "Not a member of a character class"
147   in
148
149   let rec flatten_character_class_into_rev
150     (node : (regex_token_type, regex_rule) syntax_tree) (res : uchar list) :
151     uchar list =
152     match node with
153     | Node (Character_class, [ entry; q ]) ->
154       flatten_character_class_into_rev q (extract_character entry :: res)
155     | Node (Character_class, [ entry ]) -> extract_character entry :: res
156     | _ -> failwith "Not a character class"
157   in
158   List.rev (flatten_character_class_into_rev node [])
159 in
160
161 let build_character_class (c : uchar list) : uchar regex =
162   let rec build_character_class_into (c : uchar list) (r : uchar regex) =
163     match c with
164     | [] -> r
165     | a :: dash :: b :: q when dash = u '-' ->
166       build_character_class_into q (add_character_range a b r)
167     | a :: q -> build_character_class_into q (add_character a r)
168   in
169   build_character_class_into c Regex.Empty

```

```

170   in
171
172   match node with
173   | Node (Regex_union, [ node ]) -> regex_of_regex_syntax_tree node
174   | Node (Regex_union, [ node1; Leaf { token_type = Or }; node2 ]) ->
175       Regex.Union
176         (regex_of_regex_syntax_tree node1, regex_of_regex_syntax_tree node2)
177   | Node (Regex_concatenation, [ node ]) -> regex_of_regex_syntax_tree node
178   | Node (Regex_concatenation, [ node1; node2 ]) ->
179       Regex.Concatenation
180         (regex_of_regex_syntax_tree node1, regex_of_regex_syntax_tree node2)
181   | Node (Regex_primitive, [ node; Leaf { token_type = Star } ]) ->
182       Regex.Star (regex_of_regex_syntax_tree node)
183   | Node (Regex_primitive, [ node; Leaf { token_type = Plus } ]) ->
184       let inner = regex_of_regex_syntax_tree node in
185       Regex.Concatenation (inner, Regex.Star inner)
186   | Node (Regex_primitive, [ node; Leaf { token_type = Question } ]) ->
187       Regex.Union (Regex.Epsilon, regex_of_regex_syntax_tree node)
188   | Node (Regex_primitive, [ Leaf { token_type = Character; value } ]) ->
189       Regex.Symbol (u value.[0])
190   | Node (Regex_primitive, [ Leaf { token_type = Escape; value } ]) ->
191       Regex.Symbol (expand_escape value)
192   | Node
193       ( Regex_primitive,
194         [
195           Leaf { token_type = Open_paren };
196           node;
197           Leaf { token_type = Close_paren };
198         ] ) ->
199       regex_of_regex_syntax_tree node
200   | Node
201       ( Regex_primitive,
202         [
203           Leaf { token_type = Open_bracket };

```

```
204         node;  
205         Leaf { token_type = Close_bracket };  
206     ] ) ->  
207     let character_class = flatten_character_class node in  
208     build_character_class character_class  
209     | _ -> failwith "Invalid rule"  
210  
211 let parse_regex (src : string) : uchar regex =  
212     let tokens = tokenize_regex_token_rules Eof Unknown src in  
213     let automaton = construit_automate_LR1 regex_grammar Regex_union Eof in  
214     let tree = parse_automaton Eof tokens Regex_union in  
215     regex_of_regex_syntax_tree tree
```

regex.ml

```

1  open Automaton
2
3  type 'symbol regex =
4    | Empty
5    | Epsilon
6    | Symbol of 'symbol
7    | Concatenation of 'symbol regex * 'symbol regex
8    | Union of 'symbol regex * 'symbol regex
9    | Star of 'symbol regex
10
11  (** `automaton_of_regex r` returns an automaton with epsilon-transitions that
12     recognizes the same language as `r` *)
13  let automaton_of_regex (r : 'symbol regex) : ('symbol, int) epsilon_automaton =
14    let rec automaton_of_regex_with_context (r : 'symbol regex)
15      (state_count : int) : ('symbol, int) epsilon_automaton * int =
16      match r with
17      | Empty ->
18        ( {
19          states = [];
20          initial_states = [];
21          final_states = [];
22          transitions = [];
23        },
24          state_count )
25      | Epsilon ->
26        ( {
27          states = [ state_count ];
28          initial_states = [ state_count ];
29          final_states = [ state_count ];
30          transitions = [];
31        },
32          state_count + 1 )
33      | Symbol s ->

```

5 – Annexes • 5.6 Code

```

34     ( {
35         states = [ state_count; state_count + 1 ];
36         initial_states = [ state_count ];
37         final_states = [ state_count + 1 ];
38         transitions = [ (state_count, Automaton.Symbol s, state_count + 1) ];
39     },
40     state_count + 2 )
41 | Concatenation (l, r) ->
42     let left_automaton, state_count =
43         automaton_of_regex_with_context l state_count
44     in
45     let right_automaton, state_count =
46         automaton_of_regex_with_context r state_count
47     in
48     let cross_transitions =
49         left_automaton.final_states
50         |> List.map (fun left_final_state ->
51             right_automaton.initial_states
52             |> List.map (fun right_initial_state ->
53                 ( left_final_state,
54                     Automaton.Epsilon,
55                     right_initial_state )))
56     |> List.flatten
57 in
58
59     ( {
60         states = left_automaton.states @ right_automaton.states;
61         initial_states = left_automaton.initial_states;
62         final_states = right_automaton.final_states;
63         transitions =
64             left_automaton.transitions @ right_automaton.transitions
65             @ cross_transitions;
66     },
67     state_count )

```

```

68 | Union (l, r) ->
69 |   let left_automaton, state_count =
70 |     automaton_of_regex_with_context l state_count
71 |   in
72 |   let right_automaton, state_count =
73 |     automaton_of_regex_with_context r state_count
74 |   in
75 |   ( {
76 |     states = left_automaton.states @ right_automaton.states;
77 |     initial_states =
78 |       left_automaton.initial_states @ right_automaton.initial_states;
79 |     final_states =
80 |       left_automaton.final_states @ right_automaton.final_states;
81 |     transitions =
82 |       left_automaton.transitions @ right_automaton.transitions;
83 |   },
84 |     state_count )
85 | Star r' ->
86 |   let inner_automaton, state_count =
87 |     automaton_of_regex_with_context r' state_count
88 |   in
89 |   let loopback_transitions =
90 |     inner_automaton.final_states
91 |     |> List.map (fun final_state ->
92 |       inner_automaton.initial_states
93 |       |> List.map (fun initial_state ->
94 |         (final_state, Automaton.Epsilon, initial_state)))
95 |     |> List.flatten
96 |   in
97 |   let epsilon_state = state_count in
98 |   ( {
99 |     states = epsilon_state :: inner_automaton.states;
100 |     initial_states = epsilon_state :: inner_automaton.initial_states;
101 |     final_states = epsilon_state :: inner_automaton.final_states;

```

```

102         transitions = inner_automaton.transitions @ loopback_transitions;
103     },
104     state_count + 1 )
105 in
106   fst (automaton_of_regex_with_context r 0)
107
108 let rec string_of_regex (string_of_symbol : 'a -> string) (r : 'a regex) =
109   match r with
110   | Empty -> "<empty>"
111   | Epsilon -> "<epsilon>"
112   | Concatenation (r1, r2) ->
113     string_of_regex string_of_symbol r1 ^ string_of_regex string_of_symbol r2
114   | Star r -> string_of_regex string_of_symbol r ^ "*"
115   | Union (r1, r2) ->
116     string_of_regex string_of_symbol r1
117     ^ "|"
118     ^ string_of_regex string_of_symbol r2
119   | Symbol s -> string_of_symbol s

```

utils.ml

```

1  let explode (s : string) : char list =
2      List.init (String.length s) (String.get s)
3
4  let implode (chars : char list) : string = String.of_seq (List.to_seq chars)
5  let string_of_char (c : char) : string = String.make 1 c
6
7  (* Renvoie la liste l sans ses i premiers éléments. Lève l'exception
8   * Failure "tropcourt" si la liste est trop courte et Failure "indiceneg"
9   * lorsque i est négatif. Cette fonction est récursive terminale. *)
10 let rec list_skip (l : 'a list) (i : int) : 'a list =
11     if i < 0 then failwith "indiceneg"
12     else if i = 0 then l
13     else match l with [] -> failwith "tropcourt" | x :: q -> list_skip q (i - 1)
14
15 (* Renvoie les i premiers éléments de la liste l. Lève l'exception
16 * Failure "tropcourt" si la liste est trop courte et Failure "indiceneg"
17 * lorsque i est négatif. *)
18 let rec list_beginning (l : 'a list) (i : int) : 'a list =
19     if i < 0 then failwith "indiceneg"
20     else if i = 0 then []
21     else
22         match l with
23         | [] -> failwith "tropcourt"
24         | x :: q -> x :: list_beginning q (i - 1)
25
26 (* Renvoie true si la liste a une taille égale à 1, false sinon. Cette implem
27 * est en O(1) *)
28 let is_length_1 (l : 'a list) : bool =
29     match l with
30     | [_] -> true
31     | _ -> false
32
33 (* Affiche une liste *)

```


5 – Annexes • 5.6 Code

```

34 let print_list (l: 'a list) (print_: 'a -> unit) =
35   let rec print_list_inner (l': 'a list) =
36     match l' with
37     | [] -> ()
38     | [x] -> print_ 'a x
39     | x::q -> (print_ 'a x; print_string "; "; print_list_inner q)
40   in
41   print_string "[";
42   print_list_inner l;
43   print_endline "]"
44
45   (* Dépile n éléments de la pile s et les renvoie dans une liste. Le i-ième
46    * élément de la liste est le i-ième à avoir été dépilé. Si jamais on
47    * ne peut pas dépiler, lève l'erreur Stack.Empty *)
48   let pop_n (s : 'a Stack.t) (n : int) : 'a list =
49     List.init n (fun _ -> Stack.pop s)
50
51   let rec seq_find (f : 'a -> bool) (s : 'a Seq.t) : 'a option =
52     match s () with
53     | Seq.Nil -> None
54     | Seq.Cons (e, t) -> if f e then Some e else seq_find f t
55
56   (* Fonctions pour gérer correctement des Uchar.t *)
57   type uchar = Uchar.t
58   let u = Uchar.of_char
59
60   let uchar_list_of_string (s: string) : uchar list =
61     let n = String.length s in
62     (* Fais uchar_list_of_string à partir de l'indice i. Concatène le résultat
63      * après List.rev accu *)
64     let rec aux (i: int) (accu: uchar list) =
65       if i = n then
66         List.rev accu
67       else

```

5 – Annexes • 5.6 Code

```

68     let utf_decode_next_uchar = String.get_utf_8_uchar s i in
69     (* Récupère la longueur en char de ce utf_decode ~ *)
70     let k = Uchar.utf_decode_length utf_decode_next_uchar in
71     (* Récupère le uchar de ce utf_decode *)
72     let next_char = Uchar.utf_decode_uchar utf_decode_next_uchar in
73     (* Note : si le caractère était invalide, on a next_char = U+FFFD et
74      * k = 1 *)
75     aux (i+k) (next_char::accu)
76 in
77 aux 0 []
78
79 let explode_uchar = uchar_list_of_string
80
81 let unicode_aware_length (s: string) : int =
82     s |> explode_uchar |> List.length
83
84
85 (* Cette fonction NE renvoie PAS Uchar.to_int u. En réalité, elle renvoie la
86  * représentation interne de u (i.e. les 1, 2, 3 ou 4 octets UTF-8) *)
87 let int_list_of_uchar (u: uchar) : int list =
88     let codepoint = Uchar.to_int u in
89     if codepoint <= 0x7F then
90         [codepoint]
91     else if codepoint <= 0x7FF then
92         (* bbb bbbb bbbb -> 110. .... 10.. .... *)
93         let first_5_bits = codepoint lsr 6 in
94         let last_6_bits = codepoint mod 64 in
95         [0b1100_0000 lor first_5_bits; 0b1000_0000 lor last_6_bits]
96     else if codepoint <= 0xFFFF then
97         (* bbbb bbbb bbbb bbbb -> 1110 .... 10.. .... 10.. .... *)
98         let first_4_bits = codepoint lsr 12 in
99         let next_6_bits = (codepoint lsr 6) mod 64 in
100        let last_6_bits = codepoint mod 64 in
101        [

```

5 – Annexes • 5.6 Code

```

102         0b1110_0000 lor first_4_bits;
103         0b1000_0000 lor next_6_bits;
104         0b1000_0000 lor last_6_bits
105     ]
106     else if codepoint <= 0xFFFFF then
107         (* bbbb bbbb bbbb bbbb bbbb bbbb -> 1111 0... 10.. .... 10.. .... 10.. ....
108            *)
109         let first_3_bits = codepoint lsr 18 in
110         let next_6_bits = (codepoint lsr 12) mod 64 in
111         let next_next_6_bits = (codepoint lsr 6) mod 64 in
112         let last_6_bits = codepoint mod 64 in
113         [
114             0b1111_0000 lor first_3_bits;
115             0b1000_0000 lor next_6_bits;
116             0b1000_0000 lor next_next_6_bits;
117             0b1000_0000 lor last_6_bits
118         ]
119     else
120         failwith (
121             "int_of_char: " ^
122             (string_of_int codepoint) ^
123             " n'est pas un caractère Unicode valide..."
124         )
125
126
127     let char_list_of_uchar (u: uchar) : char list =
128         List.map char_of_int (int_list_of_uchar u)
129
130     let string_of_uchar (u: uchar) : string =
131         implode (char_list_of_uchar u)
132
133     let implode_uchar (l: uchar list) : string =
134         let char_list_list = List.map char_list_of_uchar l in
135         let char_list = List.concat char_list_list in

```

```

136     implode char_list
137
138
139 module Hashset = struct
140     type 'a t = { mutable data : ('a, unit) Hashtbl.t; mutable rw : bool }
141
142     let create () : 'a t = { data = Hashtbl.create 8; rw = true }
143     let mem (t : 'a t) a = Hashtbl.find_opt t.data a <> None
144
145     let add (t : 'a t) a =
146         if not t.rw then (
147             t.data <- Hashtbl.copy t.data;
148             t.rw <- true);
149
150         Hashtbl.replace t.data a ()
151
152     let mem_add (t : 'a t) a =
153         let added = not (mem t a) in
154         if added then add t a;
155         added
156
157     let remove (t : 'a t) a =
158         if not t.rw then (
159             t.data <- Hashtbl.copy t.data;
160             t.rw <- true);
161         Hashtbl.remove t.data a
162
163     let length (t : 'a t) = Hashtbl.length t.data
164     let iter f (t : 'a t) = Hashtbl.iter (fun a _ -> f a) t.data
165     let is_empty (t : 'a t) = length t = 0
166
167     let remove_one (t : 'a t) =
168         match (Hashtbl.to_seq_keys t.data) () with
169         | Nil -> raise (Invalid_argument "Cannot remove from an empty set")

```

5 – Annexes • 5.6 Code

```

170 | Cons (key, _) ->
171 |   remove t key;
172 |   key
173
174 let union (t1 : 'a t) (t2 : 'a t) =
175 |   let res = { data = Hashtbl.create (length t1 + length t2); rw = true } in
176 |   iter (add res) t1;
177 |   iter (add res) t2;
178 |   res
179
180 let singleton (e : 'a) =
181 |   let res = create () in
182 |   add res e;
183 |   res
184
185 let intersection (t1 : 'a t) (t2 : 'a t) =
186 |   let res = create () in
187 |   iter (fun e -> if mem t2 e then add res e) t1;
188 |   res
189
190 let to_list (t : 'a t) =
191 |   let res = ref [] in
192 |   iter (fun e -> res := e :: !res) t;
193 |   !res
194
195 let of_list (l : 'a list) =
196 |   let res = create () in
197 |   List.iter (add res) l;
198 |   res
199
200 let equals (t1 : 'a t) (t2 : 'a t) =
201 |   t1.data == t2.data
202 |   || length t1 = length t2
203 |   && Hashtbl.to_seq t1.data |> Seq.for_all (fun (k, ()) -> mem t2 k)

```

5 – Annexes • 5.6 Code

```

204
205   let copy (t : 'a t) =
206     t.rw <- false;
207     { data = t.data; rw = false }
208 end
209
210 let hashtbl_remove_one (t : ('k, 'v) Hashtbl.t) =
211   match (Hashtbl.to_seq_keys t) () with
212   | Nil -> raise (Invalid_argument "Cannot remove from an empty set")
213   | Cons (key, _) ->
214     let value = Hashtbl.find t key in
215     Hashtbl.remove t key;
216     (key, value)
217
218 module TerminalSet = struct
219   let available_bits_per_int = 31
220
221   type 'a mapping = {
222     direct : ('a, int) Hashtbl.t;
223     reverse : 'a array;
224     item_count : int;
225   }
226
227   type 'a t = { mapping : 'a mapping; mutable length : int; data : int array }
228
229   let build_mapping (token_types : 'a list) : 'a mapping =
230     let direct = Hashtbl.create 8 in
231     List.iter
232       (fun token_type ->
233         if not (Hashtbl.mem direct token_type) then
234           Hashtbl.add direct token_type (Hashtbl.length direct))
235       token_types;
236
237   let reverse =

```

```

238     Array.init (Hashtbl.length direct) (fun _ -> List.hd token_types)
239   in
240
241   Hashtbl.iter (fun k v -> reverse.(v) <- k) direct;
242
243   { direct; item_count = Hashtbl.length direct; reverse }
244
245   let create (mapping : 'a mapping) =
246     let data_length =
247       int_of_float
248         (ceil
249           (float_of_int mapping.item_count
250             /. float_of_int available_bits_per_int))
251     in
252     { mapping; length = 0; data = Array.make data_length 0 }
253
254   let add (s : 'a t) (item : 'a) : unit =
255     let i = Hashtbl.find s.mapping.direct item in
256     let prev = s.data.(i / available_bits_per_int) in
257     s.data.(i / available_bits_per_int) <-
258       s.data.(i / available_bits_per_int)
259       lor (1 lsl (i mod available_bits_per_int));
260     if prev <> s.data.(i / available_bits_per_int) then s.length <- s.length + 1
261
262   let length (s : 'a t) = s.length
263
264   let iter (f : 'a -> unit) (s : 'a t) =
265     for i = 0 to Array.length s.data - 1 do
266       if s.data.(i) <> 0 then
267         for j = 0 to available_bits_per_int - 1 do
268           if s.data.(i) land (1 lsl j) <> 0 then
269             f s.mapping.reverse.(j + (i * available_bits_per_int))
270         done
271     done

```

```

272
273 let copy (s : 'a t) =
274   { mapping = s.mapping; length = s.length; data = Array.copy s.data }
275
276 let equals (s1 : 'a t) (s2 : 'a t) =
277   if s1.length != s2.length || s1.mapping != s2.mapping then false
278   else
279     let equal_so_far = ref true in
280     let i = ref 0 in
281     while !equal_so_far && !i < Array.length s1.data do
282       equal_so_far := s1.data.[!i] = s2.data.[!i];
283       incr i
284     done;
285     !equal_so_far
286
287 let singleton (mapping : 'a mapping) (item : 'a) : 'a t =
288   let res = create mapping in
289   add res item;
290   res
291
292 let is_empty (s : 'a t) = s.length = 0
293
294 let intersection (s1 : 'a t) (s2 : 'a t) : 'a t =
295   let res =
296     {
297       mapping = s1.mapping;
298       data =
299         Array.init (Array.length s1.data) (fun i ->
300           s1.data.[i] land s2.data.[i]);
301       length = 0;
302     }
303   in
304   iter (fun _ -> res.length <- res.length + 1) res;
305   res

```



```
306
307   let mem (s : 'a t) (item : 'a) =
308     let idx = Hashtbl.find_opt s.mapping.direct item in
309     match idx with
310     | None -> false
311     | Some idx ->
312       s.data.(idx / available_bits_per_int)
313       land (1 lsl (idx mod available_bits_per_int))
314       <> 0
315
316   let to_seq (s : 'a t) : 'a Seq.t =
317     let res = ref (fun () -> Seq.Nil) in
318     iter
319       (fun item ->
320         let res' = !res in
321         res := fun () -> Seq.Cons (item, res'))
322     s;
323     !res
324   end
```

`while_cli.ml`

```
1 | open Caml_light
2 | open While
3 |
4 | let main () =
5 |   let test_source =
6 |     if Array.length Sys.argv <= 1 then
7 |       failwith "Merci de donner un argument : le nom de fichier"
8 |     else begin
9 |       let test_source_fp = open_in Sys.argv.(1) in
10 |       let test_source =
11 |         really_input_string test_source_fp (in_channel_length test_source_fp)
12 |       in
13 |       close_in test_source_fp;
14 |       test_source
15 |     end
16 |   in
17 |   let program = parse_caml_light_ast test_source in
18 |   whilify_program program |> string_of_ast
19 |
20 |
21 | let () = print_endline (main ())
22 |
```

while.ml

```

1  open Utils
2  open Caml_light
3  open Rectify
4  open Rectify_helper
5
6  (* Renvoie l'AST de la fonction récursive terminale f une fois avoir été
7   * transformée en boucle while (la sortie est toujours l'AST d'une fonction).
8   * Les appels récursifs sont supposés être faits sur des noms de variables
9   * uniquement (i.e. la fonction appelée ne doit pas être le résultat d'une
10  * expression) et la liste de variables pouvant conduire à un appel récursif
11  * sont listées dans `recursive_call_names`.
12  *)
13  let fonction_vers_while
14    (name: variable)
15    (parameters: pattern list)
16    (body: expression)
17    (recursive_call_names: variable list) : expression =
18
19    (* Renvoie une liste de définitions de refs correspondant aux arguments *)
20    let rec parameter_list_to_ref_list (p: string list) (inner_expr: expression) :
21      expression =
22      match p with
23      | [] ->
24        (LetBinding ({
25          bindings = [
26            Variable {
27              lhs = Ident "res_ref";
28              value = FunctionApplication {
29                receiver = Variable "ref";
30                arguments = [Variable "None"]
31              }
32            }
33          ]
34        }

```

5 – Annexes • 5.6 Code

```

34         is_rec = false;
35         inner = inner_expr
36     )))
37 | x::q ->
38     parameter_list_to_ref_list q
39     (LetBinding {
40         bindings = [
41             Variable {
42                 lhs = Ident (x^"_ref");
43                 value = FunctionApplication {
44                     receiver = Variable "ref";
45                     arguments = [Variable x]
46                 }
47             }
48         ];
49         is_rec = false;
50         inner = inner_expr
51     })
52 in
53
54 (* res_ref := Some(inner) *)
55 let modify_res (inner: expression) : expression =
56     ReferenceAssignment {
57         receiver = Variable "res_ref";
58         value =
59             Parenthesised {
60                 style = Parenthesis;
61                 inner = FunctionApplication {
62                     receiver = Variable "Some";
63                     arguments = [Parenthesised{style=Parenthesis; inner}]
64                 }
65             }
66     }
67 in

```

5 – Annexes • 5.6 Code

```

68
69  (* Renvoie une liste de définitions de variables temporaires correspondant aux
70   * anciennes valeurs des arguments *)
71  let rec parameter_list_to_let_list (p: string list) (inner_expr: expression) :
72    expression =
73    match p with
74    | [] -> inner_expr
75    | x::q ->
76      parameter_list_to_let_list q
77      (LetBinding {
78        bindings = [
79          Variable {
80            lhs = Ident x;
81            value = Dereference(Variable(x^"_ref"))
82          }
83        ];
84        is_rec = false;
85        inner = inner_expr
86      })
87  in
88
89  (* modifie `res_ref` quand on trouve la valeur de l'expression *)
90  let rec replace_result (p: string list) (inner_expr: expression) :
91    expression =
92    match inner_expr with
93    | Parenthesised { inner ; style } ->
94      Parenthesised {
95        inner = replace_result p inner ;
96        style = style
97      }
98    | TypeCoercion {inner ; typ } ->
99      TypeCoercion {
100        inner = replace_result p inner ;
101        typ = typ

```

```

102     }
103   | FunctionApplication { receiver ; arguments } ->
104     begin match receiver with
105     | Parenthesised { inner=Variable(rec_call_name) ; style=_ }
106     | Variable(rec_call_name)
107       when List.mem rec_call_name recursive_call_names ->
108       let seq = Caml_light.Sequence (
109         List.map
110           (fun (nom, valeur) ->
111             Caml_light.ReferenceAssignment {
112               receiver = Variable(nom^"_ref");
113               value = valeur
114             }
115           )
116         (List.combine p arguments)
117       )
118       in
119       Parenthesised {
120         style = BeginEnd;
121         inner = seq
122       }
123   | _ ->
124     modify_res (FunctionApplication { receiver ; arguments })
125   end
126 | If { condition ; body ; else_body } ->
127   If {
128     condition = condition ;
129     body = replace_result p body ;
130     else_body =
131       Option.map (replace_result p) else_body
132   }
133 | Sequence(l) ->
134   let n = List.length l in
135   Sequence (

```

5 – Annexes • 5.6 Code

```

136         List.mapi (fun i x -> if i = (n-1) then replace_result p x else x) 1
137     )
138 | Match { value ; cases } ->
139     Match {
140         value ;
141         cases = List.map
142             (fun (pattern, expr) -> (pattern, replace_result p expr))
143             cases
144     }
145 | Try { value ; cases } ->
146     Try {
147         value ;
148         cases = List.map
149             (fun (pattern, expr) -> (pattern, replace_result p expr))
150             cases
151     }
152 | FunctionLiteral f ->
153     modify_res (FunctionLiteral f)
154 | LetBinding {
155     bindings : binding list;
156     is_rec : bool;
157     inner : expression;
158 } ->
159     LetBinding {
160         bindings = List.map
161             (fun (b: binding) ->
162                 match b with
163                 | Variable v -> (Variable v: binding)
164                 | Function { name ; parameters ; body ; return_type } ->
165                     (* TODO à repenser *)
166                     Function {
167                         name ;
168                         parameters ;
169                         body ;

```

5 – Annexes • 5.6 Code

```

170         return_type
171     }
172     ) bindings ;
173     is_rec ;
174     inner = replace_result p inner
175 }
176 | v -> modify_res v
177 in
178
179 let wrap_in_while (body: expression) : expression =
180   Sequence [
181     WhileLoop {
182       condition = InfixOperation {
183         lhs = Dereference(Variable "res_ref");
184         rhs = Variable "None";
185         operation = Eq
186       };
187       body;
188     };
189     FunctionApplication {
190       receiver = FieldAccess { receiver = Variable "Option"; target = "get"};
191       arguments = [
192         Parenthesised {
193           style = Parenthesis;
194           inner = Dereference(Variable "res_ref")
195         }
196       ]
197     }
198   ]
199 in
200
201 let params = List.filter_map
202   (fun (x: pattern) -> match x with
203     | (TypeCoercion {inner = Ident name; _})

```


5 – Annexes • 5.6 Code

```

204 | Ident name -> Some name
205 | _ -> None
206 ) parameters in
207 replace_result params body
208 |> parameter_list_to_let_list params
209 |> wrap_in_while
210 |> parameter_list_to_ref_list params
211
212
213 let parse_file name =
214   let input_channel = open_in name in
215   let content = really_input_string input_channel
216     (in_channel_length input_channel) in
217   close_in input_channel;
218   parse_caml_light_ast content
219
220
221 let whilify_bindings (def : value_definition) : value_definition list =
222   (* Renvoie les nouvelles value_definition, et le renommage appliqué aux
223    * fonctions modifiées *)
224   let whilify_bindings_after_rectification (def : value_definition)
225     (info: recursive_call_info) :
226     value_definition list * ((label * label) list) =
227     if
228       not def.is_rec
229       || not info.all_recursive_functions_are_toplevel
230       || not (is_length_1 info.recursively_defined_functions)
231     then
232       [ def ], []
233     else begin
234       match List.hd def.bindings with
235       | Function { name; parameters; return_type; body } ->
236         let new_name = name ^ "_whilified" in
237         let body_while = fonction_vers_while new_name parameters

```

```

238             body info.rectifying_set
239         in
240         [
241             {
242                 bindings = [
243                     Function {
244                         name = new_name;
245                         parameters;
246                         body = body_while;
247                         return_type;
248                     }
249                 ];
250                 is_rec = false
251             };
252             ], [(name, new_name)]
253         | _ -> [ def ], []
254     end
255 in
256
257 let rectify_then_whileify (def : value_definition) : rectify_result =
258     match rectify_bindings def.bindings with
259     | NewBindings (b, Some info) ->
260         let update_bindings (def : value_definition)
261             (substitutions : (label * label) list) :
262             value_definition list * ((label * label) list) =
263             match def with
264             | {
265                 is_rec = false;
266                 bindings = [
267                     Function (
268                         {
269                             body = FunctionApplication {
270                                 receiver = Variable callee;
271                                 arguments

```

5 – Annexes • 5.6 Code

```

272         }
273     } as f
274 )
275 ]
276 } ->
277 (* C'est un intercepteur créé pendant la rectification : on le met à
278 * jour pour appelé la fonction rectifiée-whilifiée *)
279 (
280     [{
281         is_rec = false;
282         bindings = [
283             Function {
284                 f with
285                 body = FunctionApplication {
286                     receiver = Variable (
287                         List.assoc_opt callee substitutions
288                         |> Option.value ~default:callee
289                     );
290                     arguments;
291                 };
292             }
293         ];
294     }],
295     []
296 )
297 | _ -> whilify_bindings_after_rectification def info
298 in
299
300 NewBindings (
301     List.fold_left (fun (defs, subs) next_def ->
302         let next_defs, next_subs = update_bindings next_def subs in
303         defs @ next_defs, subs @ next_subs
304     ) ([], []) b
305 |> fst,

```

```
306         None
307     )
308     | autre -> autre
309 in
310
311 try_transform_bindings_deep def rectify_then_whilify
312
313
314 let whilify_program (program: phrase list) =
315   let whilify_phrase (phrase: phrase) : phrase list =
316     match phrase with
317     | ValueDefinition v -> begin
318       whilify_bindings v
319       |> List.map (fun v -> ValueDefinition v)
320     end
321     | _ -> [phrase]
322 in
323 program |> List.map whilify_phrase |> List.concat
324
325
```

Fichier de tests

```

1  (* Commentaire *)
2
3  let rec fizzbuzz_a_partir (max: int) (n: int) : unit =
4      if n > max then
5          ()
6      else begin
7          if n mod 15 = 0 then
8              print_endline "fizzbuzz"
9          else if n mod 5 = 0 then
10             print_endline "buzz"
11          else if n mod 3 = 0 then
12             print_endline "fizz"
13          else
14             (print_int n; print_newline ())
15          ;
16          fizzbuzz_a_partir max (n+1)
17      end
18
19
20  let fizzbuzz (max: int) : unit = fizzbuzz_a_partir max 1
21
22  let rec fibo_jusqua (n: int) (i: int) (last: int) (lastlast: int) =
23      if n <= 1 then
24          1
25      else if i = n then
26          last+lastlast
27      else
28          fibo_jusqua n (i+1) (lastlast) (last+lastlast)
29
30  let fibo n = fibo_jusqua n 2 1 1
31
32  let rec map (f: 'a -> 'b) (l: 'a list) : 'b list =
33      match l with

```

5 – Annexes • 5.6 Code

```

34 | [] -> []
35 | x::q -> (f x)::(map f q)
36
37
38 let rec sum (l : int list) : int =
39     match l with
40     | [] -> 0
41     | x :: q -> x + sum q
42
43
44 type 'a abr =
45     Feuille of 'a
46     | Noeud of 'a * 'a abr * 'a abr
47
48
49 let rec recherche_abr (abr: 'a abr) (valeur: 'a) : bool =
50     match abr with
51     | Feuille a -> a = valeur
52     | Noeud(t, g, d) ->
53         begin
54             if valeur <= t then
55                 recherche_abr g valeur
56             else
57                 recherche_abr d valeur
58             end
59
60 let rec insert_abr (abr: 'a abr) (valeur: 'a) : 'a abr =
61     match abr with
62     | Feuille a ->
63         begin
64             if valeur < a then Noeud(valeur, Feuille(valeur), Feuille(a))
65             else if valeur > a then Noeud(a, Feuille(a), Feuille(valeur))
66             else Feuille a
67             end

```

```
68 | Noeud(t, g, d) ->  
69 |   begin  
70 |     if valeur <= t then  
71 |       Noeud(t, insert_abr g valeur, d)  
72 |     else  
73 |       Noeud(t, g, insert_abr d valeur)  
74 |     end
```

Fichier de tests après transformation en boucles

```

1  let fizzbuzz_a_partir_rectified_whilified (max : int) (n : int) (acc : unit) :
2      unit =
3      let res_ref = ref None in
4      let acc_ref = ref acc in
5      let n_ref = ref n in
6      let max_ref = ref max in
7      while !res_ref = None do
8          let acc = !acc_ref in
9          let n = !n_ref in
10         let max = !max_ref in
11         if n > max then res_ref := Some acc
12         else (
13             if n mod 15 = 0 then print_endline "fizzbuzz"
14             else if n mod 5 = 0 then print_endline "buzz"
15             else if n mod 3 = 0 then print_endline "fizz"
16             else (
17                 print_int n;
18                 print_newline ();
19                 begin
20                     max_ref := max;
21                     n_ref := n + 1;
22                     acc_ref := acc
23                 end)
24         done;
25         Option.get !res_ref
26
27 let fizzbuzz_a_partir (max : int) (n : int) : unit =
28     fizzbuzz_a_partir_rectified_whilified max n ()
29
30 let fizzbuzz (max : int) : unit = fizzbuzz_a_partir max 1
31
32 let fibo_jusqua_rectified_whilified (n : int) (i : int) (last : int)
33     (lastlast : int) cont =

```



```

34   let res_ref = ref None in
35   let cont_ref = ref cont in
36   let lastlast_ref = ref lastlast in
37   let last_ref = ref last in
38   let i_ref = ref i in
39   let n_ref = ref n in
40   while !res_ref = None do
41     let cont = !cont_ref in
42     let lastlast = !lastlast_ref in
43     let last = !last_ref in
44     let i = !i_ref in
45     let n = !n_ref in
46     if n <= 1 then res_ref := Some (cont 1)
47     else if i = n then res_ref := Some (cont (last + lastlast))
48     else begin
49       n_ref := n;
50       i_ref := i + 1;
51       last_ref := lastlast;
52       lastlast_ref := last + lastlast;
53       cont_ref := cont
54     end
55   done;
56   Option.get !res_ref
57
58   let fibo_jusqua (n : int) (i : int) (last : int) (lastlast : int) =
59     fibo_jusqua_rectified_whilified n i last lastlast (fun res -> res)
60
61   let fibo n = fibo_jusqua n 2 1 1
62
63   let map_rectified_whilified (f : 'a -> 'b) (l : 'a list)
64     (cont : 'b list -> 'ret) : 'ret =
65     let res_ref = ref None in
66     let cont_ref = ref cont in
67     let l_ref = ref l in

```

5 – Annexes • 5.6 Code

```

68   let f_ref = ref f in
69   while !res_ref = None do
70     let cont = !cont_ref in
71     let l = !l_ref in
72     let f = !f_ref in
73     match l with
74     | [] -> res_ref := Some (cont [])
75     | x :: q ->
76       let new_cont (a_12 : 'b list) : 'ret = cont (f x :: a_12) in
77       begin
78         f_ref := f;
79         l_ref := q;
80         cont_ref := new_cont
81       end
82   done;
83   Option.get !res_ref
84
85   let map (f : 'a -> 'b) (l : 'a list) : 'b list =
86     map_rectified_whilified f l (fun res -> res)
87
88   let sum_rectified_whilified (l : int list) (acc : int) : int =
89     let res_ref = ref None in
90     let acc_ref = ref acc in
91     let l_ref = ref l in
92     while !res_ref = None do
93       let acc = !acc_ref in
94       let l = !l_ref in
95       match l with
96       | [] -> res_ref := Some acc
97       | x :: q ->
98         let new_acc (a_7 : int) : int = x + a_7 in
99         begin
100           l_ref := q;
101           acc_ref := new_acc acc

```

5 – Annexes • 5.6 Code

```

102         end
103     done;
104     Option.get !res_ref
105
106 let sum (l : int list) : int = sum_rectified_whilified l 0
107
108 type 'a abr = Feuille of 'a | Noeud of 'a * 'a abr * 'a abr
109
110 let recherche_abr_rectified_whilified (abr : 'a abr) (valeur : 'a)
111     (cont : bool -> 'ret) : 'ret =
112     let res_ref = ref None in
113     let cont_ref = ref cont in
114     let valeur_ref = ref valeur in
115     let abr_ref = ref abr in
116     while !res_ref = None do
117         let cont = !cont_ref in
118         let valeur = !valeur_ref in
119         let abr = !abr_ref in
120         match abr with
121         | Feuille a -> res_ref := Some (cont (a = valeur))
122         | Noeud (t, g, d) ->
123             if valeur <= t then begin
124                 abr_ref := g;
125                 valeur_ref := valeur;
126                 cont_ref := cont
127             end
128             else begin
129                 abr_ref := d;
130                 valeur_ref := valeur;
131                 cont_ref := cont
132             end
133     done;
134     Option.get !res_ref
135

```

5 – Annexes • 5.6 Code

```

136 let recherche_abr (abr : 'a abr) (valeur : 'a) : bool =
137     recherche_abr_rectified_whilified abr valeur (fun res -> res)
138
139 let insert_abr_rectified_whilified (abr : 'a abr) (valeur : 'a)
140     (cont : 'a abr -> 'ret) : 'ret =
141     let res_ref = ref None in
142     let cont_ref = ref cont in
143     let valeur_ref = ref valeur in
144     let abr_ref = ref abr in
145     while !res_ref = None do
146         let cont = !cont_ref in
147         let valeur = !valeur_ref in
148         let abr = !abr_ref in
149         match abr with
150         | Feuille a ->
151             res_ref :=
152                 Some
153                     (cont
154                         (if valeur < a then Noeud (valeur, Feuille valeur, Feuille a)
155                             else if valeur > a then Noeud (a, Feuille a, Feuille valeur)
156                             else Feuille a))
157         | Noeud (t, g, d) ->
158             if valeur <= t then
159                 let new_cont (a_74 : 'a abr) : 'ret = cont (Noeud (t, a_74, d)) in
160                 begin
161                     abr_ref := g;
162                     valeur_ref := valeur;
163                     cont_ref := new_cont
164                 end
165             else
166                 let new_cont (a_91 : 'a abr) : 'ret = cont (Noeud (t, g, a_91)) in
167                 begin
168                     abr_ref := d;
169                     valeur_ref := valeur;

```

```
170         cont_ref := new_cont
171     end
172 done;
173 Option.get !res_ref
174
175 let insert_abr (abr : 'a abr) (valeur : 'a) : 'a abr =
176     insert_abr_rectified_whilified abr valeur (fun res -> res)
```